

Yamba — Documentation d'utilisation de l'API de bout en bout

Yamba — documentation générée le 06/09/2026 depuis `docs/livrables/03-YAMBA-DOCUMENTATION-API.md`

Table des matières

Sommaire

1. Vue d'ensemble

- 1.1 Une seule porte d'entrée : le gateway
- 1.2 Table de routage : préfixe → service → chemin réécrit
- 1.3 Les cinq services et leurs ports
- 1.4 Les documents OpenAPI vivants
- 1.5 Comment lire la suite

2. Conventions transverses

- 2.1 Authentification d'un membre : cookies, jetons, rotation
- 2.2 Authentification du back-office : cookies admin_* et TOTP
- 2.3 La fenêtre sudo : 403 SUDO_REQUIRED et rejeu
- 2.4 Locale : x-locale et preferredLocale
- 2.5 Identifiant de corrélation
- 2.6 Format des erreurs : quatre formes, une seule à traduire
- 2.7 Sémantique des statuts
- 2.8 Limiteur de débit
- 2.9 Mode maintenance (503 MAINTENANCE)
- 2.10 Pagination par curseur
- 2.11 Montants, devises, dates, identifiants
- 2.12 Idempotence et rejeu
- 2.13 CORS, même origine et test en réseau local

3. Guides pas à pas

- 3.1 Inscription → vérification OTP → connexion → /auth/me
- 3.2 Devenir Voyageur : profil, Stripe Connect, activation
- 3.3 Publier un trajet
- 3.4 Rechercher, lire un trajet public, le mettre en favori
- 3.5 Réserver et vivre le deal : de l'intention de paiement à la notation
- 3.6 Messagerie : fil, rendez-vous, numéro
- 3.7 Lien de suivi pour le destinataire (sans compte)
- 3.8 Signaler un trajet ou un membre
- 3.9 Alertes de route (saved-routes)
- 3.10 Profil et avatar (téléversement ImageKit signé)
- 3.11 RGPD : export et effacement (sous sudo)
- 3.12 Session admin : connexion → TOTP → KPIs → un geste journalisé

4. Catalogue des codes d'erreur métier

- 4.1 Demande de réservation (details.type: "booking", 409, deal-service)
- 4.2 Cycle de vie, transport, règlement (details.type: "booking", 409, deal-service)
- 4.3 Compte, session, sudo (auth-service)
- 4.4 Trajets et favoris (trip-service)
- 4.5 Messagerie (message-service)
- 4.6 Gateway et back-office

5. Webhooks entrants et événements sortants

- 5.1 Webhook Stripe — POST :6003/webhooks/stripe (direct, jamais via le gateway)
- 5.2 Webhook email (Resend) — POST /api/webhooks/email/resend
- 5.3 Événements sortants : outbox, topics, garanties

6. Bonnes pratiques d'intégration

- 6.1 Écrire un client : jetons, rafraîchissement, 401 / 403, reprise après 503
- 6.2 Sécurité : ce qu'un client ne doit jamais faire
- 6.3 Versionnement et compatibilité

7. Référence exhaustive des endpoints

auth-service — port 6001

- auth-service › auth
- auth-service › me
- auth-service › reports
- auth-service › carrier
- auth-service › saved-routes
- auth-service › users
- auth-service › admin-auth
- auth-service › admin
- auth-service › schémas utilisés (129)

trip-service — port 6002

- trip-service › —
- trip-service › trips-search
- trip-service › trips-favorites
- trip-service › trips-public
- trip-service › trips
- trip-service › trips-lifecycle
- trip-service › trips-documents
- trip-service › uploads
- trip-service › admin
- trip-service › schémas utilisés (62)

deal-service — port 6003

- deal-service › system
- deal-service › deals
- deal-service › admin

- deal-service › me
- deal-service › schémas utilisés (114)

message-service — port 6005

- message-service › messages
- message-service › admin
- message-service › schémas utilisés (28)

notification-service — port 6004

- notification-service › —
- notification-service › schémas utilisés (10)

8. Annexes

8.1 Schémas de sécurité (tels que déclarés dans les documents OpenAPI)

8.2 Variables d'environnement côté client

8.3 Glossaire

8.4 Recommandations d'expert (hors périmètre livré)

Lot 3 des livrables. Ce document explique comment un client — le front web `user-ui`, le back-office `admin-ui`, une future application mobile (D36) ou un simple script `curl` — utilise l'API Yamba de bout en bout : par quelle porte entrer, comment s'authentifier, comment lire une erreur, et dans quel ordre appeler les routes pour vivre un parcours complet (inscription, publication d'un trajet, réservation, transport, remise, notation, messagerie, signalement, RGPD, back-office). La **référence exhaustive des 173 endpoints** (méthode, chemin, authentification, permission, paramètres, corps, réponses, schémas) est générée automatiquement depuis les cinq documents OpenAPI 3.1 des services et insérée au chapitre 7 ; les chapitres 1 à 6 et les annexes sont la partie narrative et pratique, écrite à la main à partir du code réel.

Conventions de lecture : les surfaces publiques de l'API (chemins, clés d'événements, codes d'erreur, messages) sont en **anglais**, la documentation est en **français**. Le symbole  signale une **porte** : un point où le comportement actuel est volontairement non figé, incomplet ou à durcir avant un usage donné (par exemple la mise en production ou le client mobile). Les identifiants de décisions (D2, D3, D36...) renvoient au registre `context/YAMBA-REGISTRE-DECISIONS-ROADMAP-v1.3.md` ; les identifiants A-xxx aux arbitrages de chantier.

Précédence en cas de divergence : le code et ses tests > le registre des décisions > ce document. Les exemples `curl` de ce document correspondent à des routes réelles montées dans les routeurs des services, vérifiées contre la référence générée.

Sommaire

1. [Vue d'ensemble : le gateway, les cinq services, les documents OpenAPI](#)
2. [Conventions transverses](#) — authentification membre et admin, fenêtre sudo, locale, corrélation, format des erreurs, sémantique des statuts, limiteur, maintenance, pagination, montants, dates, idempotence, CORS
3. [Guides pas à pas \(curl\)](#) — inscription → connexion ; devenir Voyageur ; publier ; rechercher ; réserver et vivre le deal ; messagerie ; lien de suivi ; signalement ; alertes de route ; profil et avatar ; RGPD ; session admin
4. [Catalogue des codes d'erreur métier](#)
5. [Webhooks entrants et événements sortants](#)
6. [Bonnes pratiques d'intégration, sécurité, compatibilité](#)
7. [Référence exhaustive des endpoints \(générée\)](#)
8. [Annexes](#) — schémas de sécurité, variables d'environnement côté client, glossaire, recommandations d'expert hors périmètre

1. Vue d'ensemble

1.1 Une seule porte d'entrée : le gateway

Toute requête d'un client passe par l'**API gateway** (`apps/api-gateway/src/main.ts`), un serveur Express qui écoute sur le **port 8080** et expose l'ensemble de la plateforme sous le préfixe `/api`. Le gateway ne contient aucune logique métier : il applique le CORS, pose un identifiant de corrélation, applique un limiteur de débit, refuse les écritures en mode maintenance, puis **proxifie** chaque requête vers le microservice propriétaire du chemin, en réécrivant le préfixe. Un client n'a donc jamais besoin de connaître les ports des services : il parle à `http://localhost:8080/api/...` en développement, et à l'URL du gateway de l'environnement en recette ou en production.

Trois adresses « système » vivent directement au gateway, hors proxy :

Route	Rôle	Particularités
GET <code>/api/status</code>	Sonde publique pour un moniteur externe (D70). Interroge le <code>/health</code> des cinq services (2 s chacun) et agrège : <code>ok</code> , <code>maintenance</code> , <code>degraded</code> , <code>down</code> .	Déclarée avant le limiteur, mise en cache 10 s. HTTP 200 pour <code>ok</code> / <code>maintenance</code> , 503 pour <code>degraded</code> / <code>down</code> . Le corps ne contient jamais d'URL interne ni d'erreur brute : <code>{ status, services: [{ name, reachable, status, ms }], at }</code> .
GET <code>/gateway-health</code>	Santé du gateway lui-même, même forme que les <code>/health</code> des services.	Sans préfixe <code>/api</code> . Le contrôle <code>maintenance</code> est <code>ok: false</code> quand le mode lecture seule est actif.
GET <code>/api/maintenance</code>	État public du mode maintenance : <code>{ enabled, message: { fr, en }, scheduledAt }</code> .	Sert les bandeaux des deux fronts ; exempté du blocage.

1.2 Table de routage : préfixe → service → chemin réécrit

L'ordre de déclaration compte : les préfixes les plus spécifiques sont déclarés avant le « catch-all » vers l'auth-service. La table ci-dessous est l'image exacte du fichier `main.ts` du gateway.

Préfixe côté client (<code>/api/...</code>)	Service cible	Chemin reçu par le service	Domaine
<code>/api/trips/*</code>	trip-service :6002	<code>/trips/*</code>	trajets, recherche, favoris, documents, paramètres de prix
<code>/api/uploads/*</code>	trip-service :6002	<code>/uploads/*</code>	signature ImageKit, suppression d'un fichier
<code>/api/deals/*</code>	deal-service :6003	<code>/deals/*</code>	demandes, transitions, transport, remise, litige, notation, lien de suivi
<code>/api/me/bookings/*</code>	deal-service :6003	<code>/me/bookings/*</code>	« Mes envois » (Expéditeur)
<code>/api/me/deals/*</code>	deal-service :6003	<code>/me/deals/*</code>	deals reçus (Voyageur, tous trajets)
<code>/api/me/wallet/*</code>	deal-service :6003	<code>/me/wallet/*</code>	portefeuille Voyageur et paiements Expéditeur

Préfixe côté client (/api/...)	Service cible	Chemin reçu par le service	Domaine
/api/track/*	deal-service : 6003	/track/*	page destinataire sans compte (D69)
/api/admin/disputes/*	deal-service : 6003	/admin/disputes/*	file de médiation
/api/admin/alerts	deal-service : 6003	/admin/alerts	alertes de seuil (D59)
/api/admin/finances/*	deal-service : 6003	/admin/finances/*	files d'exception, rapport, export
/api/admin/deals/*	deal-service : 6003	/admin/deals/*	fiche argent, rapprochement, rejou, chronologie
/api/admin/trips/*	trip-service : 6002	/admin/trips/*	trajets, masquage
/api/admin/tickets/*	trip-service : 6002	/admin/tickets/*	vérification des billets
/api/admin/conversations/*	message-service : 6005	/admin/conversations/*	lecture journalisée d'un fil, signalements de messages
/api/messages/*	message-service : 6005	/messages/*	conversations, rendez-vous, numéro
/api/me/notifications/*	notification-service : 6004	/me/notifications/*	notifications in-app
/api/webhooks/email/*	notification-service : 6004	/webhooks/email/*	webhook du fournisseur d'email (Resend)
tout le reste (/api/auth/*, /api/carrier/*, /api/saved-routes/*, /api/users/*, /api/me/following, /api/reports, /api/admin/* restant)	auth-service : 6001	chemin inchangé (/api/...)	comptes, sessions, profils, alertes de route, signalements, back-office transverse

Deux subtilités à retenir :

- **L'auth-service est le seul service qui reçoit le chemin complet**, préfixe /api inclus : ses routeurs sont montés sous app.use("/api", ...) . Les autres services reçoivent un chemin réécrit sans /api . Quand on lit la référence générée, les chemins sont exprimés **du point de vue du service** (/auth/login , /trips/search , /deals/{id}) : il suffit de préfixer par /api pour obtenir l'URL client.
- **Le webhook Stripe ne passe pas par le gateway**. Il est monté directement sur le deal-service (POST http://localhost:6003/webhooks/stripe) car la signature Stripe porte sur les octets bruts du corps, que le gateway re-séréalise (voir §5.1).

1.3 Les cinq services et leurs ports

Service	Port	Responsabilité	Particularités d'infrastructure
auth-service	6001	inscription par OTP email, connexion (mot de passe, Google), rotation des sessions, fenêtre sudo, profil et avatar, onboarding Voyageur + Stripe Connect, alertes de route, profils publics et abonnements, signalements, export / effacement RGPD, tout le back-office transverse (KPIs, pilotage, journal, paramètres, statut, maintenance, comptes admin, utilisateurs, sanctions)	Redis pour les OTP, les sessions (j t i) et les fenêtres sudo ; crons (rappels, expirations) ; emails par dictionnaires de locale
trip-service	6002	trajets (brouillon, publication, pause, annulation, archivage), recherche publique paginée, facettes, favoris, documents de trajet, signature d'upload ImageKit, paramètres de prix publics, back-office trajets et billets	machine à états du trajet ; portes de publication (onboarding, Stripe) ; compteurs de demande en Redis
deal-service	6003	le cœur transactionnel : intention de paiement, création de la demande, machine à états du deal (9 statuts), transport (récupération, jalons, code de livraison, remise), règlement (confirmation, versement), litiges, notation double-aveugle, lien de suivi destinataire, portefeuille, files financières admin	PaymentProvider (Stripe capture manuelle / Fake), outbox transactionnelle + relais Kafka (topic booking-events), crons (expiration 24 h, versements, rappels, alertes, rétention) ; webhook Stripe direct
notification-service	6004	boîte aux lettres : deux consommateurs Kafka (booking-events , messaging-events) qui matérialisent les notifications in-app et envoient les emails, webhook Resend, purge nocturne	dédoublonnage par eventId (modèle ConsumedEvent), offsets commités après traitement, analytics PostHog par projection en liste blanche
message-service	6005	coordination Expéditeur ↔ Voyageur (D61) : une conversation par deal ouverte à l'acceptation, rendez-vous comme objet métier, fil avec messages système, numéro révélé au plus tôt 2 h avant le rendez-vous, signalement d'un message, lecture admin journalisée	propre outbox + relais sur le topic messaging-events , relance email des non-lus, purge à un an

Les cinq services partagent **une seule base MongoDB** (schéma Prisma à la racine prisma/schema.prisma) et un Redis. Chacun expose GET /health (toujours HTTP 200, corps ok ou degraded , contrôles Mongo + Redis en 2 s), et chacun appelle initSentry("<service>") au démarrage : un 5xx est capturé, tagué du service et de l'identifiant de corrélation.

1.4 Les documents OpenAPI vivants

Chaque service **génère son document OpenAPI 3.1 au démarrage** à partir des schémas Zod du paquet @packages/api-contracts (décision D3 : le même objet Zod valide les requêtes et produit la spécification ; la documentation ne peut pas dériver du code). Les documents sont aussi versionnés dans le dépôt (apps/<service>/openapi.json) et la CI vérifie qu'ils sont identiques à ce que le code génère (generate + diff).

Service	Document vivant	Visionneuse	Version du document	Fichier versionné
auth-service	http://localhost:6001/openapi.json	http://localhost:6001/docs (Scalar)	1.0.0	apps/auth-service/openapi.json

Service	Document vivant	Visionneuse	Versión du document	Fichier versionné
trip-service	http://localhost:6002/openapi.json	http://localhost:6002/docs	0.3.0	apps/trip-service/openapi.json
deal-service	http://localhost:6003/openapi.json	http://localhost:6003/docs	0.1.0	apps/deal-service/openapi.json
notification-service	http://localhost:6004/openapi.json	http://localhost:6004/docs	0.1.0	apps/notification-service/openapi.json
message-service	http://localhost:6005/openapi.json	— (pas de route /docs ; ouvrir le JSON dans une visionneuse externe) 📄	1.0.0	apps/message-service/openapi.json

Les visionneuses Scalar sont servies depuis un CDN et lisent le document vivant : elles sont « incapables de mentir ». Un test de l'auth-service exige que **chaque route montée soit documentée et qu'aucune route documentée n'existe pas** (A145) ; les schémas de l'auth-service décrivent les contrôleurs historiques au réel mais **ne gardent pas encore l'entrée** en `safeParse` — c'est une porte 📄 explicitement gravée pour le chantier mobile.

Les numéros de version des documents sont **indépendants les uns des autres** et ne suivent pas (encore) une politique de compatibilité formelle ; voir §6.3.

1.5 Comment lire la suite

Le chapitre 2 pose les conventions valables partout ; il faut le lire une fois. Le chapitre 3 déroule les parcours avec des appels `curl` complets : chaque appel indique quel service répond réellement derrière le gateway, les erreurs possibles et la réaction attendue du client. Le chapitre 4 est le catalogue des codes métier à traduire côté client. Le chapitre 5 décrit ce qui entre par webhook et ce qui sort par Kafka. Le chapitre 6 s'adresse à qui écrit un client (mobile en particulier). Le chapitre 7 est la référence générée. Les annexes ferment le document.

2. Conventions transverses

2.1 Authentification d'un membre : cookies, jetons, rotation

Un membre (rôle `USER`, éventuellement `CARRIER`) s'authentifie auprès de l'auth-service, qui émet **deux JWT** :

- **access_token** — jeton d'accès signé avec `ACCESS_TOKEN_SECRET`, porteur de `{ id, roles }`, durée de vie **15 minutes**. C'est lui que vérifient les middlewares de toutes les routes protégées.
- **refresh_token** — jeton de rafraîchissement signé avec `REFRESH_TOKEN_SECRET`, porteur de `{ id, jti, rememberMe, createdAt }`. Le **jti** est l'identifiant unique de la session, enregistré dans Redis avec ses métadonnées (`device`, `ip`, `userAgent`, D65 2A).

Ils sont posés en **cookies** `httpOnly` par `POST /auth/login`, `POST /auth/register/verify`, `POST /auth/google` et `POST /auth/refresh` : `path=/`, `sameSite=lax` hors production, `sameSite=none + secure` en production. Le cookie `access_token` vit 15 minutes ; le cookie `refresh_token` est un **cookie de session** (disparaît à la fermeture du navigateur) sauf si la connexion a été faite avec `rememberMe: true`, auquel cas il vit **30 jours** (A62).

La politique de session côté serveur (D27, `session-policy.ts`) borne la vie réelle de la session, indépendamment du cookie : **standard = 60 min d'inactivité, 7 jours de vie absolue ; rememberMe = 7 jours d'inactivité, 30 jours de vie absolue**. Le TTL Redis de la clé de session est recalculé à chaque rotation comme `min(fenêtre d'inactivité, vie absolue restante depuis createdAt)` : la rotation ne peut jamais repousser la session au-delà du plafond.

Rotation. `POST /auth/refresh` lit le cookie `refresh_token`, vérifie sa signature, cherche le `jti` en Redis (clé absente = session expirée ou révoquée → 401), **révoque l'ancien jti** et émet une nouvelle paire avec un nouveau `jti` et le **même createdAt**. Le libellé d'appareil de la connexion est conservé. Un `refresh` ne renvoie rien d'autre qu'un `{ success, message }` : les nouveaux jetons sont dans les cookies de la réponse.

Révocation. `POST /auth/logout` supprime le `jti` courant et efface les cookies. `GET /auth/me/sessions` liste les sessions du membre (la courante est marquée `current: true`), `DELETE /auth/me/sessions/{jti}` en coupe une (la sienne = déconnexion), `DELETE /auth/me/sessions` coupe toutes les autres. Le changement de mot de passe et le changement d'email révoquent toutes les autres sessions. Une sanction `SUSPENDED` révoque les sessions et fait répondre 401 à chaque lecture.

Le mode Bearer (mobile, D36). Les middlewares `isAuthenticated` et `isOptionallyAuthenticated` acceptent indifféremment le cookie `access_token` **ou** un en-tête `Authorization: Bearer <access_token>` (le cookie a priorité s'il est présent). C'est la porte prévue pour un client natif qui n'a pas de jar de cookies. 📄 **Porte importante** : aujourd'hui, `POST /auth/login` et `POST /auth/refresh` ne renvoient les jetons **que dans des cookies** `Set-Cookie` ; le corps de la réponse ne contient pas les jetons, et `/auth/refresh` lit exclusivement le cookie `refresh_token`. Un client mobile devra donc, au choix, gérer un jar de cookies (ce que font les clients HTTP natifs d'iOS et d'Android) ou attendre la variante « jetons dans le corps » gravée pour le jalon 4 (D36 : « auth par tokens (refresh) — pas de cookies »). Un intégrateur `curl` utilise simplement un jar (`-c jar -b jar`), comme dans tous les exemples du chapitre 3.

Ce que le middleware vérifie à chaque requête protégée (`packages/middleware/isAuthenticated.ts`) : signature et expiration du JWT, existence du compte en base, `isDeleted` (→ 401 `ACCOUNT_DELETED`), `accountStatus === "SUSPENDED"` (→ 401 `ACCOUNT_SUSPENDED`). La requête est enrichie de `req.user` (le document `User` complet) et `req.roles`. Le corps des 401 de ce middleware est **plat** : `{ "message": "Unauthorized! Token missing." }`, éventuellement avec `code`.

Compte restreint. Le middleware `requireActiveAccount` est posé après `isAuthenticated` sur les seules routes de **création** : `POST /trips`, `POST /trips/{id}/publish`, `POST /deals/payment-intents`, `POST /deals`. Un compte `RESTRICTED` ou `SUSPENDED` y reçoit **403 { message, code: "ACCOUNT_RESTRICTED" }** ; ses deals en cours continuent (les autres routes ne sont pas bloquées). Les sanctions ne font jamais l'objet d'une écriture inter-services : elles n'agissent que par les lectures.

Authentification optionnelle. `GET /users/{slug}/public`, `GET /trips/search` et `GET /trips/{id}/public` passent par `isOptionallyAuthenticated` : sans jeton, ou avec un jeton expiré, la route répond en mode « non connecté » (jamais 401) ; avec un jeton valide, la réponse est personnalisée (`isFavorite`, état d'abonnement, `hidden: true` pour le propriétaire d'un profil masqué).

2.2 Authentification du back-office : cookies `admin_*` et TOTP

Les sessions admin sont **séparées** des sessions membre (D54) : autres cookies, autre préfixe Redis (`admin_jti:`), autres revendications JWT. Un compte ADMIN connecté comme membre n'ouvre **aucune** route `/admin/*`.

Le parcours est en deux étapes :

1. `POST /auth/admin/login { email, password }` — exige `User.adminRoles` non vide ; pose le cookie `admin_preauth` (5 minutes) et répond `{ next: "TOTP" }` si la 2FA est déjà active, `{ next: "SETUP" }` sinon. Un compte sans profil admin reçoit 401.
2. `POST /auth/admin/totp/verify { code }` (code TOTP à 6 chiffres ou code de secours) — ou, au premier accès, `POST /auth/admin/totp/setup` (renvoie `secret + otpauthUrl` à scanner) puis `POST /auth/admin/totp/enable { code }`. En cas de succès, pose `admin_access_token` (15 min) et `admin_refresh_token` ; le JWT porte `adm: true` et `amr: ["pwd", "totp"]`.

`isAdminAuthenticated` (`packages/middleware/isAdminAuthenticated.ts`) lit **uniquement** `admin_access_token` (ou un Bearer), exige `adm === true` et `"totp" ∈ amr`, puis vérifie en base : compte non supprimé, rôle ADMIN, 2FA active (`totpEnabledAt`). Jeton absent, expiré ou incomplet → **401** ; compte qui ne remplit plus les conditions → **403** « Access denied. ». Il charge `req.adminRoles` (profils cumulés, union des permissions, D60 1A).

`requireAdminPermission("<permission>")` applique ensuite la **matrice ADMIN_PERMISSIONS** (`packages/libs/api-contracts/src/admin/admin-users.schema.ts`) : SUPER_ADMIN passe partout ; les autres profils (MEDIATOR, SUPPORT, FINANCE, OPS, PRIVACY) sont bornés permission par permission. Un refus est un **403 explicite** `{ message, code: "ADMIN_PERMISSION_DENIED", permission }` — jamais un 404 : la route existe, c'est le profil qui manque. Chaque route admin de la référence générée affiche sa permission (`x-permission`). Rotation et déconnexion : `POST /auth/admin/refresh`, `POST /auth/admin/logout` ; sessions : `GET /admin/me/sessions`, `DELETE /admin/me/sessions/{jti}`.

2.3 La fenêtre sudo : 403 SUDO_REQUIRED et rejeu

Les gestes sensibles d'un membre — changer son mot de passe (`POST /auth/me/password`), changer son email (`POST /auth/me/email/request`), ouvrir le tableau de bord Stripe (`POST /carrier/stripe/dashboard-link`), télécharger ses données (`POST /auth/me/data-export`), effacer son compte (`POST /auth/me/erasure`) — exigent une **fenêtre sudo** ouverte (D65 1A) :

1. `POST /auth/me/sudo/request` — envoie un code par email (portée OTP sudo).
2. `POST /auth/me/sudo/verify { code }` — ouvre une fenêtre de **15 minutes liée à la session courante** (`sudo:<userId>:<jti>` en Redis, le `jti` étant lu dans le cookie `refresh_token`). Une autre session du même membre n'en hérite pas. Réponse : `{ active: true, expiresAt }`.
3. `GET /auth/me/sudo` — état : `{ active, expiresAt | null }`.

Un geste sensible appelé sans fenêtre répond **403** via `requireSudo` (`apps/auth-service/src/utils/sudo.ts`) :

```
{ "status": "error", "message": "Sudo required: confirm with the code sent by email.", "details": { "code": "SUDO_REQUIRED", "windowMinutes": 15 } }
```

Le client ouvre alors la porte (demande + vérification du code), puis **rejoue exactement le même appel**. C'est ce que fait le composant `SudoGate` du front.

■ **Porte de durcissement** : le middleware d'erreurs n'expose `details` en production que si `details.type` appartient à la liste « safe » (§2.6) ; l'erreur `SUDO_REQUIRED` porte un `code` mais pas de `type`. Hors production le client reçoit bien `details.code` ; **avant la mise en production**, il faudra soit ajouter un `type` reconnu à cette erreur, soit étendre la liste — sans quoi le front ne pourrait plus distinguer ce 403 d'un autre. La même remarque vaut pour les codes `DELIVERY_CODE_IN_MESSAGE` et `T00_EARLY` du message-service (§4).

2.4 Locale : `x-locale` et `preferredLocale`

La liste des langues est unique (`packages/libs/api-contracts/src/locale.ts`) : `SUPPORTED_LOCALES = ["fr", "en"]`, défaut `fr`. Le client envoie la langue de son interface dans l'en-tête `x-locale` à chaque requête (le client axios du front le fait par un intercepteur ; `apiFetch` par `localeHeaders()`). `resolveLocale` est tolérant : `fr-FR`, `FR`, `en-US`, `en;q=0.9` sont ramenés à `fr / en` ; absent ou inconnu → `fr`.

Règle d'usage : **la langue d'un email est celle du destinataire** (`User.preferredLocale`), jamais celle de l'appelant. L'en-tête `x-locale` ne sert que pour les flux **sans compte** (OTP d'inscription, mot de passe oublié) et pour initialiser `preferredLocale` à l'inscription. Le membre change sa langue par `PATCH /auth/me/locale { locale }` (400 `LOCALE_UNSUPPORTED` sinon). La recherche accepte en plus un paramètre `locale` (`fr | en`) qui pilote le **formatage serveur des dates** dans les cartes de résultat.

2.5 Identifiant de corrélation

Le gateway pose l'en-tête `x-correlation-id` (UUID v4) sur toute requête qui n'en apporte pas ; `express-http-proxy` le transmet aux services. Le deal-service, le notification-service et le message-service le reprennent comme identifiant de requête (`pino-http`) et **le renvoient dans la réponse** ; il est copié dans chaque événement outbox (`correlationId`) et suit donc la transition jusqu'aux consommateurs Kafka, et jusqu'à Sentry en cas de 5xx. Un client qui veut corréler ses propres journaux avec le support peut fournir son propre identifiant ; il doit surtout **journaliser celui de la réponse** à côté de toute erreur remontée aux utilisateurs.

2.6 Format des erreurs : quatre formes, une seule à traduire

La plateforme a **quatre formes** d'erreur, toutes fidèles au code réel (`packages/error-handler, common.ts`) :

1. **ErrorResponse** — toute `AppError` sérialisée par `errorMiddleware` : `ValidationError` 400, `AuthError` 401, `ForbiddenError` 403, `NotFoundError` 404, `ConflictError` 409, `RateLimitError` 429, `DatabaseError` 500. `json { "status": "error", "message": "...", "errors": { "champ": "message" }, "details": { "type": "booking", "code": "QUOTE_DIVERGENCE", "...": "..." } }` - `errors` (facultatif) : erreurs par champ, exposé quand `details.errors` est présent — pour les formulaires. - `details` (facultatif) : contexte structuré. **En production, il n'est exposé que si `details.type` appartient à la liste safe** : `otp`, `booking`, `password`, `register`, `locale`, `favorite`, `oauth`. Hors production, tout `details` est exposé (débogage). Les traces de pile ne sortent jamais.

2. **UnhandledError** — exception hors `AppError` : { "status": "error", "error": "Something went wrong, please try again!" } (champ `error`, pas message). Capturée par Sentry.
3. **UnauthorizedResponse** — les 401 et 403 émis **directement** par les middlewares d'authentification, hors `errorMiddleware` : { "message": "..." }, parfois avec `code` (`ACCOUNT_DELETED`, `ACCOUNT_SUSPENDED`, `ACCOUNT_RESTRICTED`, `ADMIN_PERMISSION_DENIED` + `permission`).
4. **Réponses du gateway** — 429 du limiteur { "error": "Too many requests, please try again later!" } ; 503 de maintenance { "code": "MAINTENANCE", "message", "messages": { "fr", "en" }, "retryAfterSeconds": 300 }.

À cela s'ajoutent deux réponses typées renvoyées hors middleware par des contrôleurs : `PublicNotFound` (404 de `GET /trips/{id}/public`) et `ErasureBlockedResponse` (409 de `POST /auth/me/erasure` : { code: "ERASURE_BLOCKED", blockers, counts }).

Règle d'or pour un client : ne jamais afficher message tel quel. Les messages sont en anglais et techniques. Le client construit ses propres libellés, dans la langue de l'utilisateur, à partir du **statut HTTP**, de **details.code** (ou `code`) et des données structurées (`attemptsLeft`, `lockUntilSeconds`, `blockers`, `cap`, `limit`...). Le catalogue du chapitre 4 est fait pour cela.

⚠ **Note sémantique du trip-service** : ses contrôleurs historiques lèvent `ValidationError` (→ 400) pour presque tout, y compris « Trip not found. » et « Unauthorized. » (propriété). Un client du trip-service doit donc traiter un 400 comme « la demande est refusée » et lire `message` pour distinguer, en attendant la PR de bascule vers 403/404 (■ `fix/error-semantics`, cataloguée). Les autres services respectent la sémantique ci-dessous.

2.7 Sémantique des statuts

Statut	Sens dans Yamba	Réaction attendue du client
200 / 201	Succès. 201 pour une création (<code>POST /deals</code> , <code>POST /trips</code> , <code>POST /reports</code> , un message posté, une notation...).	Lire le corps ; pour un deal, se fier à <code>allowedActions</code> .
400	Requête malformée (Zod), <code>ObjectId</code> invalide, règle de forme (photo manquante, description trop courte), et — trip-service — toute erreur métier. Codes typés possibles : <code>OWN_TARGET</code> , <code>REASON_NOT_ALLOWED</code> , <code>DELIVERY_CODE_IN_MESSAGE</code> , <code>LOCALE_UNSUPPORTED</code> , <code>OTP (OTP_INCORRECT</code> ...).	Corriger la saisie ; afficher <code>errors</code> par champ ; ne pas réessayer à l'identique.
401	Pas de session valide : jeton absent, expiré, révoqué ; compte effacé ou suspendu ; OTP faux ou expiré ; identifiants invalides.	Si la requête exigeait une session : tenter un refresh puis rejouer ; si le refresh échoue, considérer l'utilisateur déconnecté (voir §6.1). Pour un OTP : proposer de redemander un code.
403	Authentifié mais interdit : pas partie au deal, pas propriétaire du trajet, compte restreint (<code>ACCOUNT_RESTRICTED</code>), fenêtre sudo fermée (<code>SUDO_REQUIRED</code>), profil admin sans permission, conversation inexistante avant l'acceptation.	Ne jamais réessayer aveuglément. <code>SUDO_REQUIRED</code> → ouvrir la porte puis rejouer ; sinon expliquer et rediriger.
404	Ressource inexistante, supprimée, invisible pour l'appelant (trajet masqué, profil masqué, lien de suivi révoqué). Un 404 ne dit jamais si la ressource existe pour quelqu'un d'autre.	Page « introuvable » ; retirer l'élément du cache local.
409	Conflit métier : la machine à états a refusé, une écriture concurrente a gagné, une limite est atteinte, un doublon existe. Presque toujours porteur d'un <code>details.code</code> (type: "booking").	Recharger la ressource (<code>GET /deals/{id}</code>), afficher le libellé du code, proposer l'action que <code>allowedActions</code> autorise désormais.
429	Trop de requêtes (limiteur du gateway) ou trop de tentatives OTP (<code>RateLimitError</code> , verrou par paliers).	Respecter <code>RateLimit-Reset</code> / <code>lockUntilSeconds</code> ; ne pas boucler.
501	Webhook Stripe non configuré (<code>STRIPE_WEBHOOK_SECRET</code> absent).	Configuration serveur, pas une erreur client.
503	Maintenance (écritures bloquées, code: "MAINTENANCE", <code>Retry-After: 300</code>) ; plateforme dégradée sur <code>/api/status</code> ; webhook email non configuré.	Afficher le bandeau (<code>messages.fr</code> / <code>.en</code>), garder la saisie, réessayer après <code>retryAfterSeconds</code> .

2.8 Limiteur de débit

Le gateway applique `express-rate-limit` **après** `/api/status` et avant tout proxy : fenêtre de **15 minutes, 100 requêtes** par adresse IP (`trust proxy` activé). Les réponses en échec ne sont pas comptées (`skipFailedRequests`), et les en-têtes standards `RateLimit-Limit`, `RateLimit-Remaining`, `RateLimit-Reset` sont renvoyés. Au-delà : 429 { "error": "Too many requests, please try again later!" }. ■ Le limiteur prévoit 1 000 requêtes pour une requête « authentifiée » (`req.user`), mais le gateway ne decode pas les jetons : cette branche est inerte aujourd'hui, la limite effective est 100 pour tous. Un client qui charge une page riche (liste + notifications + conversations) doit **mutualiser** ses appels et **ne pas boucler** sur un 401 (le disjoncteur du front existe précisément pour cela, §6.1).

Des limiteurs **métier** s'ajoutent côté services : verrou OTP par paliers (5 essais → 1 min, puis 30 min, puis 24 h ; email d'alerte de sécurité dès le deuxième palier ; 6 renvois de code par heure), verrou du code de livraison (3 échecs → 15 min), relance email des messages non lus (au plus une par heure et par conversation), 20 alertes de route par membre, 5 régénérations de code par deal.

2.9 Mode maintenance (503 MAINTENANCE)

Le gateway relit toutes les 10 s le document `PlatformSettings` clé `maintenance` (réglé par l'admin via `PUT /admin/maintenance`), avec repli sur la dernière valeur connue ; la variable `MAINTENANCE_MODE=on` l'emporte sur la base (pour le jour où Mongo est la panne). En maintenance, **seules les écritures** (`POST`, `PUT`, `PATCH`, `DELETE`) sont refusées, **sauf** sous `/api/auth/`, `/api/admin/` et `/api/maintenance` — un membre peut donc se connecter et l'admin lever la maintenance. Réponse : 503, `Retry-After: 300`, corps { code: "MAINTENANCE", message, messages: { fr, en }, `retryAfterSeconds: 300` }. Les lectures continuent normalement.

2.10 Pagination par curseur

Il n'y a **jamais** de pagination par numéro de page. Trois formes coexistent :

- **Recherche de trajets** — `GET /trips/search?...&cursor=<nextCursor>&limit=<n>` : la réponse porte `nextCursor` (id du dernier trajet de la page, null en dernière page) et `totalCount`.
- **Journal admin** — `GET /admin/audit?cursor=<id du dernier élément>` : plus récents en premier.

- **Fil de conversation** — GET /messages/conversations/{id}?cursor=<id du plus ancien message chargé> : le curseur charge les messages **plus anciens** ; la page est rendue du plus ancien au plus récent.

Les listes sans curseur sont bornées côté serveur (par exemple GET /me/notifications : les **50** dernières + unreadCount) ou filtrées (?status=).

2.11 Montants, devises, dates, identifiants

- **Tout montant est un entier en centimes**, accompagné d'un code de devise (currencyCode, ISO 4217) : totalShipperCents, transportCents, pricePerKgCents, refundAmountCents, payoutAmountCents... Jamais de flottant. **Exception assumée et documentée** : les **cartes de résultat de recherche** (YambaTripResult) exposent des prix en **unités** (minPrice, pricePerKg, totalForWeight, en euros, déjà divisés par 100) et un **symbole** de devise (currency) : c'est un DTO d'affichage, pas une donnée de calcul.
- Le **devis** d'une réservation est **recalculé côté serveur** (D34) et **figé** dans le Booking (D17) : le client envoie expectedTotalCents (ce qu'il a affiché) et le serveur refuse toute divergence (409 QUOTE_DIVERGENCE).
- Les **dates** sont des chaînes **ISO 8601 en UTC** (2026-09-20T08:30:00.000Z), sauf les champs explicitement « locaux » d'un trajet (departureDateLocal, departureTimeLocal + fuseau originTimezone) et les dates **formatées serveur** des cartes de recherche (travelDate, departureTime, pilotées par locale).
- Les **identifiants** sont des ObjectId MongoDB sérialisés (24 caractères hexadécimaux) ; un identifiant malformé donne 400. Les membres sont désignés publiquement par leur **slug** (/users/{slug}/public, targetRef d'un signalement), jamais par leur id.

2.12 Idempotence et rejeu

- **Intention de paiement** : POST /deals/payment-intents ne persiste rien ; une intention abandonnée expire chez le fournisseur. POST /deals consomme paymentIntentId **une seule fois** : rejouer la création avec la même intention donne 409 PAYMENT_ALREADY_USED. Un client qui perd la réponse d'un POST /deals doit **relire** GET /me/bookings plutôt que recréer.
- **Versement** : le transfert Stripe au Voyageur porte une clé d'idempotence égale à l'id du deal ; un rejeu par le cron ne paie jamais deux fois.
- **Emails** : sendTransactionalEmail accepte idempotencyKey ; les envois déclenchés par les événements sont dédoublonnés par eventId (§5.3).
- **Favoris** : POST /DELETE /trips/{id}/favorite sont idempotents.
- **Lien de suivi** : un seul par deal, POST /deals/{id}/tracking-link renvoie l'existant.
- **Lecture d'une conversation** : GET /messages/conversations/by-deal/{bookingId} **crée** la conversation au premier accès et la renvoie ensuite.
- **Transitions** : toute transition d'un deal est une **écriture conditionnelle** (statut attendu + verrou optimiste sur updatedAt) ; un rejeu concurrent perd et reçoit 409 TRANSITION_NOT_ALLOWED. Rejouer une transition déjà appliquée n'est donc jamais destructif.

2.13 CORS, même origine et test en réseau local

Le gateway autorise les origines http://localhost:3000 (user-ui), http://localhost:3001 (admin-ui), et les mêmes ports sur 192.168.x.x et 10.x.x.x — avec credentials: true. **Les requêtes sans en-tête Origin (curl, serveur à serveur) sont acceptées**. Un front servi depuis une autre origine reçoit une erreur « Not allowed by CORS » (500).

En développement et en recette, la configuration recommandée (D48) est la **même origine** : next.config.js proxifie /api/* vers le gateway quand API_PROXY_TARGET=http://localhost:8080 est posé, et le navigateur appelle NEXT_PUBLIC_API_BASE_URL=/api. Les cookies sont alors first-party sur n'importe quel hôte (poste, IP LAN, téléphone) ; l'en-tête Origin du navigateur traverse le proxy, d'où la présence des IP privées dans la liste. La configuration historique (URL absolue http://<hôte>:8080/api) reste possible, à condition que **tous** les appareils utilisent le même hôte pour le front et l'API : un front sur localhost avec une API sur l'IP LAN donne un login 200 suivi d'un /auth/me 401 (cookies liés à l'hôte).

3. Guides pas à pas

Conventions communes à tous les exemples de ce chapitre :

- BASE=http://localhost:8080/api (le gateway). Les chemins sont donnés **côté client** ; la mention « répond : service » indique le service réellement atteint derrière le proxy.
- Les sessions membres sont tenues dans un **jar de cookies** : -c jar.txt pour enregistrer les Set-Cookie, -b jar.txt pour les renvoyer. Un client natif ferait la même chose avec un jar, ou (porte D36) avec un en-tête Authorization: Bearer ...
- Chaque appel envoie x-locale : c'est la langue des emails **sans compte** et de la première preferredLocale.
- Les identifiants (665f...) et les codes (482913) sont des exemples ; les vraies valeurs viennent des réponses précédentes ou des emails reçus (Mailpit en local).
- Les réponses sont abrégées (...) quand la référence du chapitre 7 en donne la forme complète.

3.1 Inscription → vérification OTP → connexion → /auth/me

Étape 1 — démarrer l'inscription (répond : auth-service). Le consentement est obligatoire : versions des CGU et de la politique de confidentialité. La demande vit **10 minutes en Redis** ; rien n'est écrit en base avant la vérification.

```
curl -s -X POST "$BASE/auth/register" \
-H "Content-Type: application/json" -H "x-locale: fr" \
-d '{
  "firstName": "Awa", "lastName": "Diallo",
  "email": "awa.diallo@example.com", "password": "Kinshasa-2026!",
  "termsAccepted": true, "termsVersion": "2026-06-01", "privacyVersion": "2026-06-01"
}'
```

Réponse 200 :

```
{ "message": "OTP sent to your email. Please verify your account.", "verificationToken": "3f1c..." }
```

Erreurs : 400 (champ manquant, mot de passe trop faible — `details.type: "password"` avec un code de règle tel que `PASSWORD_CONTAINS_PERSONAL_INFO`, ou `details.type: "register"`), 409 (email déjà utilisé — proposer la connexion ou le mot de passe oublié), 429 (verrou anti-spam : 6 envois de code par heure ; lire `details.lockUntilSeconds`).

Étape 2 — vérifier le code reçu par email (répond : `auth-service`). Le `verificationToken` identifie la demande en attente ; l'`otp` est le code à six chiffres.

```
curl -s -X POST "$BASE/auth/register/verify" \
-H "Content-Type: application/json" -H "x-locale: fr" \
-d '{ "verificationToken": "3f1c...", "otp": "482913" }'
```

Réponse 201 : { `"success": true, "message": "Account created successfully"` } — le `User` et le `ConsentLog` sont écrits, l'email de bienvenue part ; **aucune session n'est ouverte** à cette étape (le membre se connecte ensuite). Erreurs : 401 code faux ou expiré avec `details.type: "otp"` — c'est le cas à traiter avec soin :

```
{ "status": "error", "message": "Incorrect code. 4 attempt(s) left before this code is invalidated.",
  "details": { "type": "otp", "code": "OTP_INCORRECT", "attemptsLeft": 4, "locked": false } }
```

Le client construit son message depuis `code`, `attemptsLeft`, `lockUntilSeconds` et `otpInvalidated` — jamais depuis `message`. Après 5 échecs le code est **invalidé** (`OTP_INVALIDATED`) et un verrou d'une minute s'applique (`OTP_LOCKED`, 429 avec `lockUntilSeconds`), puis 30 minutes, puis 24 heures ; un email d'alerte de sécurité part dès le deuxième palier. Pour recevoir un nouveau code : `POST /auth/register/resent { verificationToken }` ; pour abandonner : `POST /auth/register/cancel { verificationToken }`.

Étape 3 — se connecter (répond : `auth-service`). `rememberMe: true` allonge le cookie de rafraîchissement à 30 jours (session `rememberMe` : 7 jours d'inactivité, 30 jours de vie absolue).

```
curl -s -X POST "$BASE/auth/login" \
-H "Content-Type: application/json" -H "x-locale: fr" -c jar.txt \
-d '{ "email": "awa.diallo@example.com", "password": "Kinshasa-2026!", "rememberMe": true }'
```

Réponse 200 :

```
{ "message": "Login successful!",
  "user": { "id": "665f1c2ab3d4e5f6a7b8c9d0", "email": "awa.diallo@example.com", "firstName": "Awa", "lastName": "Diallo", "roles": ["USER"] } }
```

Les jetons sont dans les en-têtes `Set-Cookie` (`access_token` 15 min, `refresh_token` 30 jours), pas dans le corps. Erreurs : 401 identifiants invalides **ou compte suspendu** (même statut : ne pas révéler lequel), 400 corps invalide. Variante Google (D47) : `POST /auth/google { credential, rememberMe?, consent? }` — un nouveau compte sans `consent` reçoit `status: "CONSENT_REQUIRED"` sans rien créer ; le client redemande le consentement puis rappelle avec `consent: { termsVersion, privacyVersion }`.

Étape 4 — lire le membre courant (répond : `auth-service`).

```
curl -s "$BASE/auth/me" -b jar.txt -H "x-locale: fr"
```

Réponse 200 : { `"success": true, "user": { ...le document User sans passwordHash... }, "roles": ["USER"] }` }. Un 401 ici signifie que le jeton d'accès a expiré (15 min) : le client appelle `POST /auth/refresh` **une fois**, puis rejoue :

```
curl -s -X POST "$BASE/auth/refresh" -b jar.txt -c jar.txt
curl -s "$BASE/auth/me" -b jar.txt
```

Si le `refresh` répond 401 (« Session expired or invalid »), la session est finie : inactivité dépassée, plafond absolu atteint, j'ti révoqué depuis un autre appareil, mot de passe changé. Le client passe à l'état « déconnecté » et n'insiste pas (disjoncteur de 30 s dans le front, §6.1). Déconnexion : `POST /auth/logout -b jar.txt`.

Mot de passe oublié, même mécanique OTP : `POST /auth/password/forgot { email }` répond **toujours 200** (ne révèle pas l'existence du compte), puis `POST /auth/password/verify { email, otp }` → jeton de réinitialisation, puis `POST /auth/password/reset { ... }` ; `POST /auth/password/resent` renvoie un code.

3.2 Devenir Voyageur : profil, Stripe Connect, activation

Un membre publie des trajets une fois son **onboarding Voyageur** terminé. Trois routes (répondent : `auth-service`), toutes sous session :

```
# 1. Le profil public de transporteur (créé ou met à jour la CarrierPage)
curl -s -X POST "$BASE/carrier/onboarding/profile" -b jar.txt \
-H "Content-Type: application/json" \
-d '{ "name": "Awa D.", "bio": "Paris à Kinshasa tous les mois.", "phoneE164": "+33612345678",
      "address": { "formattedAddress": "12 rue de la Paix, 75002 Paris, France", "city": "Paris", "country": "France", "countryCode":
"FR", "lat": 48.8687, "lng": 2.3312 } }'
# → 200 { "success": true, "message": "...", "carrierPage": { "onboardingStep": "...", "stripeOnboardingComplete": false, ... } }

# 2. Le lien d'onboarding Stripe Connect (compte Express créé au premier appel ; URL à usage unique, jamais stockée)
curl -s -X POST "$BASE/carrier/onboarding/stripe" -b jar.txt
# → 200 { "success": true, "url": "https://connect.stripe.com/setup/e/acct_.../" }

# 3. Après le retour de Stripe : l'état du compte
curl -s "$BASE/carrier/onboarding/stripe/status" -b jar.txt
# → 200 { "success": true, "status": "complete", "chargesEnabled": true, "payoutsEnabled": true, "detailsSubmitted": true }

# 4. Terminer : accorde le rôle CARRIER et envoie l'email de fin d'onboarding
curl -s -X POST "$BASE/carrier/onboarding/complete" -b jar.txt
# → 200 { "success": true, "message": "..." }
```

Le client redirige l'utilisateur vers l'URL Stripe (navigateur système sur mobile), puis **interroge stripe/status** au retour. `status` vaut `not_started`, `pending` ou `complete`. Les drapeaux (`chargesEnabled`, `payoutsEnabled`) sont ensuite **maintenus par le webhook Stripe `account.updated`** (§5.1) sans que le Voyageur repasse par l'onboarding. Plus tard, le tableau de bord Express (IBAN, historique) s'ouvre par `POST /carrier/stripe/dashboard-link` — **sous sudo** (403 `SUDO_REQUIRED` sinon, 409 `STRIPE_ACCOUNT_MISSING` s'il n'y a pas encore de compte).

Deux portes métier dépendent de cet état : la **publication d'un trajet** (le trip-service exige `onboarding` + `chargesEnabled`) et surtout **l'acceptation d'un deal** (D31 : la porte est vérifiée à l'acceptation, 409 `CARRIER_ONBOARDING_REQUIRED`).

3.3 Publier un trajet

Le trajet naît en **brouillon**, éventuellement incomplet, puis est **publié** ; ou est créé directement publié avec `publish: true` si les portes passent. Répond : `trip-service (/api/trips → /trips)`. Les montants sont en **centimes**.

```
curl -s -X POST "$BASE/trips" -b jar.txt -H "Content-Type: application/json" -d '{
  "transportMode": "PLANE", "tripType": "ONE_WAY",
  "originLabel": "Paris-Charles de Gaulle (CDG)", "originCity": "Paris", "originCountry": "France", "originCountryCode": "FR",
  "originLat": 49.0097, "originLng": 2.5479, "originTimezone": "Europe/Paris",
  "destinationLabel": "Kinshasa N'djili (FIH)", "destinationCity": "Kinshasa", "destinationCountry": "RD Congo",
  "destinationCountryCode": "CD",
  "destinationLat": -4.3857, "destinationLng": 15.4446, "destinationTimezone": "Africa/Kinshasa",
  "departureAt": "2026-10-12T20:30:00.000Z", "arrivalAt": "2026-10-13T05:10:00.000Z",
  "flightType": "DIRECT",
  "acceptedCategories": ["CLOTHES", "DOCUMENTS", "BOOKS", "PHONE"],
  "pricePerKgCents": 1200, "capacityKg": 15,
  "checkedBag23PriceCents": null, "cabinBag12PriceCents": 6000,
  "familyConditions": [{ "familyKey": "ELECTRONICS_DEVICES", "mode": "SURCHARGE", "surchargePct": 20 }],
  "pickupLocations": [{ "kind": "AIRPORT", "details": "Terminal 2E, hall de départ", "flexibility": "EXACT" }],
  "deliveryLocations": [{ "kind": "AIRPORT", "details": "Arrivées, sortie principale", "flexibility": "EXACT" },
    { "kind": "CITY_AREA", "details": "Gombe", "flexibility": "RADIUS", "radiusKm": 5 }],
  "instantBooking": false, "maxSlots": null, "publish": false
}'
```

Réponse 201 : { "success": true, "message": "...", "trip": { "id": "66a0...", "status": "DRAFT", "pricingModel": "PER_KG", "minPriceCents": ..., ... } } (`minPriceCents` et `departureHourLocal` sont recalculés côté serveur). Puis :

```
curl -s -X POST "$BASE/trips/66a0.../publish" -b jar.txt
# → 200 { "success": true, "message": "Trip published" }
```

Portes de publication, toutes refusées en **400** avec le message de la machine (`canPerform`) : `onboarding` Voyageur incomplet, Stripe sans `chargesEnabled`, champs requis manquants (`mode`, villes, départ futur, ≥ 1 catégorie), moins d'un point de récupération ou de remise, `capacityKg` absent avec `pricePerKgCents` (porte A28). Un compte `RESTRICTED` reçoit **403 `ACCOUNT_RESTRICTED`** (middleware, avant le contrôleur). Le cycle de vie continue par `pause`, `resume`, `unpublish`, `cancel`, `restore`, `archive` ; `GET /trips/my?status=PUBLISHED` liste ses trajets **avec `allowedActions`** — le client n'affiche que les boutons présents dans cette liste. `PUT /trips/{id}` modifie (la capacité est immuable après publication). Les documents de trajet (billet à vérifier) passent par un `upload direct ImageKit` puis `POST /trips/{id}/documents` ; la vérification est faite par le support (`/admin/tickets`).

3.4 Rechercher, lire un trajet public, le mettre en favori

La recherche est **publique** (répond : `trip-service`) ; connecté, elle personnalise `isFavorite`. Filtres durs non négociables : `status=PUBLISHED`, départ futur, trajets masqués par Yamba exclus.

```
curl -s "$BASE/trips/search?from=Paris&to=Kinshasa&dateFrom=2026-10-
01&mode=plane&categories=clothes,documents&sort=earliest&locale=fr&limit=20" \
-H "x-locale: fr" -b jar.txt
```

Réponse 200 (enveloppe **sans** `success`) :

```
{ "trips": [ { "id": "66a0...", "fromCity": "Paris", "toCity": "Kinshasa", "travelDate": "dim. 12 oct.", "departureTime": "22:30",
  "pricePerKg": 12, "remainingKg": 15, "minPrice": 0, "currency": "€", "transportMode": "plane",
  "allowedCategories": ["clothes", "documents", "books", "phone"], "travelerFirstName": "Awa", "travelerLastName": "D.",
  "rating": 4.8, "reviewCount": 12, "isFavorite": false, "viewsCount": 37 } ],
"nextCursor": null, "totalCount": 1 }
```

Les catégories et le mode sont en **convention UI** (`clothes`, `plane`) et non en `enum Prisma (CLOTHES, PLANE)` ; les valeurs invalides sont ignorées silencieusement. Avec `weightKg=3`, chaque carte porte en plus `transportForWeight` et `totalForWeight` (en euros). `GET /trips/search/facets` donne les compteurs par filtre ; `GET /trips/pricing/params (public)` donne commission, planchers, coefficients — le wizzard calcule le même devis que le serveur (D34).

```
curl -s "$BASE/trips/66a0.../public" -H "x-locale: fr" -b jar.txt
# → 200 { "success": true, "trip": { origin, destination, dates, tripper: { firstName, lastName, ... }, pricing, locations, ... } }
# → 404 { PublicNotFound } si inexistant, supprimé, non publié ou masqué par Yamba ; 400 si l'id n'est pas un ObjectId
curl -s -X POST "$BASE/trips/66a0.../favorite" -b jar.txt # idempotent ; 409 OWN_TRIP / TRIP_NOT_FAVORITABLE (details.type "favorite")
curl -s "$BASE/trips/favorites" -b jar.txt
```

3.5 Réserver et vivre le deal : de l'intention de paiement à la notation

C'est le parcours central. Tous les appels de cette section répondent : **deal-service**. Deux acteurs : l'**Expéditeur** (jar `shipper.txt`) et le **Voyageur** propriétaire du trajet (jar `carrier.txt`). Le deal traverse les statuts `PENDING` → `ACCEPTED` → `PICKED_UP` → `DELIVERED` → `COMPLETED` (fins alternatives : `DECLINED`, `EXPIRED`, `CANCELLED`, `DISPUTED`). Règle absolue : **le client reflète `allowedActions`, il ne décide jamais** ; toute limite est appliquée côté serveur.

Étape 1 — l'intention de paiement (D37, étape 1 de 2). Le serveur recalcule le devis avec le moteur unique et **autorise** le montant chez le fournisseur (capture manuelle). Rien n'est persisté. `expectedTotalCents` est ce que l'Expéditeur a vu à l'écran.

```
curl -s -X POST "$BASE/deals/payment-intents" -b shipper.txt -H "Content-Type: application/json" -d '{
  "tripId": "66a0...", "product": "PARCEL", "family": "CLOTHES_TEXTILE", "sizeClass": "M", "weightKg": 3,
  "protection": "BASIC", "expectedTotalCents": 4032
}'
```

Réponse 201 :

```
{ "provider": "STRIPE", "paymentIntentId": "pi_3P...", "clientSecret": "pi_3P..._secret_...",
  "amountCents": 4032, "currencyCode": "EUR",
  "quote": { "pricingModel": "PER_KG", "weightKg": 3, "pricePerKgCents": 1200, "sizeClass": "M", "sizeCoef": 1.1,
    "transportCents": 3600, "commissionPct": 12, "commissionCents": 432, "premiumCents": 0,
    "serviceCents": 432, "totalShipperCents": 4032, "currencyCode": "EUR" } }
```

Le client confirme ensuite le paiement avec `clientSecret` (Payment Element sur le web, Payment Sheet sur mobile). Avec le fournisseur `FAKE` (hors production seulement), `clientSecret` est null et l'autorisation est immédiate. Erreurs 409 (details.type: "booking") : `QUOTE_DIVERGENCE` (le prix a changé : **rafraîchir le devis et ré-afficher, jamais débiteur un montant non vu**), `FAMILY_REFUSED`, `CAPACITY_EXCEEDED`, `TRIP_NOT_BOOKABLE`, `OWN_TRIP`, `NEW_ACCOUNT_CAP` (plafond progressif D71 : details.cap ∈ `DECLARED_VALUE` | `WEIGHT` | `SHIPMENTS_PER_MONTH`, limit, value). 403 `ACCOUNT_RESTRICTED` pour un compte restreint. 400 si le devis est impossible (errors.quote porte le code du moteur).

Étape 2 — créer la demande (PENDING). Même saisie + `paymentIntentId` + description, valeur déclarée, destinataire, lieux choisis parmi ceux du trajet, chartes acceptées. En **une transaction Mongo** : réservation conditionnelle des kilos (CAP-01), `Booking` avec cinq instantanés figés (trajet, prix, colis, destinataire, lieux), deux événements outbox (`booking.requested`, `booking.payment_authorized`).

```
curl -s -X POST "$BASE/deals" -b shipper.txt -H "Content-Type: application/json" -d '{
  "tripId": "66a0...", "paymentIntentId": "pi_3P...", "expectedTotalCents": 4032,
  "product": "PARCEL", "family": "CLOTHES_TEXTILE", "sizeClass": "M", "weightKg": 3, "protection": "BASIC",
  "description": "Deux pagnes et un boubou brodé, emballés sous vide.",
  "declaredValueCents": 15000, "photoUrls": ["https://ik.imagekit.io/yamba/parcels/pagne-1.jpg"],
  "recipient": { "firstName": "Marie", "lastName": "Kabeya", "phoneE164": "+243812345678", "email": "marie.k@example.com" },
  "pickupPlace": { "kind": "AIRPORT", "details": "Terminal 2E" }, "deliveryPlace": { "kind": "CITY_AREA", "details": "Gombe" },
  "charterAccepted": true, "termsAccepted": true
}'
```

Réponse 201 : { "bookingId": "66b1...", "status": "PENDING", "expiresAt": "2026-09-07T10:15:00.000Z", "totalShipperCents": 4032, "currencyCode": "EUR" }. Le Voyageur est notifié par le relais (push + email avec l'échéance) ; la **fenêtre d'acceptation de 24 h** court (DEA-01) — passée, un cron ferme le deal en `EXPIRED` et libère l'autorisation. Erreurs 409 supplémentaires : `PAYMENT_NOT_AUTHORIZED` (le paiement n'a pas été confirmé côté client), `PAYMENT_MISMATCH` (montant ou devise différents de l'autorisation), `PAYMENT_ALREADY_USED` (intention déjà consommée : **relire `GET /me/bookings` plutôt que recréer**).

Étape 3 — lire le deal, dans la vue de son rôle. La forme du DTO dépend du rôle de l'appelant ; les deux vues sont des listes blanches strictes.

```
curl -s "$BASE/deals/66b1..." -b shipper.txt
# → 200 { "success": true, "viewerRole": "SHIPPER", "deal": { "status": "PENDING", "pricing": { ...complet... }, "recipient": { ... },
  "carrier": { firstName, ... }, "expiresAt", "allowedActions": ["cancel"], "cancellationPreview": { ... }, "deliveryCode": null, ... } }
curl -s "$BASE/deals/66b1..." -b carrier.txt
# → 200 { "success": true, "viewerRole": "CARRIER", "deal": { "status": "PENDING", "pricing": { "transportCents": 3600, "currencyCode": "EUR", ... }, "shipper": { ... }, "allowedActions": ["accept", "decline"], ... } }
# (jamais de code de livraison, jamais de hash, jamais les totaux Expéditeur dans la vue Voyageur)
```

403 si l'appelant n'est ni l'Expéditeur ni le Voyageur ; 404 si le deal n'existe pas. Listes : `GET /me/bookings?status=PENDING` (Expéditeur), `GET /me/deals` (Voyageur, tous trajets), `GET /deals?tripId=66a0...` (Voyageur, un trajet), `GET /me/wallet` (portefeuille).

Étape 4 — accepter (Voyageur). La charte du transporteur doit être cochée ; la **porte D31** (profil complet + Stripe) est vérifiée ici ; le paiement est **capturé** maintenant (une autorisation expire en ~7 jours), puis, en une transaction conditionnelle, `PENDING` → `ACCEPTED` + `outbox booking.accepted`. La **conversation** du deal devient disponible (§3.6).

```
curl -s -X POST "$BASE/deals/66b1.../accept" -b carrier.txt -H "Content-Type: application/json" -d '{ "charterAccepted": true }'
# → 200 { "bookingId": "66b1...", "status": "ACCEPTED", "refundAmountCents": null, "currencyCode": "EUR" }
```

Erreurs 409 : `TRANSITION_NOT_ALLOWED` (déjà accepté, expiré, annulé, ou écriture concurrente — `details.reason` porte le motif de la machine), `CARRIER_ONBOARDING_REQUIRED` (envoyer le Voyageur terminer son onboarding, §3.2), `PAYMENT_STATE_CONFLICT` (l'autorisation n'est plus capturable : l'Expéditeur doit refaire une demande). Alternatives : `POST /deals/{id}/decline { "reason": "TIMING" }` (motif facultatif parmi 5 ; remboursement intégral), et côté Expéditeur `POST /deals/{id}/cancel { "reason": "..." }` — remboursement intégral tant que le deal est `PENDING` ou dans les 48 h suivant l'acceptation, retenue de 50 % au-delà (ANN-01, valeurs réglables par l'admin) ; `cancellationPreview` dans la vue Expéditeur annonce le montant **avant** le geste, et la réponse rend `refundAmountCents`.

Étape 5 — récupérer le colis (Voyageur). Les 5 points d'inspection sont **tous** obligatoires (CNF-04) et 1 à 5 photos doivent déjà être sur ImageKit (upload direct signé, §3.10 ; aucun octet ne passe par l'API). `ACCEPTED` → `PICKED_UP`, et le serveur **génère le code de livraison** à six chiffres, stocké deux fois (bcrypt pour la validation, AES-256-GCM pour le ré-affichage à l'Expéditeur — D43).

```
curl -s -X POST "$BASE/deals/66b1.../pickup" -b carrier.txt -H "Content-Type: application/json" -d '{
  "checklist": [ "CONTENT_MATCHES", "WEIGHT_OK", "NO_FORBIDDEN", "PACKAGING_OK", "ITEMS_IDENTIFIED" ],
  "photoUrls": [ "https://ik.imagekit.io/yamba/pickups/66b1-1.jpg", "https://ik.imagekit.io/yamba/pickups/66b1-2.jpg" ],
  "notes": "Colis conforme, 2,9 kg à la pesée."
}'
# → 200 { "bookingId": "66b1...", "status": "PICKED_UP", "refundAmountCents": null, "currencyCode": "EUR" }
```

Le code n'est jamais dans cette réponse, jamais dans un événement, jamais dans un email, jamais dans une vue Voyageur. L'Expéditeur le lit sur `GET /deals/{id}` (vue Shipper, champ `deliveryCode`, statut `PICKED_UP` uniquement, jamais dans les listes) et le transmet au destinataire **de vive voix**. En cas de doute : `POST /deals/{id}/code/regenerate` (Expéditeur, 5 régénérations maximum → 409 `CODE_REGENERATION_LIMIT` ; la réponse est la **seule** surface d'écriture qui rend le nouveau code : { `bookingId`, `deliveryCode`, `codeRegenerationsLeft` }). Refus à la récupération : `POST /deals/{id}/pickup/refuse { "reason": "OVERWEIGHT" }` (annulation + remboursement).

Étape 6 — jalons de suivi facultatifs (Voyageur). Séquence stricte `AT_AIRPORT` → `FLIGHT_DEPARTED` → `FLIGHT_ARRIVED`, sans saut ni doublon ; pas de changement de statut ; notification in-app à l'Expéditeur, sans email. L'annulation « 5 secondes » est **côté client** : appeler la route après la fenêtre d'annulation, il n'y a pas d'annulation serveur.

```
curl -s -X POST "$BASE/deals/66b1.../events" -b carrier.txt -H "Content-Type: application/json" -d '{ "step": "AT_AIRPORT" }'
# → 200 { "bookingId": "66b1...", "step": "AT_AIRPORT", "confirmedAt": "...", "trackingEvents": [ { "step": "AT_AIRPORT", "confirmedAt": "..." } ] }
# → 409 TRACKING_STEP_NOT_ALLOWED si l'ordre n'est pas respecté ou si le deal n'est pas PICKED_UP
```

Étape 7 — remettre le colis avec le code (Voyageur). Le code donné par le destinataire est comparé avec bcrypt. `PICKED_UP` → `DELIVERED`, `payoutDueAt` = `deliveredAt` + 4 jours (fenêtre de vérification de l'Expéditeur), `outbox booking.delivered`.

```
curl -s -X POST "$BASE/deals/66b1.../deliver" -b carrier.txt -H "Content-Type: application/json" \
-d '{ "code": "742891", "photoUrls": [ "https://ik.imagekit.io/yamba/deliveries/66b1-1.jpg" ] }'
# → 200 { "bookingId": "66b1...", "status": "DELIVERED", "deliveredAt": "...", "payoutDueAt": "...+4 jours" }
```

Erreurs 409 à traiter finement : `DELIVERY_CODE_INVALID` avec `details.attemptsLeft` (afficher « il reste N essais »), `DELIVERY_LOCKED` avec `details.lockedUntil` (3 échecs → verrou de 15 minutes ; désactiver la saisie jusqu'à cette date — le serveur refuse de toute façon toute comparaison pendant le verrou), `DELIVERY_CODE_UNAVAILABLE` (enregistrement antérieur au chantier B3, sans code : contacter le support), `TRANSITION_NOT_ALLOWED`.

Étape 8 — confirmer (Expéditeur) ou laisser courir. Pendant la fenêtre `DELIVERED`, l'Expéditeur peut **confirmer** (versement libéré immédiatement, geste irréversible : le droit de contester disparaît) ou **contester** ; à `payoutDueAt`, le cron confirme automatiquement (`completedBy: SYSTEM`) et déclenche le versement.

```
curl -s -X POST "$BASE/deals/66b1.../confirm" -b shipper.txt
# → 200 { "bookingId": "66b1...", "status": "COMPLETED", "completedAt": "...", "payoutStatus": "SENT", "payoutAmountCents": 3600,
"currencyCode": "EUR" }
```

`payoutStatus` vaut `SENT` si le transfert Stripe a réussi en ligne, `FAILED` si le fournisseur a refusé (le cron réessaie toutes les 5 minutes, jusqu'à 10 fois ; l'admin voit la file `/admin/finances/queue`). 409 `TRANSITION_NOT_ALLOWED` si le deal n'est pas `DELIVERED` ou si la fenêtre est passée.

Litige (Expéditeur) : depuis `DELIVERED` avant `payoutDueAt`, ou depuis `PICKED_UP` si le départ du trajet date de plus de 48 h sans remise (catégorie obligatoirement `NOT_DELIVERED`).

```
curl -s -X POST "$BASE/deals/66b1.../dispute" -b shipper.txt -H "Content-Type: application/json" -d '{
  "category": "DAMAGED", "pledgeAccepted": true, "desiredOutcome": "PARTIAL_REFUND",
  "description": "Le colis est arrivé avec l emballage déchiré et le boubou taché sur la manche gauche ; photos jointes.",
  "photoUrls": [ "https://ik.imagekit.io/yamba/disputes/66b1-1.jpg" ]
}'
# → 200 { "bookingId": "66b1...", "status": "DISPUTED", "ticketNumber": "YAM-2041", "disputedAt": "..." }
```

Le versement est **gelé**, un dossier de médiation est ouvert (le Voyageur donne sa version par `POST /deals/{id}/dispute/statement`, une seule fois), et la décision revient au médiateur (`POST /admin/disputes/{id}/resolve`) : les deux parties reçoivent l'issue, leur montant et le motif. La description fait **au moins 50 caractères** (400 sinon).

Étape 9 — noter l'autre partie (double aveugle, D53). Une fois par rôle, statut `COMPLETED`, dans les **14 jours**. Les avis restent cachés jusqu'à ce que les deux aient noté, ou jusqu'à la fin de la fenêtre.

```
curl -s "$BASE/deals/66b1.../rating" -b shipper.txt
# → 200 { "canRate": true, "ratedRole": "CARRIER", "windowEndsAt": "...", "myRating": null, "counterpartHasRated": false, "revealedAt": null, ... }
curl -s -X POST "$BASE/deals/66b1.../rating" -b shipper.txt -H "Content-Type: application/json" \
-d '{ "rating": 5, "criteria": { "PUNCTUALITY": "UP", "COMMUNICATION": "UP", "PARCEL_CARE": "UP" }, "comment": "Ponctuelle et soigneuse, je recommande." }'
# → 201 { "bookingId": "66b1...", "submittedAt": "...", "revealed": false, "revealedAt": null }
# → 409 TRANSITION_NOT_ALLOWED si déjà noté, hors fenêtre, ou deal non COMPLETED
```

Les critères hors du rôle noté sont ignorés (`PUNCTUALITY`, `COMMUNICATION`, `PARCEL_CARE` pour noter un Voyageur ; `DECLARATION_CLARITY`, `RESPONSIVENESS`, `PUNCTUALITY` pour noter un Expéditeur). Seuls les avis révélés nourrissent la réputation publique (`GET /users/{slug}/public/reviews`).

3.6 Messagerie : fil, rendez-vous, numéro

Répond : **message-service** (`/api/messages` → `/messages`). La conversation d'un deal **existe à partir de l'acceptation** (D61 2A) : avant, la demande porte déjà une description et ouvrir un fil inviterait à sortir de la plateforme. Elle est **créée au premier accès** par la route « by-deal ».

```
# Le fil d'un deal (créé au premier accès ; 403 avant ACCEPTED ou si l'appelant n'est pas partie)
curl -s "$BASE/messages/conversations/by-deal/66b1..." -b shipper.txt -H "x-locale: fr"
```

Réponse 200 (ConversationThreadResponse) :

```
{ "conversation": { "id": "66c2...", "bookingId": "66b1...", "role": "SHIPPER",
  "counterpart": { "id": "...", "firstName": "Awa", "avatarUrl": null },
  "corridor": { "originCity": "Paris", "destinationCity": "Kinshasa", "departureAt": "2026-10-12T20:30:00.000Z" },
  "bookingStatus": "ACCEPTED", "lastMessage": null, "unreadCount": 0, "nextMeetup": null,
  "access": { "canRead": true, "canWrite": true, "reason": null, "writeClosesAt": null } },
  "messages": [ { "id": "...", "kind": "SYSTEM", "authorRole": "SYSTEM", "authorId": null, "body": "deal.accepted",
    "systemKey": "deal.accepted", "systemData": {}, "photoUrls": [], "flaggedContact": false, "createdAt": "... } ],
  "meetups": [], "nextCursor": null,
  "phone": { "revealed": false, "phoneE164": null, "opensAt": "2026-10-12T18:30:00.000Z" } }
```

`access.reason` est une **clé stable** que le client traduit : `NOT_ACCEPTED_YET`, `DISPUTE_OPEN` (lecture seule pendant un litige — les échanges passent par la médiation), `DEAL_CLOSED`, `WRITE_WINDOW_OVER` (14 jours après la fin du deal, `writeClosesAt` annonce la date). Les messages `SYSTEM` et `MEETUP` portent une `systemKey` : le client affiche **sa propre copie traduite**, `body` n'est qu'un repli. `GET /messages/conversations` liste les fils (`{ items, totalUnread }`); `GET /messages/conversations/{id}?cursor=<plus ancien id chargé>` remonte l'historique.

```
# Poster un message (texte ≤ 2000 caractères, ≤ 5 photos ImageKit)
curl -s -X POST "$BASE/messages/conversations/66c2.../messages" -b shipper.txt -H "Content-Type: application/json" \
-d '{ "body": "Bonjour Awa, je serai au terminal 2E vers 19h avec le colis." }'
# → 201 { Message } — ou 400 { details: { code: "DELIVERY_CODE_IN_MESSAGE" } } si un groupe de six chiffres correspond au code du deal
# → 400 « This conversation is read-only (DISPUTE_OPEN). » si l'écriture est fermée

# Marquer lu
curl -s -X POST "$BASE/messages/conversations/66c2.../read" -b carrier.txt

# Proposer un rendez-vous (objet métier, pas un message) : ≥ 30 min à l'avance, ≤ 90 jours, créneau ≤ 12 h
curl -s -X POST "$BASE/messages/conversations/66c2.../meetups" -b carrier.txt -H "Content-Type: application/json" -d '{
  "kind": "PICKUP", "placeLabel": "CDG Terminal 2E, porte 8", "placeDetails": "Devant le comptoir Air France",
  "startAt": "2026-10-12T17:30:00.000Z", "endAt": "2026-10-12T18:30:00.000Z" }'
# → 201 { "id": "66d3...", "kind": "PICKUP", "status": "PROPOSED", "proposedByRole": "CARRIER", ... }
# Une nouvelle proposition du même type REMPLACE la précédente (contre-proposition) ; 400 « Invalid meeting slot (T00_SOON) » sinon.

# Accepter (l'AUTRE partie seulement ; garde optimiste : un changement concurrent répond 400 « This meeting was just changed »)
curl -s -X POST "$BASE/messages/conversations/66c2.../meetups/66d3.../accept" -b shipper.txt
# → 200 { "id": "66d3...", "status": "ACCEPTED", "acceptedAt": "...", ... }

# Réponses rapides traduites dans la langue du lecteur
curl -s "$BASE/messages/quick-replies" -b shipper.txt -H "x-locale: fr"
# → 200 { "items": [ { "key": "...", "kind": "PICKUP", "text": "Je suis arrivé(e), où êtes-vous ?" }, ... ] }
```

Le **numéro de l'autre partie** ne s'ouvre qu'**au plus tôt 2 heures avant le rendez-vous de récupération accepté** (à défaut, avant le départ du trajet — D61 4A), une fois par lecteur, tracé (`PhoneReveal`) et signalé dans le fil par un message système. Le client montre un bouton « Appeler » qui mène au fil (`? focus=phone`), jamais un `tel:` direct avant l'heure.

```
curl -s -X POST "$BASE/messages/conversations/66c2.../phone" -b shipper.txt
# → 200 { "phoneE164": "+33612345678", "firstName": "Awa", "revealedAt": "... }
# → 400 { "message": "The phone number opens 2026-10-12T15:30:00.000Z.", "details": { "code": "T00_EARLY" } } avant l'heure
# → 400 { details: { code: "NO_ANCHOR" } } sans rendez-vous ni date de départ
```

Le client lit `phone.opensAt` dans le fil pour afficher l'heure d'ouverture plutôt que de tenter l'appel. **Signaler un message** de l'autre partie (texte seulement, une fois par signaleur) : `POST /messages/conversations/{id}/messages/{messageId}/report { "reason": "OFF_PLATFORM", "details": "... } → 201 { reportId, createdAt } ; 409 si déjà signalé ; 400 OWN_MESSAGE / NOT_A_TEXT`. C'est un dossier de modération, pas une transition : aucun événement n'est émis ; le support le voit dans sa file (`GET /admin/conversations/reports`).

Les coordonnées (email, téléphone) écrites dans un message sont **détectées et marquées** (`flaggedContact: true`), jamais bloquées : le client peut afficher un rappel discret. Un message non lu depuis 15 minutes déclenche un email de relance (au plus un par heure et par conversation, sans le texte du message ; désactivable par `PATCH /auth/me/preferences { "messagingReminderEmails": false }`).

3.7 Lien de suivi pour le destinataire (sans compte)

Le destinataire est un tiers sans compte. L'Expéditeur crée **un** lien par deal et le partage lui-même (D69). Répond : deal-service.

```
curl -s -X POST "$BASE/deals/66b1.../tracking-link" -b shipper.txt
# → 200 { "token": "k3Jf...", "path": "/track/k3Jf...", "recipientFirstName": "Marie", "recipientPhoneE164": "+243812345678" }
# → 403 pour le Voyageur ; 409 TRACKING_NOT_AVAILABLE avant l'acceptation ou après une fin sans livraison
```

path est le chemin **côté front** (`https://<hôte du front>/track/k3Jf...`) ; la page appelle la route publique, **sans session**, soumise au limiteur anonyme du gateway :

```
curl -s "$BASE/track/k3Jf..."
# → 200 { "milestone": "IN_TRANSIT",
#       "steps": [ { "key": "ACCEPTED", "at": "..." }, { "key": "PICKED_UP", "at": "..." }, { "key": "IN_TRANSIT", "at": "..." } ],
#       "recipientFirstName": "Marie", "shipperFirstName": "Paul", "carrier": { "firstName": "Awa", "lastName": "D" },
#       "corridor": { "originCity": "Paris", "destinationCity": "Kinshasa", "departureAt": "...", "arrivalAt": "..." }
# → 404 une fois le destinataire anonymisé (rétention), le lien révoqué ou le deal supprimé
```

Le contenu est **minimal par construction** : jamais l'adresse, un numéro, le code de livraison, des photos ni des montants. Jalons : `ACCEPTED` , `PICKED_UP` , `IN_TRANSIT` , `ARRIVED` , `DELIVERED` , `CLOSED` .

3.8 Signaler un trajet ou un membre

Répond : auth-service (D68). `targetRef` est l'**identifiant public** de la cible : l'id d'un trajet, ou le **slug** d'un membre (le DTO public d'un membre ne porte pas son id). Les motifs sont **fermés par cible** : trajet → `ILLEGAL_CONTENT` , `SCAM` , `INAPPROPRIATE` , `OTHER` ; membre → `SCAM` , `INAPPROPRIATE` , `IMPERSONATION` , `OTHER` .

```
curl -s -X POST "$BASE/reports" -b shipper.txt -H "Content-Type: application/json" \
-d '{ "targetType": "TRIP", "targetRef": "66a0...", "reason": "SCAM", "details": "Le prix affiché change après contact et demande un paiement hors plateforme." }'
# → 201 { "reportId": "66e4...", "createdAt": "..." }
# → 400 OWN_TARGET (sa propre annonce / son propre profil) · 400 REASON_NOT_ALLOWED (motif hors liste pour cette cible)
# → 404 cible invisible (trajet supprimé, membre effacé, page masquée) · 409 doublon ouvert du même auteur
```

Le signaleur reçoit un accusé par email ; la cible n'apprend **jamais** qui l'a signalée ni même qu'elle l'a été. La décision est prise par le support (`PATCH /admin/reports/{id}`), sans sanction automatique.

3.9 Alertes de route (saved-routes)

Un membre enregistre jusqu'à **20** corridors surveillés ; il est prévenu (email et/ou in-app) des nouveaux trajets publiés. Origine et destination sont des instantanés Google Places ; même ville + même pays des deux côtés est refusé ; expiration à **6 mois**, prolongeable. Répond : auth-service.

```
curl -s -X POST "$BASE/saved-routes" -b shipper.txt -H "Content-Type: application/json" -d '{
  "originCity": "Paris", "originCountry": "France", "originCountryCode": "FR", "originPlaceId": "ChIJD7fiBh9u5kcRYJSMaMOCCwQ",
  "originLat": 48.8566, "originLng": 2.3522,
  "destinationCity": "Kinshasa", "destinationCountry": "RD Congo", "destinationCountryCode": "CD", "destinationLat": -4.4419,
  "destinationLng": 15.2663,
  "earliestDate": "2026-10-01", "latestDate": "2026-12-31", "emailEnabled": true, "inAppEnabled": true, "includeNearby": true }'
# → 201 { "success": true, "savedRoute": { "id": "66f5...", "expiresAt": "2027-03-06T...", "isActive": true, ... } }
# → 409 « Limit of 20 alerts reached »
curl -s "$BASE/saved-routes" -b shipper.txt # → { success, savedRoutes: [...] }
curl -s -X PATCH "$BASE/saved-routes/66f5..." -b shipper.txt -H "Content-Type: application/json" -d '{ "emailEnabled": false }'
curl -s -X POST "$BASE/saved-routes/66f5.../extend" -b shipper.txt # repousse l'expiration
curl -s -X DELETE "$BASE/saved-routes/66f5..." -b shipper.txt
```

3.10 Profil et avatar (téléversement ImageKit signé)

Aucun octet d'image ne transite par l'API Yamba (D42) : le client demande une **signature** au trip-service, téléverse **directement** chez ImageKit, puis déclare l'URL obtenue. Le même mécanisme sert aux photos de colis, de récupération, de remise, de litige, de message et aux documents de trajet.

```
# 1. La signature (trip-service, ~30 min de validité)
curl -s "$BASE/uploads/imagekit-auth" -b shipper.txt
# → 200 { "success": true, "token": "...", "expire": 1760000000, "signature": "...", "publicKey": "public_...", "urlEndpoint":
"https://ik.imagekit.io/yamba" }

# 2. Le téléversement direct chez ImageKit (hors Yamba) – multipart, dossier /avatars pour un avatar
curl -s -X POST "https://upload.imagekit.io/api/v1/files/upload" \
-F "file=@portrait.jpg" -F "fileName=portrait.jpg" -F "folder=/avatars" \
-F "publicKey=public_..." -F "signature=..." -F "expire=1760000000" -F "token=..."
# → { "fileId": "66g6...", "url": "https://ik.imagekit.io/yamba/avatars/portrait_abc.jpg", ... }

# 3. Déclarer l'avatar (auth-service) – l'URL doit appartenir à IMAGEKIT_URL_ENDPOINT ; l'ancien fichier est supprimé
curl -s -X POST "$BASE/auth/me/avatar" -b shipper.txt -H "Content-Type: application/json" \
-d '{ "fileId": "66g6...", "url": "https://ik.imagekit.io/yamba/avatars/portrait_abc.jpg" }'
# → 200 { MyProfileResponse } ; DELETE /auth/me/avatar pour le retirer
```

Le profil éditable (D67) : GET /auth/me/profile → { firstName, lastName, publicSlug, avatarUrl, birthDate, profilePublic, showCity, carrier } ; PATCH /auth/me/profile { "displayName": "Awa D.", "bio": "...", "profilePublic": true, "showCity": false }. Le **slug public ne change jamais**. Le profil public est lu par tous sur GET /users/{slug}/public (404 si masqué ou effacé, sauf pour le propriétaire qui voit hidden: true), avec .../public/reviews et .../public/trips ; abonnement par POST / DELETE /users/{slug}/follow et GET /me/following. Préférences : PATCH /auth/me/preferences { "analyticsOptIn": true } (écrit un ConsentLog), PATCH /auth/me/locale { "locale": "en" }.

3.11 RGPD : export et effacement (sous sudo)

Les deux gestes exigent la **fenêtre sudo** (§2.3). Répond : auth-service.

```
# Ouvrir la fenêtre (code reçu par email)
curl -s -X POST "$BASE/auth/me/sudo/request" -b shipper.txt
curl -s -X POST "$BASE/auth/me/sudo/verify" -b shipper.txt -H "Content-Type: application/json" -d '{ "code": "391045" }'
# → 200 { "active": true, "expiresAt": "...+15 min" }

# Export : un JSON en pièce jointe (Content-Disposition: attachment), une DataRequest EXPORT est journalisée
curl -s -X POST "$BASE/auth/me/data-export" -b shipper.txt -o mes-donnees-yamba.json -D -
# → 200, corps DataExport { exportedAt, format, profile, preferences, consents, trips, bookings, reviewsGiven, reviewsReceived, messages,
meetups, phoneReveals, savedRoutes, favorites, following, notifications, reportsMade, dataRequests, ... }
# → 403 SUDO_REQUIRED sans fenêtre · refus `EXPORT_RATE_LIMITED` si un export récent existe déjà

# Effacement : d'abord ce qui bloque
curl -s "$BASE/auth/me/erasure/blockers" -b shipper.txt
# → 200 { "blockers": ["ACTIVE_DEAL"], "counts": { "ACTIVE_DEAL": 1 } } (vide = effaçable)

# Puis le geste, avec le mot de confirmation exact
curl -s -X POST "$BASE/auth/me/erasure" -b shipper.txt -H "Content-Type: application/json" -d '{ "confirmation": "SUPPRIMER" }'
# → 200 { "success": true, "erased": true } – la session est terminée
# → 409 { "code": "ERASURE_BLOCKED", "blockers": ["ACTIVE_DEAL"], "counts": { "ACTIVE_DEAL": 1 } }
```

L'export ne contient **jamais** les coordonnées de l'autre partie, le code de livraison, les signalements visant le membre, ni les dossiers de médiation.

L'effacement est **une transaction** qui anonymise le User champ par champ (les uniques deviennent erased+<id>@anonymised.invalid / deleted-<id>); deals, litiges, avis et messages sont conservés (obligations comptables et de médiation). Liste fermée des bloqueurs : ACTIVE_DEAL, PENDING_REQUEST, PAYOUT_PENDING, RETENTION_HELD, PUBLISHED_TRIP, ADMIN_ACCOUNT — le client traduit chacun et propose l'action qui le lève (attendre la fin du deal, dépublier le trajet...).

3.12 Session admin : connexion → TOTP → KPIs → un geste journalisé

Répond : auth-service pour la session et les KPIs ; le service propriétaire pour chaque geste. Jar séparé admin.txt : les cookies s'appellent admin_preauth, admin_access_token, admin_refresh_token. Le compte doit avoir été promu par grant-admin.ts ou par une invitation (POST /admin/admins/invite → POST /auth/admin/invite/accept { token, password }).


```
# 1. Mot de passe → cookie admin_preauth (5 min)
curl -s -X POST "$BASE/auth/admin/login" -c admin.txt -H "Content-Type: application/json" \
-d '{"email": "support@yamba.example", "password": "..."}'
# → 200 { "next": "TOTP" } (ou { "next": "SETUP" } au premier accès : totp/setup → scanner otpauthUrl → totp/enable { code })

# 2. Code TOTP → session admin
curl -s -X POST "$BASE/auth/admin/totp/verify" -b admin.txt -c admin.txt -H "Content-Type: application/json" -d '{"code": "203911"}'
# → 200 { "ok": true, "usedBackupCode": false, "remainingBackupCodes": 8 }
# → 401 code faux (verrou OTP par paliers, 429)

# 3. Qui suis-je, avec quels profils
curl -s "$BASE/admin/me" -b admin.txt
# → 200 { "id": "...", "email": "...", "adminRole": "SUPPORT", "adminRoles": ["SUPPORT"], "remainingBackupCodes": 8 }

# 4. Les compteurs d'accueil – null = non visible pour ce profil
curl -s "$BASE/admin/kpis" -b admin.txt
# → 200 { "ticketsToVerify": 3, "hideProposals": 1, "suspensionProposals": 0, "reportsOpen": 2, "messageReportsOpen": 1,
#       "disputesToDecide": null, "payoutsFailed": null, ..., "generatedAt": "..." }

# 5. Un geste journalisé : décider d'un signalement (permission reports.review)
curl -s -X PATCH "$BASE/admin/reports/66e4..." -b admin.txt -H "Content-Type: application/json" \
-d '{"decision": "REVIEWED", "note": "Annonce vérifiée, prix conforme ; signalement infondé."}'
# → 200 { "id": "66e4...", "status": "REVIEWED" } · 409 déjà décidé · 403 { code: "ADMIN_PERMISSION_DENIED", permission: "reports.review" }

# 5 bis. Un geste en deux temps : SUPPORT propose une sanction, MEDIATOR / SUPER_ADMIN l'applique
curl -s -X POST "$BASE/admin/users/665f.../suspension/propose" -b admin.txt -H "Content-Type: application/json" \
-d '{"level": "RESTRICTED", "reason": "Trois signalements SCAM concordants sur des annonces différentes en 7 jours."}'
# → 200 { "ok": true, "proposedAt": "..." }
# (puis, avec un profil MEDIATOR) POST /admin/users/665f.../suspension { "level": "RESTRICTED", "reason": "...", "until": "2026-10-06T00:00:00.000Z" }

# 6. Le journal (permission audit.read) – la ligne du geste y est écrite DANS LA MÊME TRANSACTION que le geste
curl -s "$BASE/admin/audit" -b admin.txt
# → 200 { "items": [ { "id": "...", "at": "...", "admin": "support@...", "action": "REPORT_REVIEWED", "targetType": "REPORT", "targetId":
"66e4...", "before": null, "after": {...}, "ip": "...", ... }, ... ], "nextCursor": ... }
```

Les routes admin sont réparties par service propriétaire (deal-service pour les litiges et l'argent, trip-service pour les trajets et billets, message-service pour les conversations et signalements de messages, auth-service pour le reste) mais le client n'en sait rien : le gateway route sur le préfixe. Chaque écriture admin exige un **motif** (souvent ≥ 20 caractères) et produit une ligne de journal ; les lectures sensibles (fiche d'un membre, fil d'une conversation, drill-down portant des personnes) sont journalisées aussi. Les exports CSV sont encadrés : opérationnels (identifiants seulement) pour `exports.operational`, personnels pour `exports.personal` (SUPER_ADMIN / PRIVACY, motif obligatoire).

4. Catalogue des codes d'erreur métier

Les codes sont **stables** et en anglais ; ils voyagent dans `details.code` (erreurs passées par `errorMiddleware`), dans `code` (réponses directes des middlewares et du gateway) ou dans `errors.<champ>` (validation de formulaire). Le client les traduit ; il n'affiche jamais `message`. La colonne « Où » indique le service et le statut HTTP.

4.1 Demande de réservation (details.type: "booking", 409, deal-service)

Code	Cause	Remède côté client
QUOTE_DIVERGENCE	Le total recalculé par le serveur diffère de <code>expectedTotalCents</code> (prix du trajet modifié, paramètres de plateforme changés, saisie altérée).	Rafraîchir le trajet et les paramètres de prix, recalculer, ré-afficher le nouveau total, laisser l'utilisateur reconfirmer. Ne jamais reprendre avec l'ancien montant.
CAPACITY_EXCEEDED	Plus assez de kilos disponibles sur le trajet (<code>remainingKg</code>).	Proposer un poids inférieur ou un autre trajet ; recharger la carte du trajet.
FAMILY_REFUSED	Le Voyageur refuse cette famille de colis (<code>familyConditions</code> , mode REFUSE).	Griser la famille dans le wizard (les conditions sont dans le DTO public du trajet).
TRIP_NOT_BOOKABLE	Trajet non publié, parti, en pause, masqué ou supprimé.	Retour à la recherche.
OWN_TRIP	L'Expéditeur est le propriétaire du trajet.	Masquer le bouton « Réserver » sur ses propres trajets.
PAYMENT_NOT_AUTHORIZED	L'intention n'est pas au statut « autorisée » chez le fournisseur (confirmation client non faite, carte refusée).	Relancer la confirmation du paiement, puis rejouer <code>POST /deals</code> .
PAYMENT_MISMATCH	Le montant ou la devise de l'intention ne correspondent pas au devis.	Recréer une intention (<code>POST /deals/payment-intents</code>) puis <code>POST /deals</code> .
PAYMENT_ALREADY_USED	L'intention a déjà servi à créer un deal.	Relire <code>GET /me/bookings</code> : le deal existe sûrement déjà.
NEW_ACCOUNT_CAP	Plafond progressif d'un compte récent (D71) : <code>details.cap</code> ≤ <code>DECLARED_VALUE</code> , <code>WEIGHT</code> , <code>SHIPMENTS_PER_MONTH</code> , avec <code>limit</code> et <code>value</code> .	Expliquer le plafond et sa valeur ; proposer de réduire la valeur déclarée / le poids ou d'attendre.

4.2 Cycle de vie, transport, règlement (details.type: "booking", 409, deal-service)

Code	Cause	Remède
TRANSITION_NOT_ALLOWED	La machine à états a refusé : statut, rôle ou garde (<code>details.reason</code> porte le motif), ou une écriture concurrente a gagné (verrou optimiste).	Recharger <code>GET /deals/{id}</code> et n'afficher que <code>allowedActions</code> . Ne pas réessayer sans relecture.

Code	Cause	Remède
CARRIER_ONBOARDING_REQUIRED	Porte D31 à l'acceptation : profil Voyageur incomplet ou Stripe non prêt.	Emmener le Voyageur terminer l'onboarding (§3.2), puis réessayer.
PAYMENT_STATE_CONFLICT	L'état du paiement chez le fournisseur interdit l'opération (capture impossible, remboursement impossible).	Informé ; pour une acceptation, l'Expéditeur doit refaire une demande ; sinon contacter le support avec l' <code>x-correlation-id</code> .
DELIVERY_CODE_INVALID	Mauvais code de livraison ; <code>details.attemptsLeft</code> .	Afficher les essais restants ; inviter à revérifier avec le destinataire.
DELIVERY_LOCKED	3 échecs → verrou ; <code>details.lockedUntil</code> .	Désactiver la saisie jusqu'à <code>lockedUntil</code> ; proposer à l'Expéditeur de régénérer le code.
DELIVERY_CODE_UNAVAILABLE	Deal antérieur au chantier B3, sans code.	Support.
TRACKING_STEP_NOT_ALLOWED	Jalon hors séquence, doublon, ou deal non <code>PICKED_UP</code> .	Recharger <code>trackingEvents</code> et ne proposer que le jalon suivant.
CODE_REGENERATION_LIMIT	5 régénérations atteintes, ou deal non <code>PICKED_UP</code> .	Masquer le bouton ; support si nécessaire.
TRACKING_NOT_AVAILABLE	Lien de suivi demandé avant l'acceptation ou après une fin sans livraison.	Masquer « Partager le suivi » hors des statuts <code>ACCEPTED</code> , <code>PICKED_UP</code> , <code>DELIVERED</code> , <code>COMPLETED</code> .

4.3 Compte, session, sudo (auth-service)

Code	Où	Cause	Remède
ACCOUNT_DELETED	401 middleware	Compte effacé (RGPD).	Déconnecter, purger le stockage local.
ACCOUNT_SUSPENDED	401 middleware	Sanction <code>SUSPENDED</code> .	Déconnecter ; page d'information avec l'adresse du support.
ACCOUNT_RESTRICTED	403 <code>requireActiveAccount</code>	Sanction <code>RESTRICTED</code> (ou <code>SUSPENDED</code>) sur une route de création.	Expliquer que la publication / réservation est suspendue ; les deals en cours continuent.
SUDO_REQUIRED	403, <code>details</code> (+ <code>windowMinutes</code>)	Geste sensible sans fenêtre sudo.	Ouvrir la porte (<code>/auth/me/sudo/request + /verify</code>) puis rejouer. ■ exposition en production à vérifier (§2.3).
OTP_INCORRECT / OTP_INVALIDATED / OTP_LOCKED / OTP_EXPIRED	401 / 429, <code>details.type: "otp"</code> (<code>attemptsLeft</code> , <code>locked</code> , <code>lockUntilSeconds</code> , <code>otpInvalidated</code>)	Code faux, code invalidé après 5 échecs, verrou par palier (1 min / 30 min / 24 h), code périmé.	Construire le message depuis les champs ; proposer « renvoyer un code » quand <code>otpInvalidated</code> ou <code>OTP_EXPIRED</code> ; compte à rebours sur <code>lockUntilSeconds</code> .
PASSWORD_CONTAINS_PERSONAL_INFO, PASSWORD_SAME_AS_CURRENT	400, <code>details.type: "password"</code>	Règles de force / de nouveauté.	Message de règle sous le champ.
EMAIL_ALREADY_USED, EMAIL_SAME, EMAIL_CHANGE_EXPIRED	400 / 409	Changement d'email : adresse prise, identique, ou demande périmée (10 min).	Proposer une autre adresse ; relancer la demande.
LOCALE_UNSUPPORTED	400, <code>details.type: "locale"</code>	Locale hors <code>SUPPORTED_LOCALES</code> .	N'envoyer que <code>fr</code> / <code>en</code> .
GOOGLE_TOKEN_INVALID, GOOGLE_EMAIL_UNVERIFIED, GOOGLE_NOT_CONFIGURED, CONSENT_REQUIRED	401 / 400 / 200, <code>details.type: "oauth"</code> ou <code>status</code>	Jeton Google invalide, email non vérifié chez Google, client non configuré, consentement manquant pour un nouveau compte.	Redemander le jeton ; afficher l'écran de consentement puis rappeler avec <code>consent</code> .
STRIPE_ACCOUNT_MISSING	409	Tableau de bord Stripe demandé sans compte Connect.	Envoyer vers l'onboarding.
ERASURE_BLOCKED	409 (corps <code>ErasureBlockedResponse</code>)	Effacement impossible : <code>blockers</code> ∈ <code>ACTIVE_DEAL</code> , <code>PENDING_REQUEST</code> , <code>PAYOUT_PENDING</code> , <code>RETENTION_HELD</code> , <code>PUBLISHED_TRIP</code> , <code>ADMIN_ACCOUNT</code> + <code>counts</code> .	Lister les bloqueurs traduits avec l'action qui les lève ; <code>GET /auth/me/erasure/blockers</code> pour anticiper.
EXPORT_RATE_LIMITED	400 <code>data-export</code> , <code>details.code</code> + <code>details.nextAt</code>	Un export a déjà été produit dans l'intervalle minimal (un par période).	Proposer de réutiliser le fichier déjà téléchargé ; afficher <code>nextAt</code> .
OWN_TARGET, REASON_NOT_ALLOWED	400 <code>POST /reports</code>	Signalement de sa propre cible ; motif hors liste pour ce type de cible.	Masquer le bouton sur ses propres contenus ; ne proposer que les motifs de la cible.

4.4 Trajets et favoris (trip-service)

Code	Où	Cause	Remède
OWN_TRIP, TRIP_NOT_FAVORITABLE	409, <code>details.type: "favorite"</code>	Mise en favori de son propre trajet ; trajet non publié.	Masquer l'étoile.
(messages de la machine, sans code)	400	Transition refusée (<code>canPerform</code>), trajet introuvable, non-propre, portes de publication.	Lire message (■ sémantique 403/404 à venir) ; recharger <code>allowedActions</code> de <code>GET /trips/my</code> .

4.5 Messagerie (message-service)

Code	Où	Cause	Remède
DELIVERY_CODE_IN_MESSAGE	400, details.code	Un groupe de six chiffres du message correspond au code de livraison du deal.	Refuser l'envoi ; rappeler que le code se donne de vive voix. ■ exposition en production (§2.3).
access.reason : NOT_ACCEPTED_YET , DISPUTE_OPEN , DEAL_CLOSED , WRITE_WINDOW_OVER	200 (champ du DTO) puis 400 « read-only (...) » à l'écriture	Fenêtre d'écriture fermée.	Désactiver la saisie avant l'envoi, avec le motif traduit et writeClosesAt .
T00_SOON , T00_FAR , WINDOW_T00_LONG , END_BEFORE_START , INVALID_DATES	400 « Invalid meeting slot (...) »	Créneau de rendez-vous hors règles (≥ 30 min, ≤ 90 j, ≤ 12 h).	Valider le créneau dans le formulaire avec les mêmes bornes.
NOT_PROPOSED , OWN_PROPOSAL	400 « This meeting cannot be accepted (...) »	Rendez-vous déjà traité ; on n'accepte pas sa propre proposition.	Recharger le fil.
T00_EARLY (+ opensAt dans le message), NO_ANCHOR	400, details.code	Numéro demandé trop tôt ; aucun rendez-vous ni date de départ.	Afficher phone.opensAt du fil ; proposer de fixer un rendez-vous.
OWN_MESSAGE , NOT_A_TEXT	400 report	On ne signale que les textes de l'autre partie.	Masquer l'action.
(409 sans code)	report	Déjà signalé par cet utilisateur.	Marquer « signalé ».

4.6 Gateway et back-office

Code	Où	Cause	Remède
MAINTENANCE	503 gateway, Retry-After: 300	Mode lecture seule sur une écriture hors /api/auth/ , /api/admin/ .	Bandeau messages . <locale> ; conserver la saisie ; réessayer après retryAfterSeconds .
(429 { error })	gateway	100 requêtes / 15 min / IP.	Attendre RateLimit-Reset ; réduire les appels.
ADMIN_PERMISSION_DENIED (+ permission)	403 requireAdminPermission	Profil admin sans la permission de la route.	Masquer l'action selon adminRoles de GET /admin/me ; ne jamais retenter.
TRANSITION_NOT_ALLOWED (médiation)	409 deal-service admin	Litige déjà décidé, deal non DISPUTED , déclaration déjà enregistrée.	Recharger le dossier.
Rapprochement : INTENT_NOT_FOUND , CAPTURE_NOT_RECORDED , CAPTURE_RECORDED_NOT_LIVE , REFUND_NOT_RECORDED , REFUND_RECORDED_NOT_LIVE , TRANSFER_MISSING , TRANSFER_AMOUNT_MISMATCH , TRANSFER_REVERSED_NOT_MARKED , TRANSFER_MARKED_REVERSED_BUT_LIVE_OK	200 (divergences[]) de POST /admin/deals/{id}/money/reconcile)	Écarts entre la base et le fournisseur, lecture seule .	Afficher ; la correction est un geste séparé (rejeu, renversement, remboursement manuel).

5. Webhooks entrants et événements sortants

5.1 Webhook Stripe — POST :6003/webhooks/stripe (direct, jamais via le gateway)

Le webhook est **la source de vérité de l'état du paiement** : entre la base et Stripe, c'est Stripe qui a l'argent, et l'état de Yamba converge vers le sien (D40). Contraintes de câblage, visibles dans apps/deal-service/src/main.ts :

- la route est montée **avant** express.json() , avec express.raw({ type: "application/json" }) : la signature stripe-signature porte sur les **octets bruts** du corps, qu'un JSON re-sérialisé casserait ; c'est pourquoi le gateway (qui parse et re-séréalise) n'est jamais sur le chemin ;
- en développement : stripe listen --forward-to localhost:6003/webhooks/stripe ;
- deux endpoints Stripe, une URL (A87)** : les événements de la plateforme (STRIPE_WEBHOOK_SECRET) et ceux des comptes connectés (STRIPE_CONNECT_WEBHOOK_SECRET) ; la signature est essayée avec l'un puis l'autre.

Réponse	Sens
200 { received: true }	Traité (ou ignoré volontairement) : Stripe ne renverra pas.
400	En-tête absent ou signature invalide : pas de retry utile.
501	STRIPE_WEBHOOK_SECRET absent (fournisseur FAKE) : l'endpoint existe mais refuse plutôt que d'accepter sans signature.
500	Échec transitoire (base indisponible) : Stripe réessaie , c'est le filet voulu.

Événements traités (tout autre type répond 200 sans effet) :

Type Stripe	Effet dans Yamba
payment_intent.canceled	L'empreinte est morte (expiration ~7 j, annulation fournisseur) : un Booking PENDING qui la porte est annulé par SYSTEM via la machine à états. Idempotent.
payment_intent.amount_capturable_updated	Accusé simple : l'autorisation est posée. La création du deal reste pilotée par POST /deals (D37).

Type Stripe	Effet dans Yamba
account.updated (Connect)	Les drapeaux du Voyageur (chargesEnabled, payoutsEnabled, detailsSubmitted) suivent Stripe sans repasser par l'onboarding ; compte devenu prêt → ses versements bloqués repartent tout de suite.
transfer.reversed	Le transfert au Voyageur a été renversé : le deal est marqué (file REVERSED de /admin/finances/queue).
payout.failed (compte connecté)	La banque du Voyageur a refusé le virement : il est prévenu (RIB à corriger).

5.2 Webhook email (Resend) — POST /api/webhooks/email/resend

Passe par le gateway (/api/webhooks/email/* → notification-service /webhooks/email/*), le corps brut étant conservé par express.json({ verify }) pour la **signature Svix** (svix-id, svix-timestamp, svix-signature, secret RESEND_WEBHOOK_SECRET). Réponses : 503 si le secret n'est pas configuré ; 401 { message, reason } si la signature est invalide ; 200 { ok, status, suppressed } sinon — **idempotent** : un événement inconnu ou déjà appliqué répond 200 sans effet ({ ok: true, ignored: <type> }).

Type Resend	Effet
email.delivered	EmailDelivery.status = DELIVERED (par providerMessageId).
email.bounced	BOUNCED ; si le rebond est dur , l'adresse du compte est mise sur la liste de suppression (User.emailSuppressedAt, raison HARD_BOUNCE).
email.complained	COMPLAINED ; suppression (COMPLAINT).
autres (email.sent, email.opened, email.clicked, email.delivery_delayed)	Ignorés.

Conséquence pour tout flux d'email : **chaque résolveur de destinataires doit ignorer isDeleted et emailSuppressedAt** — un nouveau flux qui l'oublie est un bug. Le support peut lever une suppression (DELETE /admin/users/{id}/email-suppression).

5.3 Événements sortants : outbox, topics, garanties

Aucune transition n'existe sans son événement : **l'événement est écrit dans la collection OutboxEvent dans la même transaction Mongo que le changement d'état** (D2). Un relais par service (OutboxRelay du deal-service, MessagingOutboxRelay du message-service), protégé par un bail d'exclusivité, lit les lignes non publiées **de son seul aggregateType** (booking / conversation), valide chaque ligne contre le schéma Zod de son union discriminée, et la publie sur Redpanda (Kafka) :

Topic	Producteur	Clé de partition	Contrat	Événements
booking-events	deal-service	aggregateId (= id du deal)	BookingDomainEventSchema	booking.requested, payment_authorized, accepted, declined, expired, cancelled, refund_issued, picked_up, pickup_refused, tracking_event, code_regenerated, delivered, completed, payout_sent, disputed, verification_reminder, rating_reminder, rating_revealed, dispute_carrier_responded, dispute_resolved
messaging-events	message-service	id de la conversation	MessagingDomainEventSchema	conversation.message_posted, meetup_proposed, meetup_accepted, phone_revealed

Un topic par **domaine**, jamais par type d'événement : l'ordre par agrégat est l'invariant, le dispatch par type est fait par les consommateurs. Les topics sont créés explicitement (12 partitions, rétention 7 jours) ; l'auto-crédation est désactivée côté cluster et côté producteur. Le message Kafka porte l'en-tête **event-id** (= _id de la ligne outbox), et sa valeur est l'événement JSON :

```
{ "aggregateType": "booking", "aggregateId": "66b1...", "eventType": "booking.picked_up",
  "occurredAt": "2026-10-12T17:52:10.412Z", "correlationId": "8b1c...", "schemaVersion": 1,
  "payload": { "bookingId": "66b1...", "tripId": "66a0...", "shipperId": "...", "carrierId": "...",
    "corridor": { "originCity": "Paris", "originCountryCode": "FR", "destinationCity": "Kinshasa", "destinationCountryCode":
      "CD" },
    "category": "CLOTHES", "categoryFamily": "CLOTHES_TEXTILE", "weightKg": 3,
    "transportCents": 3600, "totalShipperCents": 4032, "currencyCode": "EUR", "actor": "CARRIER",
    "pickedUpAt": "2026-10-12T17:52:10.412Z", "photoCount": 2 } }
```

Garanties :

- **Enveloppe commune** : aggregateType, aggregateId, occurredAt (horloge serveur à la transition), correlationId (né au gateway), schemaVersion (entier ; toute évolution incompatible = nouvelle valeur, jamais de mutation d'un schéma publié).
- **Payloads riches** : chaque événement booking porte le socle BookingEventBasePayload (corridor, catégorie, poids, montants, acteur) pour que l'historique rejoué nourrisse à jamais notifications, analytics et médiation.
- **Ce qui ne voyage jamais dans un payload** : le **code de livraison** (booking.picked_up porte photoCount, booking.code_regenerated porte des compteurs), les **coordonnées du destinataire**, les emails, le numéro de téléphone (conversation.phone_revealed porte revealedUserId, pas le numéro), le texte complet d'un message (preview ≤ 140 caractères). Le dossier de litige n'est jamais dans booking.disputed (catégorie seulement).
- **Au moins une fois + dédoublement** : le relais marque publishedAt après l'accusé du broker ; une panne entre les deux rejoue l'événement. Le consommateur (notification-service, groupes notification-service et messaging-notifications, **jamais partagés, jamais renommés**) revendique l'event-id dans ConsumedEvent (unique par [consumerGroup, eventId]) **avant** tout traitement ; un doublon est ignoré ; la matérialisation des notifications est un upsert [eventId, userId] ; l'offset n'est commité qu'après traitement ; un échec définitif marque la ligne FAILED sans bloquer la partition.
- **Poison** : une ligne outbox qui échoue à la validation ou à la publication voit attempts incrémenté ; à 10, elle est **parquée** (exclue du relais, jamais supprimée : piste d'audit) et la suite de l'agrégat continue. Une panne transitoire du broker n'incrémente pas attempts : le relais passe en backoff. Un événement parqué n'est jamais purgé par la rétention.
- **Vie sans broker** : les services démarrent sans Redpanda ; les événements s'accumulent et partent à son retour ; le consommateur retente sa connexion toutes les 5 s.

- **Analytics** : seuls les événements des membres `analyticsOptIn: true` sont projetés vers PostHog, par une **liste blanche** de champs (`analyticsEventsFor`), jamais par étalement du payload.

Ces topics sont **internes** : ils ne sont pas exposés à des intégrateurs tiers (voir les recommandations en annexe pour des webhooks sortants).

6. Bonnes pratiques d'intégration

6.1 Écrire un client : jetons, rafraîchissement, 401 / 403, reprise après 503

Le client de référence est `apps/user-ui/src/lib/api-client.ts` (axios). Ses règles valent pour tout client, mobile compris (D36) :

1. **Toujours credentials: include / un jar de cookies**, et `x-locale` sur chaque requête. Base URL = `NEXT_PUBLIC_API_BASE_URL (/api` en même origine, ou l'URL absolue du gateway).
2. **Un seul refresh à la fois**. Sur un 401 d'une requête qui exigeait une session (`requireAuth`), le client appelle `POST /auth/refresh` **une fois**, met les requêtes concurrentes **en file d'attente**, puis les rejoue toutes après succès. Une requête marquée `skipAuthRefresh` (le refresh lui-même, la connexion) n'entre jamais dans cette boucle. Une requête déjà rejouée (`_retry`) ne l'est pas deux fois.
3. **Disjoncteur de 30 secondes**. Quand un refresh échoue, le client mémorise l'instant et **rejette directement** les 401 suivants pendant 30 s sans retenter ; une page qui monte dix composants authentifiés ne génère pas dix cycles « 401 → refresh → 401 ». Un refresh réussi (ou une connexion) remet le disjoncteur à zéro (`resetAuthRefreshCircuitBreaker`). L'état « déconnecté » est signalé **une fois** globalement (événement `yamba:session-expired` → fenêtre « ta session a expiré »), jamais par un toast par écran.
4. **401 n'est pas 403**. Un 403 ne se rejoue jamais après refresh : `SUDO_REQUIRED` ouvre la porte sudo puis rejoue ; `ACCOUNT_RESTRICTED` explique ; `ADMIN_PERMISSION_DENIED` masque l'action ; les autres 403 renvoient vers la page adéquate.
5. **409 = relire avant d'agir**. Sur un deal, un 409 signifie que la réalité a changé : `GET /deals/{id}` puis rendu à partir de `allowedActions`. Le client ne « devine » jamais l'action suivante.
6. **503 MAINTENANCE** : afficher `messages.<locale>`, **conserver la saisie en local**, réessayer après `retryAfterSeconds` ou sur action de l'utilisateur ; les lectures continuent, la connexion aussi. `GET /api/maintenance (public, 200)` alimente un bandeau préventif avec `scheduledAt`.
7. **429** : respecter `RateLimit-Reset` ; mutualiser les appels d'une page (une liste de deals + les notifications + le compteur de messages non lus font trois requêtes, pas trente).
8. **Sessions visibles** : proposer l'écran « mes appareils » (`GET /auth/me/sessions`) et la déconnexion à distance ; après un changement de mot de passe ou d'email, les autres sessions sont révoquées : le client doit s'attendre à un 401 sur les autres appareils.
9. **Paiement** : le `clientSecret` sert une seule fois ; sur mobile, Payment Sheet (Apple Pay / Google Pay natifs) ; ne jamais appeler `POST /deals` avant la confirmation du paiement côté fournisseur (sinon `PAYMENT_NOT_AUTHORIZED`).
10. **Images** : téléversement direct ImageKit avec la signature de `GET /uploads/imagekit-auth` ; n'envoyer à l'API que des URL du domaine `IMAGEKIT_URL_ENDPOINT`.
11. **Corrélation** : journaliser `x-correlation-id` de chaque réponse d'erreur ; c'est la clé que le support et Sentry partagent.
12. **Mobile (D36)** : utiliser le jar de cookies natif tant que la variante « jetons dans le corps » n'est pas livrée ; prévoir le stockage sécurisé (Keychain / Keystore) du `refresh_token` le jour où elle l'est ; réutiliser `@packages/pricing` et un client **généré** depuis les cinq documents OpenAPI (D3) plutôt qu'un client écrit à la main.

6.2 Sécurité : ce qu'un client ne doit jamais faire

- **Ne jamais décider une règle métier côté client** (poids maximal, fenêtre d'annulation, droit de contester, plafond d'un compte) : afficher ce que `allowedActions`, `cancellationPreview`, `access`, `canRate`, `phone.opensAt` disent. Le serveur applique tout ; le client reflète.
- **Ne jamais écrire le code de livraison** dans un message, un email, un SMS, un presse-papiers partagé, une notification push, un journal : il se donne de vive voix. Le message-service refuse d'ailleurs un texte qui le contient.
- **Ne jamais exposer un tel: direct** avant l'ouverture du numéro ; le bouton « Appeler » mène au fil (`?focus=phone`).
- **Ne jamais stocker les jetons ailleurs que dans les cookies** `httpOnly` (web) ou le stockage sécurisé de l'OS (mobile) ; ne jamais les mettre dans une URL ; ne jamais les journaliser.
- **Ne jamais appeler un service par son port** (`:6001`...) depuis un client : seul le gateway applique CORS, limiteur, maintenance et corrélation. Exception unique : le webhook Stripe, qui est un appel serveur à serveur.
- **Ne jamais recréer un deal après une réponse perdue** : relire les listes.
- **Ne jamais afficher message brut** ni interpréter des textes anglais : traduire les codes.
- **Ne jamais interroger un profil ou un trajet par un identifiant deviné** en espérant un 403 « révélateur » : la plateforme répond 404 sans distinguer inexistant et invisible.
- **Ne jamais contourner la porte sudo** en mémorisant un code ; un code par geste, valable 15 minutes pour cette session.
- Côté admin : ne jamais partager une session admin entre deux personnes (le journal porte l'auteur et l'IP) ; ne jamais copier un code de livraison depuis la base — l'admin ne l'affiche nulle part par décision (D43).

6.3 Versionnement et compatibilité

- **Le contrat est le schéma Zod** (`@packages/api-contracts`) ; les documents OpenAPI en sont dérivés et diffés en CI. Une modification de contrat est donc toujours visible dans une PR, dans les cinq `openapi.json`.
- **Règles suivies aujourd'hui** : ajouter un champ **optionnel** à une réponse ou à une requête est compatible ; renommer, retirer, ou rendre obligatoire ne l'est pas et se fait par un nouveau champ plus une dépréciation. Les DTO par rôle sont des **listes blanches** : un champ nouveau doit être **ajouté explicitement** à la vue qui le mérite (jamais par étalement).
- **Événements** : `schemaVersion` par événement ; un consommateur tolère les champs absents des événements anciens (ex. `reason` de `booking.payout_sent` absent = `DELIVERY`).
- **Énumérations fermées** (statuts, motifs, bloqueurs, codes) : un client doit prévoir un **cas par défaut** pour une valeur inconnue plutôt que planter — les listes s'allongent (`ReportReason`, `ErasureBlocker`, `NotificationType` est d'ailleurs `BookingEventType | string`).
- **Numéros de version des documents** (`1.0.0`, `0.3.0`, `0.1.0`) : ils datent la surface, ils ne promettent pas encore une politique de rupture ; aucune version n'est portée par l'URL (`/api/v1`) ni par un en-tête. Voir les recommandations en annexe.

- **Portes connues qui feront bouger le contrat** : sémantique 403/404 du trip-service, jetons dans le corps pour le mobile, `safeParse` des schémas d'auth-service, type des details de SUDO_REQUIRED et du message-service, route `/docs` du message-service.

7. Référence exhaustive des endpoints

La référence ci-dessous est **générée** depuis les cinq documents OpenAPI 3.1 (173 endpoints) et insérée à l'assemblage du livrable. Les chemins sont ceux **du service** ; préfixer par `/api` pour l'URL client (§1.2). Les schémas sont listés à la fin de chaque service.

auth-service — port 6001

Comptes, sessions, profils, alertes de route, signalements, back-office. Document vivant : `http://localhost:6001/docs` (Scalar) et `apps/auth-service/openapi.json`. Via le gateway : `http://localhost:8080/api` (préfixes ci-dessous).

auth-service › auth

Registration (email OTP), login, Google, refresh, logout, password reset

POST `/auth/register`

Start a registration — OTP sent by email

operationId : `register` · Authentification : aucune (public)

Consent (terms + privacy versions) is mandatory. The pending registration lives in Redis for 10 minutes. An existing email answers 409.

Corps (requis) : `MemberRegisterRequest`

Code	Réponse
200	<code>RegistrationStartedResponse</code> — OTP sent
400	<code>ErrorResponse</code> — Invalid request (<code>ValidationError</code> — details.errors per field when available)
409	<code>ErrorResponse</code> — Conflict (<code>ConflictError</code>)
429	<code>ErrorResponse</code> — Too many attempts — OTP lock (<code>RateLimitError</code> , security-alert email after 10 failures)
500	<code>UnhandledError</code> — Unhandled server error

POST `/auth/register/verify`

Verify the OTP and create the account

operationId : `verifyRegistration` · Authentification : aucune (public)

10 wrong codes lock the token for 30 minutes and send a security-alert email. On success: User + `ConsentLog`, welcome email.

Corps (requis) : `VerifyRegistrationRequest`

Code	Réponse
201	<code>SuccessMessageResponse</code> — Done
400	<code>ErrorResponse</code> — Invalid request (<code>ValidationError</code> — details.errors per field when available)
401	<code>ErrorResponse</code> — Wrong or expired OTP
429	<code>ErrorResponse</code> — Too many attempts — OTP lock (<code>RateLimitError</code> , security-alert email after 10 failures)
500	<code>UnhandledError</code> — Unhandled server error

POST `/auth/register/resend`

Send the OTP again (same token)

operationId : `resendRegistrationOtp` · Authentification : aucune (public)

Corps (requis) : `VerificationTokenRequest`

Code	Réponse
200	<code>RegistrationStartedResponse</code> — OTP sent again
400	<code>ErrorResponse</code> — Invalid request (<code>ValidationError</code> — details.errors per field when available)
404	<code>ErrorResponse</code> — Not found (<code>NotFoundError</code>)
429	<code>ErrorResponse</code> — Too many attempts — OTP lock (<code>RateLimitError</code> , security-alert email after 10 failures)
500	<code>UnhandledError</code> — Unhandled server error

POST `/auth/register/cancel`

Cancel a pending registration

operationId : `cancelRegistration` · Authentification : aucune (public)

Corps (requis) : `VerificationTokenRequest`

Code	Réponse
200	SuccessMessageResponse — Done
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
500	UnhandledError — Unhandled server error

POST `/auth/login`

Login with email + password

operationId : `login` · Authentication : aucune (public)

Sets access_token and refresh_token cookies; rememberMe = 30-day refresh (A62). A session record (device, ip, user agent) is written (D65). Suspended account → 401.

Corps (requis) : `MemberLoginRequest`

Code	Réponse
200	MemberLoginResponse — Logged in
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Invalid credentials or suspended account
500	UnhandledError — Unhandled server error

POST `/auth/google`

Sign in with Google (D47)

operationId : `googleSignIn` · Authentication : aucune (public)

Verifies the Google ID token. Existing email → linked; new account → needs consent, otherwise status `CONSENT_REQUIRED` without creating anything.

Corps (requis) : `GoogleSignInRequest`

Code	Réponse
200	GoogleSignInResponse — Logged in, or consent required
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Invalid Google token
500	UnhandledError — Unhandled server error

POST `/auth/refresh`

Rotate the session (refresh cookie)

operationId : `refreshTokens` · Authentication : aucune (public)

Reads refresh_token, checks the jti in Redis, issues a new pair. The front replays queued requests after a 401 (api-client circuit breaker).

Code	Réponse
200	SuccessMessageResponse — Done
401	UnauthorizedResponse — Refresh token missing, revoked or expired
500	UnhandledError — Unhandled server error

POST `/auth/logout`

Logout — revokes the refresh jti and clears cookies

operationId : `logout` · Authentication : aucune (public)

Code	Réponse
200	SuccessMessageResponse — Done
500	UnhandledError — Unhandled server error

POST `/auth/password/forgot`

Password reset — send an OTP

operationId : `requestPasswordReset` · Authentication : aucune (public)

Always 200, never reveals whether the account exists.

Corps (requis) : `PasswordForgotRequest`

Code	Réponse
200	MessageResponse — Sent if the account exists

Code	Réponse
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
429	ErrorResponse — Too many attempts — OTP lock (RateLimitError, security-alert email after 10 failures)
500	UnhandledError — Unhandled server error

POST /auth/password/verify

Verify the reset OTP

operationId : verifyPasswordReset · Authentication : aucune (public)

Corps (requis) : PasswordVerifyRequest

Code	Réponse
200	PasswordVerifyResponse — Verified — reset token
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	ErrorResponse — Wrong or expired OTP
429	ErrorResponse — Too many attempts — OTP lock (RateLimitError, security-alert email after 10 failures)
500	UnhandledError — Unhandled server error

POST /auth/password/resend

Send the reset OTP again

operationId : resendPasswordReset · Authentication : aucune (public)

Corps (requis) : PasswordForgotRequest

Code	Réponse
200	MessageResponse — Sent if the account exists
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
429	ErrorResponse — Too many attempts — OTP lock (RateLimitError, security-alert email after 10 failures)
500	UnhandledError — Unhandled server error

POST /auth/password/reset

Set the new password with the reset token

operationId : resetPassword · Authentication : aucune (public)

Password rules (length, no email inside). Other sessions are revoked; a passwordChanged email is sent.

Corps (requis) : PasswordResetRequest

Code	Réponse
200	MessageResponse — Reset
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	ErrorResponse — Reset token invalid or expired
500	UnhandledError — Unhandled server error

auth-service › me

The authenticated member: profile, preferences, sessions, sudo, GDPR

GET /auth/me

The authenticated member

operationId : getMe · Authentication : cookieAuth, bearerAuth

The User record without passwordHash, plus the effective roles.

Code	Réponse
200	MeResponse — Member
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

PATCH /auth/me/locale

Set the preferred locale (D44) — email language

operationId : updateMyLocale · Authentication : cookieAuth, bearerAuth

Corps (requis) : UpdateLocaleRequest

Code	Réponse
200	UpdateLocaleResponse — Saved
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

PATCH /auth/me/preferences

Preferences: reminder emails, locale, analytics consent (D66)

operationId : updateMyPreferences · Authentication : cookieAuth, bearerAuth

A change of analyticsOptIn is traced in ConsentLog (COOKIES).

Corps (requis) : UpdateMyPreferencesRequest

Code	Réponse
200	PreferencesResponse — Saved
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

POST /auth/me/sudo/request

Sudo — send the code by email (D65 1A)

operationId : requestSudoCode · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	SuccessMessageResponse — Done
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
429	ErrorResponse — Too many attempts — OTP lock (RateLimitError, security-alert email after 10 failures)
500	UnhandledError — Unhandled server error

POST /auth/me/sudo/verify

Sudo — open the 15-minute window for THIS session

operationId : verifySudo · Authentication : cookieAuth, bearerAuth

The window is bound to the refresh jti: another session of the same member does not inherit it.

Corps (requis) : SudoVerifyRequest

Code	Réponse
200	SudoWindowResponse — Window open
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	ErrorResponse — Wrong or expired code
429	ErrorResponse — Too many attempts — OTP lock (RateLimitError, security-alert email after 10 failures)
500	UnhandledError — Unhandled server error

GET /auth/me/sudo

Sudo — is the window open, until when

operationId : getSudoStatus · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	SudoStatus — Status
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

GET /auth/me/sessions

Connected devices (D65 2A)

operationId : listMySessions · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	MemberSessionsResponse — Sessions, current first
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

DELETE /auth/me/sessions

Revoke every OTHER session

operationId : revokeMyOtherSessions · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	RevokeSessionResponse — Revoked count
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

DELETE /auth/me/sessions/{jti}

Revoke one session (current allowed — cookies cleared)

operationId : revokeMySession · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
jti	path	oui	string	Session id (refresh token jti)

Code	Réponse
200	RevokeSessionResponse — Revoked
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /auth/me/password

Change the password (sudo)

operationId : changeMyPassword · Authentication : cookieAuth, bearerAuth

Requires the sudo window (403 SUDO_REQUIRED). Other sessions revoked, passwordChanged email. A Google-only account sets its first password.

Corps (requis) : ChangePasswordRequest

Code	Réponse
200	PasswordChangedResponse — Changed
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
403	ErrorResponse — Forbidden — details.code = SUDO_REQUIRED when the sudo window is closed (D65)
500	UnhandledError — Unhandled server error

POST /auth/me/email/request

Change the email — code sent to the NEW address (sudo)

operationId : requestEmailChange · Authentication : cookieAuth, bearerAuth

Corps (requis) : RequestEmailChange

Code	Réponse
200	EmailChangeRequestedResponse — Code sent
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
403	ErrorResponse — Forbidden — details.code = SUDO_REQUIRED when the sudo window is closed (D65)
409	ErrorResponse — Email already used
500	UnhandledError — Unhandled server error

POST /auth/me/email/confirm

Confirm the new email with the code

operationId : confirmEmailChange · Authentication : cookieAuth, bearerAuth

The old address is warned (emailChanged), other sessions are revoked.

Corps (requis) : ConfirmEmailChange

Code	Réponse
200	EmailChangeConfirmedResponse — Changed
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)

Code	Réponse
401	ErrorResponse — Wrong or expired code
409	ErrorResponse — Conflict (ConflictError)
500	UnhandledError — Unhandled server error

POST /auth/me/data-export

GDPR — download my data as JSON (sudo, D63 2A)

operationId : exportMyData · Authentication : cookieAuth, bearerAuth

Content-Disposition attachment; a DataRequest EXPORT is written.

Code	Réponse
200	DataExport — JSON export
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
403	ErrorResponse — Forbidden — details.code = SUDO_REQUIRED when the sudo window is closed (D65)
500	UnhandledError — Unhandled server error

GET /auth/me/erasure/blockers

GDPR — what prevents erasure right now (D63 3A)

operationId : getMyErasureBlockers · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	objet { blockers, counts } — Blockers (empty = erasable)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

POST /auth/me/erasure

GDPR — erase my account now (sudo, D63 3A)

operationId : eraseMyAccount · Authentication : cookieAuth, bearerAuth

ONE transaction anonymising the User field by field; bookings, disputes, reviews and messages are kept. 409 ERASURE_BLOCKED with the closed list of blockers.

Corps (requis) : EraseMyAccountRequest

Code	Réponse
200	ErasedResponse — Erased — session ended
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
403	ErrorResponse — Forbidden — details.code = SUDO_REQUIRED when the sudo window is closed (D65)
409	ErasureBlockedResponse — Blocked by a live deal
500	UnhandledError — Unhandled server error

GET /auth/me/profile

My editable profile (D67)

operationId : getMyProfile · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	MyProfileResponse — Profile
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

PATCH /auth/me/profile

Update names, birth date, carrier display name / bio, visibilities (D67 1A)

operationId : updateMyProfile · Authentication : cookieAuth, bearerAuth

Errors per field in details.errors (INVALID_DATE, IN_THE_FUTURE, TOO_YOUNG, NO_CARRIER_PAGE). The public slug never changes.

Corps (requis) : UpdateMyProfileRequest

Code	Réponse
200	MyProfileResponse — Saved
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)

Code	Réponse
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

POST /auth/me/avatar

Declare the avatar uploaded to ImageKit (D67 2A)

operationId : setMyAvatar · Authentication : cookieAuth, bearerAuth

The URL must belong to IMAGEKIT_URL_ENDPOINT; the previous file is deleted.

Corps (requis) : SetMyAvatarRequest

Code	Réponse
200	MyProfileResponse — Saved
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

DELETE /auth/me/avatar

Remove the avatar

operationId : deleteMyAvatar · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	MyProfileResponse — Removed
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

auth-service › reports

Report a trip or a member (D68)

POST /reports

Report a trip or a member (D68 1A)

operationId : createReport · Authentication : cookieAuth, bearerAuth

targetRef = trip id or member public slug. 400 OWN_TARGET / REASON_NOT_ALLOWED, 404 invisible target (deleted trip, erased member, hidden page), 409 open duplicate. Acknowledgement email to the reporter; the target never learns who reported.

Corps (requis) : CreateReportRequest

Code	Réponse
201	CreateReportResponse — Recorded
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
404	ErrorResponse — Not found (NotFoundError)
409	ErrorResponse — Conflict (ConflictError)
500	UnhandledError — Unhandled server error

auth-service › carrier

Carrier onboarding and Stripe Connect

POST /carrier/onboarding/profile

Carrier onboarding — step PROFILE

operationId : saveCarrierProfile · Authentication : cookieAuth, bearerAuth

Creates or updates the CarrierPage (name, bio, phone, primary address).

Corps (requis) : CarrierProfileRequest

Code	Réponse
200	CarrierProfileResponse — Saved
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

POST /carrier/onboarding/stripe

Carrier onboarding — Stripe Connect account link

operationId : createStripeConnectLink · Authentication : cookieAuth, bearerAuth

Creates the Express account on first call; returns a single-use onboarding URL.

Code	Réponse
200	StripeLinkResponse — Onboarding URL
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

GET /carrier/onboarding/stripe/status

Stripe account status (charges / payouts)

operationId : getStripeStatus · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	StripeStatusResponse — Status
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

POST /carrier/onboarding/complete

Carrier onboarding — complete

operationId : completeCarrierOnboarding · Authentication : cookieAuth, bearerAuth

Grants the CARRIER role and sends the onboarding-complete email.

Code	Réponse
200	SuccessMessageResponse — Done
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

POST /carrier/stripe/dashboard-link

Stripe Express dashboard link (sudo, A84)

operationId : createStripeDashboardLink · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	StripeLinkResponse — Dashboard URL
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
403	ErrorResponse — Forbidden — details.code = SUDO_REQUIRED when the sudo window is closed (D65)
409	ErrorResponse — No Stripe account yet
500	UnhandledError — Unhandled server error

auth-service · saved-routes

Route alerts (max 20 per member, 6-month expiry)

POST /saved-routes

Create a route alert (6-month expiry)

operationId : createSavedRoute · Authentication : cookieAuth, bearerAuth

Corps (requis) : SavedRouteRequest

Code	Réponse
201	SavedRouteResponse — Created
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
409	ErrorResponse — Limit of 20 alerts reached
500	UnhandledError — Unhandled server error

GET /saved-routes

My route alerts

operationId : listSavedRoutes · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	SavedRoutesResponse — Alerts
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

PATCH /saved-routes/{id}

Update a route alert

operationId : updateSavedRoute · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Saved route id

Corps (requis) : SavedRouteRequest

Code	Réponse
200	SavedRouteResponse — Updated
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

DELETE /saved-routes/{id}

Delete a route alert

operationId : deleteSavedRoute · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Saved route id

Code	Réponse
200	SuccessMessageResponse — Done
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /saved-routes/{id}/extend

Extend a route alert by 6 months

operationId : extendSavedRoute · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Saved route id

Code	Réponse
200	SavedRouteResponse — Extended
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

auth-service › users

Public profiles and follows

GET /users/{slug}/public

Public profile (D28 / D29 / D67)

operationId : getUserPublic · Authentication : aucune (public)

Optional auth (follow state). 404 when the member is erased or has hidden the page — unless the caller is the owner (hidden: true).

Paramètre	Dans	Requis	Type	Description
slug	path	oui	string	Member public slug (immutable, D28)

Code	Réponse
200	PublicUserResponse — Profile
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

GET /users/{slug}/public/reviews

Revealed reviews of a member, paginated

operationId : listUserPublicReviews · Authentication : aucune (public)

Paramètre	Dans	Requis	Type	Description
slug	path	oui	string	Member public slug (immutable, D28)
kind	query	non	string	Role reviewed
limit	query	non	integer	Page size
cursor	query	non	ObjectId	Id of the last item of the previous page

Code	Réponse
200	PublicReviewsResponse — Reviews
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

GET /users/{slug}/public/trips

Upcoming published trips of a carrier, paginated

operationId : listUserPublicTrips · Authentication : aucune (public)

Paramètre	Dans	Requis	Type	Description
slug	path	oui	string	Member public slug (immutable, D28)
limit	query	non	integer	Page size
cursor	query	non	ObjectId	Id of the last item of the previous page

Code	Réponse
200	PublicTripsResponse — Trips
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /users/{slug}/follow

Follow a member (D46)

operationId : followUser · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
slug	path	oui	string	Member public slug (immutable, D28)

Corps (facultatif) : FollowPreferencesRequest

Code	Réponse
200	FollowResponse — Followed
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

DELETE /users/{slug}/follow

Unfollow

operationId : unfollowUser · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
slug	path	oui	string	Member public slug (immutable, D28)

Code	Réponse
200	SuccessMessageResponse — Done
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

PATCH /users/{slug}/follow

Follow preferences — notify on next trip

operationId : updateFollowPreferences · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
slug	path	oui	string	Member public slug (immutable, D28)

Corps (requis) : FollowPreferencesRequest

Code	Réponse
200	FollowResponse — Saved
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

GET /me/following

Members I follow — those with an upcoming trip first

operationId : listMyFollowing · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	FollowingResponse — Following
401	UnauthorizedResponse — Missing, invalid or expired token; suspended account (isAuthenticated)
500	UnhandledError — Unhandled server error

auth-service › admin-auth

ADMIN two-step login (password + TOTP), sessions, invitations

POST /auth/admin/login

ADMIN login step 1 — password

operationId : adminLogin · Authentication : aucune (public)

Requires User.adminRoles. Sets the admin_preauth cookie (5 min); next = TOTP or SETUP.

Corps (requis) : AdminLoginRequest

Code	Réponse
200	AdminLoginResponse — Pre-authenticated
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — Invalid credentials or not an admin
500	UnhandledError — Unhandled server error

POST /auth/admin/totp/setup

TOTP setup — secret and otpauth URL (pre-auth cookie)

operationId : adminTotpSetup · Authentication : aucune (public)

Code	Réponse
200	AdminTotpSetupResponse — Secret
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
500	UnhandledError — Unhandled server error

POST /auth/admin/totp/enable

TOTP enable — first code, returns the backup codes ONCE

operationId : adminTotpEnable · Authentication : aucune (public)

Corps (requis) : AdminTotpCodeRequest

Code	Réponse
200	AdminTotpEnableResponse — Enabled
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
500	UnhandledError — Unhandled server error

POST /auth/admin/totp/verify

ADMIN login step 2 – TOTP or backup code

operationId : adminTotpVerify · Authentication : aucune (public)

Opens the ADMIN session: admin_access_token / admin_refresh_token cookies (claims adm + amr pwd,totp).

Corps (requis) : AdminTotpCodeRequest

Code	Réponse
200	AdminTotpVerifyResponse — Session open
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
429	ErrorResponse — Too many attempts — OTP lock (RateLimitError, security-alert email after 10 failures)
500	UnhandledError — Unhandled server error

POST /auth/admin/refresh

Rotate the ADMIN session

operationId : adminRefresh · Authentication : aucune (public)

Code	Réponse
200	OkResponse — Done
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
500	UnhandledError — Unhandled server error

POST /auth/admin/logout

ADMIN logout

operationId : adminLogout · Authentication : aucune (public)

Code	Réponse
200	OkResponse — Done
500	UnhandledError — Unhandled server error

POST /auth/admin/invite/accept

Accept an admin invitation (public token, D56)

operationId : acceptAdminInvite · Authentication : aucune (public)

Sets the password of the invited account; the token is single-use and expires.

Corps (requis) : AcceptAdminInviteRequest

Code	Réponse
200	objet { ok, email } — Accepted
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

GET /admin/me

The ADMIN account and its profiles

operationId : getAdminMe · Authentication : adminCookieAuth · Permission admin : (session)

Code	Réponse
200	AdminMeResponse — Admin
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
500	UnhandledError — Unhandled server error

GET /admin/me/sessions

ADMIN sessions

operationId : listAdminSessions · Authentication : adminCookieAuth · Permission admin : (session)

Code	Réponse
200	objet { items } — Sessions
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)

Code	Réponse
500	UnhandledError — Unhandled server error

DELETE /admin/me/sessions/{jti}

Revoke one ADMIN session

operationId : revokeAdminSession · Authentication : adminCookieAuth · Permission admin : (session)

Paramètre	Dans	Requis	Type	Description
jti	path	oui	string	Session id (refresh token jti)

Code	Réponse
200	OkResponse — Done
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
500	UnhandledError — Unhandled server error

auth-service · admin

Back-office: KPIs, pilotage, audit, settings, status, maintenance, admins, users, privacy, reports

GET /admin/kpis

Home counters (D57) — null = not visible to this profile

operationId : getAdminKpis · Authentication : adminCookieAuth · Permission admin : kpi.read

Code	Réponse
200	AdminHomeKpis — KPIs
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

GET /admin/pilotage/series

Pilotage series (D59, 60 s Redis cache)

operationId : getPilotageSeries · Authentication : adminCookieAuth · Permission admin : pilotage.read

Paramètre	Dans	Requis	Type	Description
granularity	query	non	string	Default week
months	query	non	integer	Window (default 3 for week, 12 for month)

Code	Réponse
200	PilotageSeriesResponse — Series
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

GET /admin/pilotage/corridors

Corridors — demand vs supply (D59)

operationId : getPilotageCorridors · Authentication : adminCookieAuth · Permission admin : pilotage.read

Paramètre	Dans	Requis	Type	Description
days	query	non	integer	Window, default 30

Code	Réponse
200	CorridorsResponse — Corridors
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

GET /admin/pilotage/drilldown

Drilldown of one point of a series (D60 3A, journaled)

operationId : getPilotageDrilldown · Authentication : adminCookieAuth · Permission admin : pilotage.read

Paramètre	Dans	Requis	Type	Description
metric	query	oui	PilotageMetric	Metric
granularity	query	non	string	Default week
period	query	oui	string	Period key of the series point

Code	Réponse
200	PilotageDrilldownResponse — Items
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

GET /admin/audit

Admin journal, newest first

operationId : listAdminAudit · Authentication : adminCookieAuth · Permission admin : audit.read

Paramètre	Dans	Requis	Type	Description
cursor	query	non	ObjectId	Id of the last item of the previous page

Code	Réponse
200	AdminAuditResponse — Journal page
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

GET /admin/settings

Platform settings + catalogue (D62)

operationId : getSettings · Authentication : adminCookieAuth · Permission admin : settings.read

Code	Réponse
200	AdminSettingsResponse — Settings
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

PATCH /admin/settings

Change settings (D62 5A)

operationId : updateSettings · Authentication : adminCookieAuth · Permission admin : settings.read + per-key scope

Reason ≥ 20 chars, optimistic version. Scope enforced per key in the service: BUSINESS = SUPER_ADMIN, OPERATIONS = OPS. One SETTING_CHANGED journal line per key, email to every SUPER_ADMIN. Never retroactive.

Corps (requis) : UpdateSettingsRequest

Code	Réponse
200	SettingsWriteResponse — Changed
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
409	ErrorResponse — Version conflict
500	UnhandledError — Unhandled server error

GET /admin/settings/history

Settings change history

operationId : getSettingsHistory · Authentication : adminCookieAuth · Permission admin : settings.read

Paramètre	Dans	Requis	Type	Description
key	query	non	string	Filter on one key
cursor	query	non	ObjectId	Id of the last item of the previous page

Code	Réponse
200	SettingsHistoryResponse — History

Code	Réponse
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

POST /admin/settings/reset

Reset settings to the catalogue defaults (SETTINGS_RESET)

operationId : resetSettings · Authentication : adminCookieAuth · Permission admin : settings.read + per-key scope

Corps (requis) : ResetSettingsRequest

Code	Réponse
200	SettingsWriteResponse — Reset
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
409	ErrorResponse — Conflict (ConflictError)
500	UnhandledError — Unhandled server error

GET /admin/status

Service status page (D64 5A) — not a monitoring tool

operationId : getAdminStatus · Authentication : adminCookieAuth · Permission admin : status.read

Code	Réponse
200	AdminStatusResponse — Status
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

GET /admin/maintenance

Planned maintenance state (D64 1A)

operationId : getMaintenance · Authentication : adminCookieAuth · Permission admin : status.read

Code	Réponse
200	MaintenanceState — State
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

PUT /admin/maintenance

Switch the read-only mode (journaled, SUPER_ADMINS emailed)

operationId : updateMaintenance · Authentication : adminCookieAuth · Permission admin : maintenance.write

Corps (requis) : UpdateMaintenanceRequest

Code	Réponse
200	MaintenanceState — State
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

GET /admin/admins

Admin accounts

operationId : listAdmins · Authentication : adminCookieAuth · Permission admin : admins.manage

Code	Réponse
200	objet { items } — Admins
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

POST /admin/admins/invite

Invite an admin (D56) — email with a single-use token

operationId : inviteAdmin · Authentication : adminCookieAuth · Permission admin : admins.manage

Corps (requis) : InviteAdminRequest

Code	Réponse
200	InviteAdminResponse — Existing account: roles granted
201	InviteAdminResponse — Invited
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

PATCH /admin/admins/{id}

Change the profiles of an admin (D60 1A: union of permissions)

operationId : updateAdminRole · Authentication : adminCookieAuth · Permission admin : admins.manage

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	User id

Corps (requis) : UpdateAdminRoleRequest

Code	Réponse
200	UpdateAdminRoleResponse — Updated
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

DELETE /admin/admins/{id}

Revoke the admin profiles (sessions revoked)

operationId : revokeAdmin · Authentication : adminCookieAuth · Permission admin : admins.manage

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	User id

Code	Réponse
200	OkResponse — Done
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
404	ErrorResponse — Not found (NotFoundError)
409	ErrorResponse — Last SUPER_ADMIN or yourself
500	UnhandledError — Unhandled server error

GET /admin/users

Search members (D60 2A)

operationId : searchAdminUsers · Authentication : adminCookieAuth · Permission admin : users.read

Paramètre	Dans	Requis	Type	Description
q	query	non	string	Name, email, phone
role	query	non	string	Role
accountStatus	query	non	AccountStatus	Sanction status
carrierStatus	query	non	string	Carrier status
stripeReady	query	non	string	Connect with payouts enabled
createdFrom	query	non	string	Created after
createdTo	query	non	string	Created before
sort	query	non	string	Sort
dir	query	non	string	Direction

Paramètre	Dans	Requis	Type	Description
cursor	query	non	ObjectId	Id of the last item of the previous page
limit	query	non	integer	Page size (50)

Code	Réponse
200	AdminUsersResponse — Page
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

GET /admin/users/export

CSV export of members (SUPER_ADMIN or PRIVACY, reason ≥ 20, journaled EXPORTED)

operationId : exportAdminUsers · Authentication : adminCookieAuth · Permission admin : exports.personal

Paramètre	Dans	Requis	Type	Description
reason	query	oui	string	Why this export

Code	Réponse
200	string — text/csv (BOM, RFC 4180, formula prefixes neutralised)
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

GET /admin/users/{id}

Member file

operationId : getAdminUserFile · Authentication : adminCookieAuth · Permission admin : users.read

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	User id

Code	Réponse
200	AdminUserFile — File
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /admin/users/{id}/suspension/propose

Propose a sanction (SUPPORT / MEDIATOR)

operationId : proposeSuspension · Authentication : adminCookieAuth · Permission admin : users.suspension.propose

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	User id

Corps (requis) : ProposeSuspensionRequest

Code	Réponse
200	SuspensionProposedResponse — Proposed
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /admin/users/{id}/suspension

Apply RESTRICTED or SUSPENDED (SUPER_ADMIN) — effective through reads only

operationId : applySuspension · Authentication : adminCookieAuth · Permission admin : users.suspension.apply

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	User id

Corps (requis) : `ApplySuspensionRequest`

Code	Réponse
200	SuspensionAppliedResponse — Applied
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

DELETE `/admin/users/{id}/suspension`

Lift the sanction

operationId : `liftSuspension` · Authentication : `adminCookieAuth` · Permission admin : `users.suspension.apply`

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	User id

Corps (facultatif) : `LiftSuspensionRequest`

Code	Réponse
200	SuspensionAppliedResponse — Lifted (ACTIVE)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

DELETE `/admin/users/{id}/email-suppression`

Lift an email suppression after a bounce / complaint (D35 4A, journalé)

operationId : `unsuppressEmail` · Authentication : `adminCookieAuth` · Permission admin : `users.email.unsuppress`

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	User id

Code	Réponse
200	OkResponse — Done
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
404	ErrorResponse — Not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST `/admin/users/{id}/erase`

GDPR — erase a member on request (D63 6A, PRIVACY)

operationId : `adminEraseUser` · Authentication : `adminCookieAuth` · Permission admin : `users.erase`

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	User id

Corps (requis) : `AdminEraseUserRequest`

Code	Réponse
200	ErasedResponse — Erased
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
404	ErrorResponse — Not found (NotFoundError)
409	ErasureBlockedResponse — Blocked by a live deal
500	UnhandledError — Unhandled server error

GET /admin/privacy/requests

GDPR requests register (exports, erasures) — journaled read

operationId : listDataRequests · Authentication : adminCookieAuth · Permission admin : privacy.requests.read

Paramètre	Dans	Requis	Type	Description
cursor	query	non	ObjectId	Id of the last item of the previous page

Code	Réponse
200	DataRequestsResponse — Requests
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

GET /admin/reports

Reported trips and members (D68 3A) — priority from 3 open reports on a target

operationId : listAdminReports · Authentication : adminCookieAuth · Permission admin : reports.review

Paramètre	Dans	Requis	Type	Description
status	query	non	ReportStatus	Default OPEN

Code	Réponse
200	AdminReportsResponse — Queue
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
500	UnhandledError — Unhandled server error

PATCH /admin/reports/{id}

Decide on a report — REVIEWED / DISMISSED, journaled (no automatic sanction)

operationId : reviewAdminReport · Authentication : adminCookieAuth · Permission admin : reports.review

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Report id

Corps (requis) : ReviewReportRequest

Code	Réponse
200	objet { id, status } — Decided
400	ErrorResponse — Invalid request (ValidationError — details.errors per field when available)
401	UnauthorizedResponse — No ADMIN session: admin_access_token missing, expired or without amr pwd+totp (isAdminAuthenticated)
403	ErrorResponse — The ADMIN profile lacks the route permission (requireAdminPermission, ADMIN_PERMISSIONS matrix)
404	ErrorResponse — Not found (NotFoundError)
409	ErrorResponse — Already reviewed
500	UnhandledError — Unhandled server error

auth-service · schémas utilisés (129)

AcceptAdminInviteRequest

Champ	Type	Requis	Description
token	string	oui	
password	string	oui	

AccountStatus — enum : ACTIVE , RESTRICTED , SUSPENDED

AdminAccount

Champ	Type	Requis	Description
id	ObjectId	oui	
firstName	string	oui	
lastName	string	oui	
email	string	oui	
adminRole	AdminRole	oui	

Champ	Type	Requis	Description
adminRoles	array	oui	
totpEnabled	boolean	oui	
inviteAccepted	boolean	oui	false while the invited account has no password yet
createdAt	string (date-time)	oui	

AdminAuditItem

Champ	Type	Requis	Description
id	ObjectId	oui	
at	string (date-time)	oui	
admin	string	oui	
action	string	oui	
targetType	string	oui	
targetId	string null	oui	
before	null null	oui	
after	null null	oui	
ip	string null	oui	

AdminAuditResponse

Champ	Type	Requis	Description
items	array	oui	
nextCursor	ObjectId null	oui	

AdminEraseUserRequest — Erasure on behalf of a member (request received by email) — PRIVACY or SUPER_ADMIN, journaled

Champ	Type	Requis	Description
reason	string	oui	

AdminHomeKpis — Operational counters; null = not visible to this profile (D57)

Champ	Type	Requis	Description
disputesToDecide	integer null	oui	
retentionsHeld	integer null	oui	
ticketsToVerify	integer null	oui	
hiddenTrips	integer null	oui	
hideProposals	integer null	oui	
suspensionProposals	integer null	oui	
restrictedUsers	integer null	oui	
suspendedUsers	integer null	oui	
publishedTrips	integer null	oui	
activeDeals	integer null	oui	
payoutsFailed	integer null	oui	
payoutsReversed	integer null	oui	
manualRefundProposals	integer null	oui	
pendingAdminInvites	integer null	oui	
usersTotal	integer null	oui	
completedDeals30d	integer null	oui	
messageReports0pen	integer null	non	
reports0pen	integer null	non	
generatedAt	string (date-time)	oui	

AdminLoginRequest

Champ	Type	Requis	Description
email	string (email)	oui	
password	string	oui	

AdminLoginResponse — admin_preauth cookie set; then totp/verify (TOTP) or totp/setup + totp/enable (SETUP)

Champ	Type	Requis	Description
next	enum : TOTP, SETUP	oui	

AdminMeResponse

Champ	Type	Requis	Description
id	ObjectId	oui	
email	string	oui	
firstName	string	oui	
lastName	string	oui	
adminRole	string null	oui	
adminRoles	array	oui	
remainingBackupCodes	integer	oui	

AdminReportItem

Champ	Type	Requis	Description
id	string	oui	
targetType	ReportTargetType	oui	
targetId	string	oui	
targetLabel	string	oui	
targetOwner	object null	oui	
status	ReportStatus	oui	
reason	ReportReason	oui	
details	string null	oui	
createdAt	string (date-time)	oui	
reporter	object	oui	
openCountOnTarget	integer	oui	
priority	boolean	oui	
targetTrustLevel	TrustLevel null	oui	

AdminReportsResponse

Champ	Type	Requis	Description
items	array	oui	
total	integer	oui	

AdminRole — enum : SUPER_ADMIN, MEDIATOR, SUPPORT, FINANCE, OPS, PRIVACY

AdminRoles — array : Profils cumulés d'un compte admin (au moins un) — l'union des permissions ; normalisés côté serveur (ordre canonique, doublons ignorés)

AdminSessionItem

Champ	Type	Requis	Description
jti	string	oui	
createdAt	string (date-time)	oui	
lastActivityAt	string (date-time)	oui	
current	boolean	oui	

AdminSettingsResponse

Champ	Type	Requis	Description
values	PlatformSettingsValues	oui	
defaults	PlatformSettingsValues	oui	
version	integer	oui	
updatedAt	string null	oui	
updatedBy	object null	oui	
lastChange	SettingsLastChange null	oui	
catalog	array	oui	
fixed	array	oui	
planned	array	oui	

AdminStatusResponse — Service status page (D64 5A) — not a monitoring tool

Champ	Type	Requis	Description
at	string (date-time)	oui	
services	array	oui	
crons	array	oui	
outbox	object	oui	
emails	object	oui	
maintenance	MaintenanceState	oui	

AdminTotpCodeRequest — 6-digit TOTP or a backup code (verify)

Champ	Type	Requis	Description
code	string	oui	

AdminTotpEnableResponse — Backup codes shown ONCE

Champ	Type	Requis	Description
ok	boolean	oui	
backupCodes	array	oui	

AdminTotpSetupResponse

Champ	Type	Requis	Description
secret	string	oui	
otpauthUrl	string	oui	

AdminTotpVerifyResponse — admin_access_token / admin_refresh_token cookies set

Champ	Type	Requis	Description
ok	boolean	oui	
usedBackupCode	boolean	oui	
remainingBackupCodes	integer	oui	

AdminUserFile — Everything an operator needs on a user — never a secret, never a delivery code

Champ	Type	Requis	Description
id	ObjectId	oui	
firstName	string	oui	
lastName	string	oui	
email	string	oui	
phoneE164	string null	oui	
preferredLocale	string	oui	
emailSuppression	object null	oui	
roles	array	oui	
adminRole	AdminRole null	oui	
adminRoles	array	oui	
accountStatus	AccountStatus	oui	
suspension	object null	oui	
suspensionProposal	object null	oui	
createdAt	string (date-time)	oui	
isDeleted	boolean	oui	
isMe	boolean	oui	The admin reading the file IS this user (conflict of interest guard)
carrier	object null	oui	
shipper	object	oui	
activity	object	oui	
adminActions	array	oui	
trust	TrustAssessment null	oui	

AdminUserSummary

Champ	Type	Requis	Description
id	ObjectId	oui	

Champ	Type	Requis	Description
firstName	string	oui	
lastName	string	oui	
email	string	oui	
phoneE164	string null	oui	
roles	array	oui	
adminRole	AdminRole null	oui	
adminRoles	array	oui	
accountStatus	AccountStatus	oui	
carrierStatus	string	oui	
createdAt	string (date-time)	oui	
matchedOn	string null	oui	What the search matched: email / name / phone / dealId / ticket

AdminUsersResponse

Champ	Type	Requis	Description
items	array	oui	
total	integer	oui	
nextCursor	string null	non	C-PR7a — id du dernier élément ; absent = fin

ApplySuspensionRequest

Champ	Type	Requis	Description
level	enum : RESTRICTED, SUSPENDED	oui	
reason	string	oui	
until	string (date-time)	non	Optional end; absent = until lifted

CarrierPageDto

Champ	Type	Requis	Description
id	ObjectId	oui	
userId	ObjectId	oui	
name	string	oui	
bio	string null	oui	
phoneE164	string null	oui	
onboardingStep	string	oui	
stripeOnboardingComplete	boolean	oui	
stripeChargesEnabled	boolean	oui	
stripePayoutsEnabled	boolean	oui	
isVerified	boolean	oui	
isSuperCarrier	boolean	oui	

CarrierProfileRequest — Step PROFILE of the carrier onboarding

Champ	Type	Requis	Description
name	string	oui	
bio	string null	non	
phoneE164	string null	non	
address	object	non	

CarrierProfileResponse

Champ	Type	Requis	Description
success	boolean	oui	
message	string	oui	
carrierPage	CarrierPageDto	oui	

ChangePasswordRequest — Under sudo (D65 3A); every other session is revoked

Champ	Type	Requis	Description
newPassword	string	oui	

ConfirmEmailChange

Champ	Type	Requis	Description
code	string	oui	

CorridorStat

Champ	Type	Requis	Description
key	string	oui	ville>ville normalisées
originCity	string	oui	
originCountryCode	string null	oui	
destinationCity	string	oui	
destinationCountryCode	string null	oui	
tripsPublished	integer	oui	
requests	integer	oui	
accepted	integer	oui	
acceptanceRatePct	number null	oui	
avgPricePerKgCents	integer null	oui	
currencyCode	string null	oui	
disputes	integer	oui	
views	integer	oui	Vues des pages de trajets du corridor sur la période (Redis)
searches	integer	oui	Recherches sur ce corridor (Redis)
searchesNoResult	integer	oui	Recherches sans aucun trajet (Redis) — demande sans offre

CorridorsResponse

Champ	Type	Requis	Description
periodDays	integer	oui	
from	string (date-time)	oui	
items	array	oui	
generatedAt	string (date-time)	oui	
cached	boolean	oui	

CreateReportRequest

Champ	Type	Requis	Description
targetType	ReportTargetType	oui	
targetRef	string	oui	
reason	ReportReason	oui	
details	string	non	

CreateReportResponse

Champ	Type	Requis	Description
reportId	string	oui	
createdAt	string (date-time)	oui	

CronRun — Last heartbeat of a cron (Redis, 7 days) — D64 4A

Champ	Type	Requis	Description
service	string	oui	
name	string	oui	
ranAt	string (date-time)	oui	
durationMs	integer	oui	
ok	boolean	oui	
summary	string null	oui	
error	string null	oui	
schedule	string null	oui	

DataExport — What belongs to the member (D63 2A): never the counterpart's contact details, the delivery code, reports targeting the member, internal notes or mediation files

Champ	Type	Requis	Description
exportedAt	string (date-time)	oui	

Champ	Type	Requis	Description
format	string	oui	
profile	object	oui	
preferences	object	oui	
addresses	array	oui	
consents	array	oui	
carrierProfile	object null	oui	
trips	array	oui	
bookings	array	oui	
reviewsGiven	array	oui	
reviewsReceived	array	oui	
messages	array	oui	
meetups	array	oui	
phoneReveals	array	oui	
savedRoutes	array	oui	
favorites	array	oui	
following	array	oui	
notifications	array	oui	
reportsMade	array	oui	
dataRequests	array	oui	

DataRequestItem

Champ	Type	Requis	Description
id	string	oui	
userId	string	oui	
userLabel	string	oui	First name + initial, or « Membre supprimé »
type	enum : EXPORT, ERASURE	oui	
channel	enum : MEMBER, ADMIN	oui	
status	enum : DONE, REFUSED	oui	
refusalReasons	array	oui	
requestedByAdmin	string null	oui	
reason	string null	oui	
requestedAt	string (date-time)	oui	
completedAt	string null	oui	

DataRequestsResponse

Champ	Type	Requis	Description
items	array	oui	
nextCursor	string null	oui	

EmailChangeConfirmedResponse

Champ	Type	Requis	Description
ok	boolean	oui	
email	string	oui	
revokedSessions	integer	oui	

EmailChangeRequestedResponse

Champ	Type	Requis	Description
ok	boolean	oui	
pendingEmail	string	oui	
expiresInMinutes	integer	oui	

EraseMyAccountRequest

Champ	Type	Requis	Description
confirmation	string	oui	

ErasedResponse

Champ	Type	Requis	Description
success	boolean	oui	
erased	boolean	oui	

ErasureBlockedResponse

Champ	Type	Requis	Description
code	string	oui	
blockers	array	oui	
counts	object	oui	

ErasureBlocker — enum : ACTIVE_DEAL , PENDING_REQUEST , PAYOUT_PENDING , RETENTION_HELD , PUBLISHED_TRIP , ADMIN_ACCOUNT — Why an account cannot be erased yet (D63 3A) — closed list, translated by the client

ErrorResponse — Enveloppe d'erreur standard (error-middleware, toute AppError)

Champ	Type	Requis	Description
status	string	oui	
message	string	oui	
errors	object	non	Erreurs par champ (formulaire) — exposé quand details.errors est présent
details		non	Contexte structuré 'safe' (ex: type=otp) — toujours exposé ; le reste hors prod uniquement

FollowPreferencesRequest

Champ	Type	Requis	Description
notifyNextTrip	boolean	oui	

FollowResponse

Champ	Type	Requis	Description
success	boolean	oui	
message	string	non	
follow	object	oui	

FollowingResponse — Members with an upcoming trip first

Champ	Type	Requis	Description
success	boolean	oui	
count	integer	oui	
following	array	oui	

GoogleSignInRequest — Google ID token (D47). A new account needs consent, otherwise CONSENT_REQUIRED

Champ	Type	Requis	Description
credential	string	oui	
rememberMe	boolean	non	
consent	object	non	

GoogleSignInResponse

Champ	Type	Requis	Description
status	enum : LOGGED_IN, CONSENT_REQUIRED	oui	
created	boolean	non	
linked	boolean	non	
user	SessionUser	non	

HealthCheck

Champ	Type	Requis	Description
ok	boolean	oui	
ms	integer	oui	
error	string null	oui	

HealthReport — Uniform /health of every service (D64 3A)

Champ	Type	Requis	Description
status	enum : ok, degraded	oui	
service	string	oui	
version	string	oui	
uptimeSeconds	integer	oui	
checks	object	oui	
at	string (date-time)	oui	

InviteAdminRequest — SUPER_ADMIN only. New account: no client role, password set through the emailed link (48h).

Champ	Type	Requis	Description
email	string (email)	oui	
firstName	string	oui	
lastName	string	oui	
adminRoles	AdminRoles	oui	

InviteAdminResponse

Champ	Type	Requis	Description
ok	boolean	oui	
userId	ObjectId	oui	
existingAccount	boolean	oui	
expiresInHours	integer	non	

LiftSuspensionRequest

Champ	Type	Requis	Description
reason	string	oui	

MaintenanceState

Champ	Type	Requis	Description
enabled	boolean	oui	
messageFr	string	oui	
messageEn	string	oui	
scheduledAt	string null	oui	Annonce affichée avant la coupure (bandeau), null = aucune
updatedAt	string null	oui	
updatedBy	string null	oui	
version	integer	oui	
envOverride	boolean	non	

MeResponse — The User record without passwordHash (additional fields allowed)

Champ	Type	Requis	Description
success	boolean	oui	
user	object	oui	
roles	array	oui	

MemberLoginRequest — rememberMe = 30-day refresh cookie (A62)

Champ	Type	Requis	Description
email	string (email)	oui	
password	string	oui	
rememberMe	boolean	non	

MemberLoginResponse — Cookies access_token / refresh_token are set on the response

Champ	Type	Requis	Description
message	string	oui	
user	SessionUser	oui	

MemberRegisterRequest — Consent is mandatory (ConsentLog written at verification)

Champ	Type	Requis	Description
firstName	string	oui	
lastName	string	oui	
email	string (email)	oui	
password	string	oui	
termsAccepted	boolean	oui	
termsVersion	string	oui	
privacyVersion	string	oui	

MemberSessionItem

Champ	Type	Requis	Description
jti	string	oui	
createdAt	string (date-time)	oui	
lastActivityAt	string (date-time)	oui	
rememberMe	boolean	oui	
device	string	oui	Browser + OS derived from the user agent, or « Appareil inconnu »
ip	string null	oui	
current	boolean	oui	

MemberSessionsResponse

Champ	Type	Requis	Description
items	array	oui	

MessageResponse

Champ	Type	Requis	Description
message	string	oui	

MyProfileResponse

Champ	Type	Requis	Description
firstName	string	oui	
lastName	string	oui	
publicSlug	string null	oui	
avatarUrl	string null	oui	
birthDate	string null	oui	
profilePublic	boolean	oui	
showCity	boolean	oui	
carrier	object null	oui	

ObjectId — string : Identifiant MongoDB (ObjectId sérialisé en hexadécimal)

OkResponse

Champ	Type	Requis	Description
ok	boolean	oui	

PasswordChangedResponse

Champ	Type	Requis	Description
ok	boolean	oui	
revokedSessions	integer	oui	
hadPassword	boolean	oui	

PasswordForgotRequest — Always 200: never reveals whether the account exists

Champ	Type	Requis	Description
email	string (email)	oui	

PasswordResetRequest

Champ	Type	Requis	Description
passwordResetToken	string	oui	

Champ	Type	Requis	Description
newPassword	string	oui	

PasswordVerifyRequest

Champ	Type	Requis	Description
email	string (email)	oui	
otp	string	oui	

PasswordVerifyResponse

Champ	Type	Requis	Description
message	string	oui	
passwordResetToken	string	oui	

PilotageDrilldownItem

Champ	Type	Requis	Description
kind	enum : USER, TRIP, DEAL	oui	
id	ObjectId	oui	
label	string	oui	Prénom + initiale, ou corridor
at	string (date-time)	oui	La date du fait pour cette mesure
status	string null	oui	
amountCents	integer null	oui	
currencyCode	string null	oui	

PilotageDrilldownResponse — Consultation journalisée quand la mesure porte des personnes (inscriptions)

Champ	Type	Requis	Description
metric	PilotageMetric	oui	
granularity	PilotageGranularity	oui	
period	string	oui	
periodStart	string (date-time)	oui	
periodEnd	string (date-time)	oui	
items	array	oui	
total	integer	oui	
truncated	boolean	oui	true si plus de 200 éléments (liste bornée)

PilotageGranularity — enum : week, month — Semaines ISO (lundi, UTC) ou mois UTC

PilotageMetric — enum : signups, tripsPublished, requests, accepted, delivered, completed, cancelled, disputes, captured, refunded, paidOut, revenue, retention

PilotageSeriesPoint

Champ	Type	Requis	Description
period	string	oui	YYYY-MM (mois) ou YYYY-Www (semaine ISO)
periodStart	string (date-time)	oui	
signups	integer	oui	
tripsPublished	integer	oui	
requests	integer	oui	
accepted	integer	oui	
delivered	integer	oui	
completed	integer	oui	
cancelled	integer	oui	
disputes	integer	oui	
volume	array	oui	Encaissé (capture) par devise
finance	array	oui	

PilotageSeriesResponse

Champ	Type	Requis	Description
granularity	PilotageGranularity	oui	
from	string (date-time)	oui	

Champ	Type	Requis	Description
to	string (date-time)	oui	
points	array	oui	Du plus ancien au plus récent, périodes vides incluses
totals	object	oui	
generatedAt	string (date-time)	oui	
cached	boolean	oui	

PlatformSettingsValues

Champ	Type	Requis	Description
pricing.commissionPct	number	oui	Commission Yamba (percent) — D16 · COM-01
pricing.commissionFloorCents	number	oui	Plancher de commission (cents) — D16 · COM-02
pricing.minBillableKg	number	oui	Poids facturable minimum (kg) — D32 · PRC-06
pricing.minTransportCents	number	oui	Prix minimum par colis (cents) — D32 · PRC-06
pricing.referenceKg	number	oui	Colis de référence (comparabilité) (kg) — D33
pricing.sizeCoefS	number	oui	Coefficient taille S (coef) — PRC-03
pricing.sizeCoefM	number	oui	Coefficient taille M (coef) — PRC-03
pricing.sizeCoefL	number	oui	Coefficient taille L (coef) — PRC-03
protection.extendedPremiumCents	number	oui	Prime Garantie étendue (cents) — D22 · GAR-06
protection.extendedCapCents	number	oui	Plafond Garantie étendue (cents) — D22 · GAR-03
cancellation.fullRefundUntilHours	number	oui	Remboursement intégral jusqu'à (hours) — ANN-01 · D21
cancellation.lateRetentionPct	number	oui	Retenue d'annulation tardive (percent) — ANN-01 · D39 · D50
rating.windowDays	number	oui	Fenêtre de notation (days) — D53 · RG-NOTE-01
dispute.responseDelayHours	number	oui	Délai de réponse au litige (hours) — D55 1A · RG-MED-02
reputation.carrier.confirmedMinDeals	number	oui	Voyageur confirmé : deals minimum (count) — D29① · REP-03
reputation.carrier.topMinDeals	number	oui	Voyageur top : deals minimum (count) — REP-03
reputation.carrier.topMinRating	number	oui	Voyageur top : note minimale (rating) — REP-03
reputation.carrier.topMaxLateCancellations	number	oui	Voyageur top : annulations tolérées (count) — REP-03
reputation.shipper.confirmedMinDeals	number	oui	Expéditeur fiable : deals minimum (count) — REP-03
reputation.shipper.topMinDeals	number	oui	Expéditeur top : deals minimum (count) — REP-03
reputation.shipper.topMinRating	number	oui	Expéditeur top : note minimale (rating) — REP-03
reputation.shipper.topMaxLateCancellations	number	oui	Expéditeur top : annulations tardives tolérées (count) — REP-03
messaging.writeDaysAfterEnd	number	oui	Fil ouvert après la fin du deal (days) — D61 2A · RG-FCH-07
messaging.phoneRevealLeadHours	number	oui	Numéro révélé avant le rendez-vous (hours) — D61 4A
messaging.retentionDays	number	oui	Conservation des conversations (days) — D61 8A · RG-FCH-22
messaging.reminderDelayMinutes	number	oui	Relance email après (minutes) — D61 6A · RG-FCH-17
messaging.reminderMinIntervalMinutes	number	oui	Au plus une relance toutes les (minutes) — D61 6A · RG-FCH-17
alerts.payoutFailedHours	number	oui	Versement en échec depuis (hours) — D59 3A
alerts.disputeUndecidedHours	number	oui	Litige décidable sans décision depuis (hours) — D59 3A · A131
alerts.retentionHeldDays	number	oui	Retenue non arbitrée depuis (days) — D59 3A
alerts.reversalOpenHours	number	oui	Renversement ouvert depuis (hours) — D59 3A
alerts.outboxParkedAttempts	number	oui	Événement parké après (count) — D59 3A
alerts.outboxLagMinutes	number	oui	Relais en retard depuis (minutes) — D59 3A
alerts.emailsFailedWindowHours	number	oui	Emails en échec : fenêtre (hours) — D59 3A
alerts.noTripPublishedDays	number	oui	Aucun trajet publié depuis (days) — D59 3A
alerts.acceptanceRateWindowDays	number	oui	Taux d'acceptation : fenêtre (days) — D59 3A
alerts.acceptanceRateMinPct	number	oui	Taux d'acceptation minimum (percent) — D59 3A
alerts.acceptanceRateMinRequests	number	oui	Taux d'acceptation : demandes minimum (count) — D59 3A
documents.maxDocsPerTrip	number	oui	Documents par trajet (count) — ex-SiteConfig
documents.maxDocSizeMb	number	oui	Taille maximale d'un document (mb) — ex-SiteConfig
trust.newAccountDays	number	oui	Compte neuf pendant (days) — D71 · CNF-06 · REP-04
trust.newAccount.maxDeclaredValueCents	number	oui	Compte neuf : valeur déclarée max par colis (cents) — D71 · CNF-06
trust.newAccount.maxWeightKg	number	oui	Compte neuf : poids max par colis (kg) — D71 · CNF-06
trust.newAccount.maxShipmentsPerMonth	number	oui	Compte neuf : envois par mois civil (count) — D71 · CNF-06
privacy.recipientRetentionDays	number	oui	Effacement du destinataire après (days) — D63 5A · RGP-02
retention.notificationsDays	number	oui	Notifications in-app (days) — D64 6A · RGP-01

Champ	Type	Requis	Description
retention.emailDeliveriesDays	number	oui	Traces d'envoi d'emails (days) — D64 6A · RGP-01
retention.consumedEventsDays	number	oui	Registre des événements consommés (days) — D64 6A
retention.outboxPublishedDays	number	oui	Événements d'outbox publiés (days) — D64 6A

PreferencesResponse

Champ	Type	Requis	Description
success	boolean	oui	
preferences	object	oui	

ProposeSuspensionRequest — SUPPORT proposes; MEDIATOR / SUPER_ADMIN executes (D56 3A)

Champ	Type	Requis	Description
level	enum : RESTRICTED, SUSPENDED	oui	
reason	string	oui	

PublicReputation

Champ	Type	Requis	Description
level	enum : NEW, CONFIRMED, TOP	oui	
ratingsAvg	number	oui	
ratingsCount	integer	oui	
completedDealsCount	integer	oui	
lateCancellationsCount	integer	oui	

PublicReview

Champ	Type	Requis	Description
id	ObjectId	oui	
rating	number	oui	
comment	string null	oui	
criteria	object null	non	
createdAt	string (date-time)	oui	
author	object	oui	

PublicReviewsResponse

Champ	Type	Requis	Description
success	boolean	oui	
reviews	array	oui	
nextCursor	string null	oui	

PublicTripPreview

Champ	Type	Requis	Description
id	ObjectId	oui	
transportMode	string null	oui	
originCity	string null	oui	
destinationCity	string null	oui	
departureAt	string null	oui	
flightType	string null	oui	
trainTripType	string null	oui	
carTripFlexibility	string null	oui	
minPriceCents	integer null	oui	
currencyCode	string	oui	

PublicTripsResponse

Champ	Type	Requis	Description
success	boolean	oui	
trips	array	oui	
nextCursor	string null	oui	

PublicUserProfile — D28 / D29 / D67: location nulled when showCity is off; 404 to others when profilePublic is off (owner sees hidden: true)

Champ	Type	Requis	Description
publicSlug	string	oui	
firstName	string	oui	
lastInitial	string	oui	
avatarUrl	string null	oui	
memberSince	string (date-time)	oui	
location	object	oui	
stats	object	oui	
tripperRating	object null	oui	
shipperRating	object	oui	
reputation	object	non	
tripper	object null	oui	
shipper	object	oui	
follow	object	oui	
isOwnProfile	boolean	oui	
isMe	boolean	non	
hidden	boolean	non	

PublicUserResponse

Champ	Type	Requis	Description
success	boolean	oui	
user	PublicUserProfile	oui	

RegistrationStartedResponse — OTP sent; the token identifies the pending registration (Redis, 10 min)

Champ	Type	Requis	Description
message	string	oui	
verificationToken	string	oui	

ReportReason — enum : ILLEGAL_CONTENT , SCAM , INAPPROPRIATE , IMPERSONATION , OTHER

ReportStatus — enum : OPEN , REVIEWED , DISMISSED

ReportTargetType — enum : TRIP , USER

RequestEmailChange — Under sudo (D65 4A); a code is sent to the NEW address

Champ	Type	Requis	Description
newEmail	string (email)	oui	

ResetSettingsRequest

Champ	Type	Requis	Description
keys	array	non	Clés à remettre par défaut ; absent ou vide = toutes
reason	string	oui	
expectedVersion	integer	oui	

ReviewReportRequest

Champ	Type	Requis	Description
decision	enum : REVIEWED, DISMISSED	oui	
note	string	non	

RevokeSessionResponse

Champ	Type	Requis	Description
ok	boolean	oui	
current	boolean	non	
revoked	integer	non	

SavedRouteDto

Champ	Type	Requis	Description
originCity	string	oui	
originCountry	string	oui	
originCountryCode	string	oui	
originCityCode	string null	non	
originRegion	string null	non	
originRegionCode	string null	non	
originPlaceId	string null	non	
originLat	number null	non	
originLng	number null	non	
destinationCity	string	oui	
destinationCountry	string	oui	
destinationCountryCode	string	oui	
destinationCityCode	string null	non	
destinationRegion	string null	non	
destinationRegionCode	string null	non	
destinationPlaceId	string null	non	
destinationLat	number null	non	
destinationLng	number null	non	
earliestDate	string null	non	
latestDate	string null	non	
emailEnabled	boolean	non	
inAppEnabled	boolean	non	
includeNearby	boolean	non	
id	ObjectId	oui	
userId	ObjectId	oui	
expiresAt	string (date-time)	oui	
isActive	boolean	oui	
lastNotifiedAt	string null	non	
createdAt	string (date-time)	oui	
updatedAt	string (date-time)	oui	

SavedRouteRequest — Origin and destination are Google Places snapshots; same city+country on both sides is refused. Max 20 alerts per member

Champ	Type	Requis	Description
originCity	string	oui	
originCountry	string	oui	
originCountryCode	string	oui	
originCityCode	string null	non	
originRegion	string null	non	
originRegionCode	string null	non	
originPlaceId	string null	non	
originLat	number null	non	
originLng	number null	non	
destinationCity	string	oui	
destinationCountry	string	oui	
destinationCountryCode	string	oui	
destinationCityCode	string null	non	
destinationRegion	string null	non	
destinationRegionCode	string null	non	
destinationPlaceId	string null	non	
destinationLat	number null	non	
destinationLng	number null	non	
earliestDate	string null	non	
latestDate	string null	non	
emailEnabled	boolean	non	

Champ	Type	Requis	Description
inAppEnabled	boolean	non	
includeNearby	boolean	non	

SavedRouteResponse

Champ	Type	Requis	Description
success	boolean	oui	
message	string	non	
savedRoute	SavedRouteDto	oui	

SavedRoutesResponse

Champ	Type	Requis	Description
success	boolean	oui	
savedRoutes	array	oui	
count	integer	oui	

ServiceStatus

Champ	Type	Requis	Description
name	string	oui	
url	string	oui	
reachable	boolean	oui	
ms	integer	oui	
report	HealthReport null	oui	
error	string null	oui	

SessionUser

Champ	Type	Requis	Description
id	ObjectId	oui	
email	string	oui	
firstName	string	oui	
lastName	string	oui	
roles	array	oui	

SetMyAvatarRequest — After a signed ImageKit upload (folder /avatars); the URL must belong to IMAGEKIT_URL_ENDPOINT (D42)

Champ	Type	Requis	Description
fileId	string	oui	
url	string (uri)	oui	

SettingDefinition

Champ	Type	Requis	Description
key	string	oui	
group	enum : pricing, protection, cancellation, rating, dispute, reputation, messaging, alerts, documents, privacy, retention, trust	oui	
label	string	oui	
description	string	oui	
rule	string	oui	
unit	enum : percent, cents, kg, coef, hours, days, minutes, count, rating, mb	oui	
default	number	oui	
min	number	oui	
max	number	oui	
step	number	oui	
scope	enum : BUSINESS, OPERATIONS	oui	
contractual	boolean	non	
consumers	array	oui	
example	string	non	

SettingsHistoryItem

Champ	Type	Requis	Description
id	ObjectId	oui	
at	string (date-time)	oui	
admin	string	oui	
action	string	oui	
key	string null	oui	
before	number null	oui	
after	number null	oui	
reason	string null	oui	
version	integer null	oui	

SettingsHistoryResponse

Champ	Type	Requis	Description
items	array	oui	
nextCursor	ObjectId null	oui	

SettingsLastChange — Dernière modification (7 jours) — affichée sur l'accueil admin (D62 5A)

Champ	Type	Requis	Description
at	string (date-time)	oui	
byName	string	oui	
keys	array	oui	

SettingsWriteResponse

Champ	Type	Requis	Description
version	integer	oui	
changed	array	oui	

StripeLinkResponse — Single-use Stripe URL (onboarding or Express dashboard), never stored

Champ	Type	Requis	Description
success	boolean	oui	
url	string (uri)	oui	

StripeStatusResponse

Champ	Type	Requis	Description
success	boolean	oui	
status	enum : not_started, pending, complete	oui	
chargesEnabled	boolean	oui	
payoutsEnabled	boolean	oui	
detailsSubmitted	boolean	non	

SuccessMessageResponse — Generic { success, message } acknowledgement

Champ	Type	Requis	Description
success	boolean	non	
message	string	oui	

SudoStatus — Sudo window bound to the current session (D65 1A)

Champ	Type	Requis	Description
active	boolean	oui	
expiresAt	string null	oui	

SudoVerifyRequest

Champ	Type	Requis	Description
code	string	oui	

SudoWindowResponse — Sudo window opened for this session (15 min, D65)

Champ	Type	Requis	Description
active	boolean	oui	
expiresAt	string (date-time)	oui	

SuspensionAppliedResponse

Champ	Type	Requis	Description
ok	boolean	oui	
accountStatus	string	oui	
at	string (date-time)	non	

SuspensionProposedResponse

Champ	Type	Requis	Description
ok	boolean	oui	
proposedAt	string (date-time)	oui	

TrustAssessment — Internal risk score (D29 ②, REP-04): decision support and progressive caps only — never an automatic sanction, never shown to members

Champ	Type	Requis	Description
score	integer	oui	0 = no risk signal, 100 = maximum; higher is riskier
level	TrustLevel	oui	
factors	array	oui	
caps	object null	oui	
capsReason	string null	oui	
signals	object	oui	

TrustLevel — enum : NEW , STANDARD , WATCH , HIGH_RISK

UnauthorizedResponse — 401 du middleware isAuthenticated (token absent, invalide, expiré, ou compte introuvable) — hors error-middleware

Champ	Type	Requis	Description
message	string	oui	

UnhandledError — 500 non géré (exception hors AppError) — champ error , pas message

Champ	Type	Requis	Description
status	string	oui	
error	string	oui	

UpdateAdminRoleRequest — C-PR3bis : la liste complète des profils (remplace)

Champ	Type	Requis	Description
adminRoles	AdminRoles	oui	

UpdateAdminRoleResponse

Champ	Type	Requis	Description
ok	boolean	oui	
adminRoles	array	oui	
adminRole	string null	oui	

UpdateLocaleRequest — One of SUPPORTED_LOCALES (D44)

Champ	Type	Requis	Description
locale	string	oui	

UpdateLocaleResponse

Champ	Type	Requis	Description
success	boolean	oui	
preferredLocale	string	oui	

UpdateMaintenanceRequest

Champ	Type	Requis	Description
enabled	boolean	oui	
messageFr	string	oui	
messageEn	string	oui	
scheduledAt	string null	oui	
reason	string	oui	
expectedVersion	integer	oui	

UpdateMyPreferencesRequest — Member preferences (D63 8A, D66 2A)

Champ	Type	Requis	Description
messagingReminderEmails	boolean	non	
analyticsOptIn	boolean	non	D66 2A — audience measurement consent (written to ConsentLog COOKIES)

UpdateMyProfileRequest — Member-editable profile (D67 1A); the public slug never changes

Champ	Type	Requis	Description
firstName	string	non	
lastName	string	non	
displayName	string	non	
bio	string null	non	
birthDate	string null	non	
profilePublic	boolean	non	
showCity	boolean	non	

UpdateSettingsRequest

Champ	Type	Requis	Description
changes	object	oui	Clés du catalogue → nouvelle valeur (seules les clés modifiées)
reason	string	oui	
expectedVersion	integer	oui	Verrou optimiste : la version lue ; 409 si elle a bougé

VerificationTokenRequest

Champ	Type	Requis	Description
verificationToken	string	oui	

VerifyRegistrationRequest

Champ	Type	Requis	Description
verificationToken	string	oui	
otp	string	oui	

trip-service — port 6002

Trajets, recherche, prix, documents, uploads. Document vivant : <http://localhost:6002/docs> (Scalar) et <apps/trip-service/openapi.json> . Via le gateway : <http://localhost:8080/api> (préfixes ci-dessous).

trip-service › —

GET /openapi.json

Ce document OpenAPI 3.1

operationId : `getOpenApiDocument` · Authentification : aucune (public)

Code	Réponse
200	objet {} — Document OpenAPI 3.1 (généré depuis Zod)

trip-service › trips-search

Recherche publique (aucune auth)

GET /trips/pricing/params

Paramètres de prix de la plateforme (public)

operationId : `getPricingParams` · Authentification : aucune (public)

Commission, planchers, coefficients de taille, Garantie étendue, kilo de référence : les valeurs en vigueur, réglées par l'admin (D62). Le wizard calcule le devis avec le moteur unique (D34) et ces valeurs ; le serveur refait le calcul à la création. `version` change à chaque modification.

Code	Réponse
200	PricingParamsResponse — Paramètres de prix en vigueur

GET /trips/search

Rechercher des trips publiés

operationId : searchTrips · Authentification : aucune (public)

Recherche publique paginée (cursor-based). Hard filters non-négociables : status=PUBLISHED et départ futur. Les trips malformés sont exclus silencieusement du mapping. ⚠ Enveloppe SANS champ success (fidèle au réel).

Paramètre	Dans	Requis	Type	Description
mode	query	non	TransportModeFilter	Filtre mode de transport (défaut : all)
from	query	non	string	Ville ou pays d'origine (match partiel, insensible à la casse)
to	query	non	string	Ville ou pays de destination (match partiel, insensible à la casse)
dateFrom	query	non	string	Borne basse ISO 8601. Ignorée si dans le passé : la recherche ne renvoie jamais de trips déjà partis (borne effective = max(now, dateFrom))
dateTo	query	non	string	Borne haute ISO 8601 sur la date de départ
categories	query	non	string	CSV de UiParcelCategory (ex: "clothes,shoes,documents"). Sémantique hasSome : au moins une catégorie acceptée doit matcher. Les valeurs invalides sont filtrées silencieusement
departureBuckets	query	non	string	CSV de DepartureBucket (ex: "morning,evening"). OR des tranches horaires (heure locale de départ). Valeurs invalides filtrées silencieusement
locale	query	non	SearchLocale	Locale de formatage serveur des dates (défaut : fr)
sort	query	non	SortOption	Tri (défaut : earliest). lowestPrice exclut les trips sans minPriceCents
superTripper	query	non	string	Uniquement les Super Trippers — booléen en query string : tout sauf "true" vaut false
profileVerified	query	non	string	Uniquement les profils vérifiés — booléen en query string : tout sauf "true" vaut false
instantBooking	query	non	string	Uniquement les trips à réservation instantanée — booléen en query string : tout sauf "true" vaut false
verifiedTicket	query	non	string	Uniquement les billets vérifiés — booléen en query string : tout sauf "true" vaut false
cursor	query	non	string	Curseur de pagination : le nextCursor de la page précédente
limit	query	non	integer	Taille de page

Code	Réponse
200	SearchTripsResponse — Page de résultats + nextCursor + totalCount
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
500	UnhandledError — Erreur serveur non gérée

GET /trips/search/facets

Counts pour les filtres de recherche

operationId : searchTripsFacets · Authentification : aucune (public)

9 counts en parallèle. Les counts par mode sont calculés SANS le filtre mode courant ; les counts des soft toggles AVEC. Mêmes hard filters que la recherche. ⚠ Enveloppe SANS champ success (fidèle au réel).

Paramètre	Dans	Requis	Type	Description
mode	query	non	TransportModeFilter	Filtre mode de transport (défaut : all)
from	query	non	string	Ville ou pays d'origine (match partiel, insensible à la casse)
to	query	non	string	Ville ou pays de destination (match partiel, insensible à la casse)
dateFrom	query	non	string	Borne basse ISO 8601. Ignorée si dans le passé : la recherche ne renvoie jamais de trips déjà partis (borne effective = max(now, dateFrom))
dateTo	query	non	string	Borne haute ISO 8601 sur la date de départ
categories	query	non	string	CSV de UiParcelCategory (ex: "clothes,shoes,documents"). Sémantique hasSome : au moins une catégorie acceptée doit matcher. Les valeurs invalides sont filtrées silencieusement
departureBuckets	query	non	string	CSV de DepartureBucket (ex: "morning,evening"). OR des tranches horaires (heure locale de départ). Valeurs invalides filtrées silencieusement
locale	query	non	SearchLocale	Locale de formatage serveur des dates (défaut : fr)

Code	Réponse
200	SearchFacetsResponse — Counts par mode et par toggle
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
500	UnhandledError — Erreur serveur non gérée

trip-service › trips-favorites

GET /trips/favorites

Mes trajets favoris

operationId : listMyFavoriteTrips · Authentification : cookieAuth, bearerAuth

Cartes de recherche (YambaTripResult, isFavorite = true) des trajets mis en favori, du plus récent au plus ancien ; les trajets passés restent listés. Auth requise.

Paramètre	Dans	Requis	Type	Description
locale	query	non	SearchLocale	Locale des libellés (sinon x-locale, sinon fr)

Code	Réponse
200	FavoriteTripsResponse — Liste
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

POST /trips/{id}/favorite

Mettre un trajet en favori (idempotent)

operationId : addTripFavorite · Authentification : cookieAuth, bearerAuth

Signet privé : jamais notifié au Voyageur. 404 si le trajet n'existe pas (jamais 403 : ne pas révéler), 403 OWN_TRIP sur son propre trajet, 409 TRIP_NOT_FAVORITABLE si le trajet n'est pas PUBLISHED. Rejouer l'action renvoie le même état.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	TripFavoriteState —
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
403	ErrorResponse — OWN_TRIP — details.type = favorite
404	ErrorResponse — Trajet introuvable ou supprimé
409	ErrorResponse — TRIP_NOT_FAVORITABLE — trajet non publié
500	UnhandledError — Erreur serveur non gérée

DELETE /trips/{id}/favorite

Retirer un trajet des favoris (idempotent)

operationId : removeTripFavorite · Authentification : cookieAuth, bearerAuth

Toujours possible, même sur un trajet passé. 404 si le trajet n'existe pas.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	TripFavoriteState —
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
404	ErrorResponse — Trajet introuvable ou supprimé
500	UnhandledError — Erreur serveur non gérée

trip-service › trips-public

Vue publique d'un trip (aucune auth)

GET /trips/{id}/public

Vue publique d'un trip publié

operationId : getPublicTrip · Authentification : aucune (public)

DTO structuré (origin/destination/dates/tripper imbriqués), filtré privacy (initiale du nom). 404 si inexistant, soft-deleted ou non-PUBLISHED — au format PublicNotFound (renvoyé hors error-middleware). 400 (ErrorResponse) si l'id n'est pas un ObjectId valide.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	PublicTripResponse — Vue publique du trip
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
404	PublicNotFound — Trip inexistant, supprimé ou non publié
500	UnhandledError — Erreur serveur non gérée

trip-service › trips

CRUD owner (auth requise)

POST /trips

Créer un trip (brouillon ou publication directe)

operationId : createTrip · Authentification : cookieAuth, bearerAuth

publish=true crée directement en PUBLISHED si les gates onboarding/Stripe passent (sinon 400). Un brouillon peut être incomplet (villes, dates nullish). minPriceCents et departureHourLocal sont recalculés côté serveur. Auth requise.

Corps (requis) : CreateTripBody

Code	Réponse
201	TripMutationResponse — Trip créé (avec documents, sans allowedActions)
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

GET /trips/my

Mes trips

operationId : getMyTrips · Authentification : cookieAuth, bearerAuth

Tous les trips de l'utilisateur (hors soft-deleted), triés par createdAt desc. Chaque trip embarque allowedActions (state machine) et un select partiel des documents {id, type, status, url}. Auth requise.

Paramètre	Dans	Requis	Type	Description
status	query	non	TripStatus	Filtrer par statut (insensible à la casse : toUpperCase() serveur)

Code	Réponse
200	TripsListResponse — Liste
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

GET /trips/{id}

Détail owner d'un trip

operationId : getTrip · Authentification : cookieAuth, bearerAuth

Vue complète (documents, user, carrierPage) + allowedActions IMBRIQUÉ dans trip. Ownership requis : le trip d'autrui renvoie 400 "Unauthorized." (voir note sémantique). Auth requise.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	TripResponse — { success, trip: { ...trip, allowedActions } }
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

PUT /trips/{id}

Modifier un trip

operationId : updateTrip · Authentification : cookieAuth, bearerAuth

Body partiel (seuls les champs envoyés sont écrits). publish=true sur un DRAFT déclenche les gates de publication (onboarding, Stripe, locations). Édition refusée par la machine (COMPLETED/ARCHIVED/CANCELLED...) → 400. Auth requise.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Corps (requis) : UpdateTripBody

Code	Réponse
200	TripMutationResponse — Trip mis à jour (sans allowedActions)
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

DELETE /trips/{id}

Supprimer (brouillon) ou annuler (alias)

operationId : deleteTrip · Authentification : cookieAuth, bearerAuth

?hard=true → soft delete d'un brouillon (isDeleted, invisible partout). Sans ?hard → alias backward-compat de cancel. Auth requise.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip
hard	query	non	string	true = soft delete du brouillon · absent = alias de cancel

Code	Réponse
200	ActionResponse — "Draft deleted." ou "Trip cancelled."
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

trip-service › trips-lifecycle

Transitions de la state machine (auth requise, owner)

POST /trips/{id}/publish

Publier un brouillon

operationId : publishTrip · Authentification : cookieAuth, bearerAuth

DRAFT → PUBLISHED. Gates : onboarding carrier, Stripe (charges enabled), champs requis (mode, villes, départ futur, ≥1 catégorie), ≥1 pickup + ≥1 delivery location. Transition refusée → 400 avec le message machine de canPerform. Auth requise (owner uniquement).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	ActionResponse — Transition publish effectuée
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

POST /trips/{id}/unpublish

Repasser en brouillon

operationId : unpublishTrip · Authentification : cookieAuth, bearerAuth

PUBLISHED/PAUSED → DRAFT. Interdit avec réservations actives (guard prêt pour le chantier Booking). Décrémente les stats carrier. Transition refusée → 400 avec le message machine de canPerform. Auth requise (owner uniquement).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	ActionResponse — Transition unpublish effectuée
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

POST /trips/{id}/pause

Mettre en pause

operationId : pauseTrip · Authentification : cookieAuth, bearerAuth

PUBLISHED → PAUSED. Le trip reste dans le pool public. Transition refusée → 400 avec le message machine de canPerform. Auth requise (owner uniquement).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	ActionResponse — Transition pause effectuée
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

POST /trips/{id}/resume

Reprendre

operationId : resumeTrip · Authentification : cookieAuth, bearerAuth

PAUSED → PUBLISHED. Date de départ non passée requise. Transition refusée → 400 avec le message machine de canPerform. Auth requise (owner uniquement).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	ActionResponse — Transition resume effectuée
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

POST /trips/{id}/cancel

Annuler

operationId : cancelTrip · Authentification : cookieAuth, bearerAuth

→ CANCELLED (cancelledAt posé). Décrémente les stats carrier, y compris depuis PAUSED. Transition refusée → 400 avec le message machine de canPerform. Auth requise (owner uniquement).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	ActionResponse — Transition cancel effectuée
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

POST /trips/{id}/restore

Restaurer en brouillon

operationId : restoreTrip · Authentification : cookieAuth, bearerAuth

CANCELLED → DRAFT (cancelledAt effacé). Date de départ non passée requise. Transition refusée → 400 avec le message machine de canPerform. Auth requise (owner uniquement).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	ActionResponse — Transition restore effectuée
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

POST /trips/{id}/archive

Archiver

operationId : archiveTrip · Authentification : cookieAuth, bearerAuth

COMPLETED/CANCELLED → ARCHIVED (one-way, pas de désarchivage MVP). Transition refusée → 400 avec le message machine de canPerform. Auth requise (owner uniquement).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	ActionResponse — Transition archive effectuée
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

trip-service › trips-documents

Justificatifs du trip (auth requise, owner)

POST /trips/{id}/documents

Ajouter des justificatifs

operationId : addTripDocuments · Authentification : cookieAuth, bearerAuth

Déduplication par fileId (les doublons sont ignorés ; si aucun nouveau → 200 "No new documents to add."). Limites serveur : siteConfig.maxDocsPerTrip (défaut 5), maxDocSizeMb (défaut 5 Mo). Un TICKET_PROOF fait passer ticketVerificationStatus NOT_SUBMITTED → PENDING. Auth requise (owner).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Corps (requis) : AddDocumentsBody

Code	Réponse
200	TripMutationResponse — Aucun nouveau document (tous dédupliqués)
201	TripMutationResponse — Documents ajoutés — trip complet renvoyé
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

DELETE /trips/{id}/documents/{documentId}

Supprimer un justificatif

operationId : removeTripDocument · Authentification : cookieAuth, bearerAuth

Supprime le document (et le fichier ImageKit, best-effort). Si c'était le dernier TICKET_PROOF, ticketVerificationStatus repasse à NOT_SUBMITTED. Auth requise (owner).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip
documentId	path	oui	ObjectId	Identifiant du document

Code	Réponse
200	ActionResponse — Document supprimé
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

trip-service › uploads

Upload direct navigateur → ImageKit (auth requise)

GET /uploads/imagekit-auth

Paramètres d'authentification ImageKit

operationId : getImageKitAuthParams · Authentification : cookieAuth, bearerAuth

Pour l'upload direct navigateur → ImageKit (token/expire/signature, ~30 min). publicKey et urlEndpoint absents si l'env n'est pas configuré. Auth requise.

Code	Réponse
200	ImageKitAuthResponse — Paramètres d'upload
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

DELETE /uploads/imagekit/{fileId}

Supprimer un fichier ImageKit

operationId : deleteImageKitFile · Authentification : cookieAuth, bearerAuth

Idempotent : un fichier déjà supprimé renvoie 200 "File was already deleted.". Auth requise.

Paramètre	Dans	Requis	Type	Description
fileId	path	oui	string	Identifiant ImageKit du fichier

Code	Réponse
200	ActionResponse — Fichier supprimé (ou déjà absent)
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
500	UnhandledError — Erreur serveur non gérée

trip-service › admin

Back-office (chantier C, D57) — session ADMIN avec TOTP, permission par profil

GET /admin/trips

Trajets — liste filtrable (D57 5A)

operationId : adminListTrips · Authentification : adminCookieAuth

Permission trips.read. Filtres : q (ville ou ObjectId), status, hidden=1 (masqués par Yamba), ticketPending=1, carrierId, from (ISO). 100 plus récents par départ, avec le nombre de réservations actives par trajet.

Paramètre	Dans	Requis	Type	Description
q	query	non	string	Ville (origine / destination) ou identifiant du trajet
status	query	non	TripStatus	
hidden	query	non	string	1 = masqués par Yamba seulement — booléen en query string : tout sauf "true" vaut false
ticketPending	query	non	string	1 = billet en attente de vérification — booléen en query string : tout sauf "true" vaut false
carrierId	query	non	string	Trajets d'un Voyageur
from	query	non	string	Départ à partir de

Code	Réponse
200	AdminTripsResponse — Liste
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
403	ErrorResponse — Profil admin sans la permission requise, ou conflit d'intérêts (son propre trajet)
500	UnhandledError — Erreur serveur non gérée

GET /admin/trips/{id}

Fiche trajet admin (D57 4A) — Voyageur, réservations, documents, journal

operationId : adminGetTripFile · Authentification : adminCookieAuth

Permission trips.read. La consultation est journalisée (AdminAction TRIP_VIEWED). Jamais de code de livraison.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Code	Réponse
200	AdminTripFile — Fiche
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
403	ErrorResponse — Profil admin sans la permission requise, ou conflit d'intérêts (son propre trajet)
404	ErrorResponse — Trajet ou document introuvable (ou supprimé)
500	UnhandledError — Erreur serveur non gérée

POST /admin/trips/{id}/hide/propose

Proposer un masquage (D57 6A) — SUPPORT

operationId : adminProposeHideTrip · Authentification : adminCookieAuth

Permission trips.hide.propose. Motif ≥ 20 caractères. Aucun effet sur la visibilité ; journal TRIP_HIDE_PROPOSED.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Corps (requis) : HideTripRequest

Code	Réponse
200	ActionResponse — Proposition enregistrée
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
403	ErrorResponse — Profil admin sans la permission requise, ou conflit d'intérêts (son propre trajet)
404	ErrorResponse — Trajet ou document introuvable (ou supprimé)
500	UnhandledError — Erreur serveur non gérée

POST /admin/trips/{id}/hide

Masquer un trajet (D57 3A) — MEDIATOR / SUPER_ADMIN

operationId : adminHideTrip · Authentification : adminCookieAuth

Permission trips.hide.apply. Pose Trip.hiddenByAdminAt + motif (≥ 20) dans la même transaction que le journal TRIP_HIDDEN ; invisible en recherche, page publique 404, non réservable ; réservations en cours préservées ; email au Voyageur. Yamba n'annule jamais un trajet.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Corps (requis) : HideTripRequest

Code	Réponse
200	ActionResponse — Trajet masqué
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
403	ErrorResponse — Profil admin sans la permission requise, ou conflit d'intérêts (son propre trajet)
404	ErrorResponse — Trajet ou document introuvable (ou supprimé)
500	UnhandledError — Erreur serveur non gérée

DELETE /admin/trips/{id}/hide

Rétablir un trajet masqué (D57 3A)

operationId : adminUnhideTrip · Authentification : adminCookieAuth

Permission trips.hide.apply. Motif ≥ 20 caractères, journal TRIP_UNHIDDEN, email au Voyageur.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Identifiant du trip

Corps (requis) : HideTripRequest

Code	Réponse
200	ActionResponse — Trajet rétabli
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
403	ErrorResponse — Profil admin sans la permission requise, ou conflit d'intérêts (son propre trajet)
404	ErrorResponse — Trajet ou document introuvable (ou supprimé)
500	UnhandledError — Erreur serveur non gérée

GET /admin/tickets

File « billets à vérifier » (D57 1A) — trajets à venir, plus anciens d'abord

operationId : adminListTicketQueue · Authentification : adminCookieAuth

Permission tickets.review. Documents TICKET_PROOF en PENDING. Les billets de trajets déjà partis passent EXPIRED à la lecture (8A) — expiredNow les compte.

Code	Réponse
200	TicketQueueResponse — File
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
403	ErrorResponse — Profil admin sans la permission requise, ou conflit d'intérêts (son propre trajet)
500	UnhandledError — Erreur serveur non gérée

GET /admin/tickets/{documentId}

Ouvrir un billet (D57 7A) — consultation journalisée

operationId : adminViewTicket · Authentification : adminCookieAuth

Permission tickets.review. Renvoie l'URL ImageKit du document ; chaque ouverture écrit un AdminAction DOCUMENT_VIEWED.

Paramètre	Dans	Requis	Type	Description
documentId	path	oui	string	Identifiant du TripDocument (ObjectId)

Code	Réponse
200	objet { } — Document
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
403	ErrorResponse — Profil admin sans la permission requise, ou conflit d'intérêts (son propre trajet)
404	ErrorResponse — Trajet ou document introuvable (ou supprimé)
500	UnhandledError — Erreur serveur non gérée

POST /admin/tickets/{documentId}/review

Valider ou rejeter un billet (D57 1A) — motif fermé au rejet

operationId : adminReviewTicket · Authentification : adminCookieAuth

Permission tickets.review. decision VERIFY | REJECT (+ reason ILLEGIBLE | DATES_MISMATCH | NAME_MISMATCH | SUSPICIOUS). Document et Trip.ticketVerificationStatus mis à jour dans la même transaction que le journal ; email au Voyageur ; un billet rejeté peut être redéposé. Un billet déjà traité → 400 ; son propre billet → 403.

Paramètre	Dans	Requis	Type	Description
documentId	path	oui	string	Identifiant du TripDocument (ObjectId)

Corps (requis) : ReviewTicketRequest

Code	Réponse
200	ActionResponse — Décision enregistrée
400	ErrorResponse — Requête invalide, trip introuvable, non-proprétaire, ou transition refusée par la state machine (ValidationError — voir note sémantique)
401	UnauthorizedResponse — Token absent, invalide, expiré, ou compte introuvable (middleware isAuthenticated)
403	ErrorResponse — Profil admin sans la permission requise, ou conflit d'intérêts (son propre trajet)
404	ErrorResponse — Trajet ou document introuvable (ou supprimé)
500	UnhandledError — Erreur serveur non gérée

trip-service › schémas utilisés (62)

ActionResponse — Réponse minimale des actions (transitions, deletes) — aucun trip renvoyé

Champ	Type	Requis	Description
success	boolean	oui	
message	string	oui	

AddDocumentsBody

Champ	Type	Requis	Description
documents	array	oui	Maximum côté serveur : paramètre documents.maxDocsPerTrip (défaut 5) · taille max par doc : documents.maxDocSizeMb (défaut 5 Mo) — D62

AdminTripFile

Champ	Type	Requis	Description
id	ObjectId	oui	
status	string	oui	
originCity	string	oui	
destinationCity	string	oui	
departureAt	string null	oui	

Champ	Type	Requis	Description
arrivalAt	string null	oui	
transportMode	string null	oui	
capacityKg	number null	oui	
reservedKg	number null	oui	
pricing	null null	oui	
createdAt	string (date-time)	oui	
publishedAt	string null	oui	
cancelledAt	string null	oui	
carrier	object	oui	
ticketVerificationStatus	string	oui	
hidden	object null	oui	
hideProposal	object null	oui	
documents	array	oui	
bookings	array	oui	
adminActions	array	oui	

AdminTripSummary

Champ	Type	Requis	Description
id	ObjectId	oui	
status	string	oui	
originCity	string	oui	
destinationCity	string	oui	
departureAt	string null	oui	
transportMode	string null	oui	
carrier	object	oui	
ticketVerificationStatus	string	oui	
hidden	boolean	oui	
hideProposed	boolean	oui	
activeBookingsCount	integer	oui	
publishedAt	string null	oui	

AdminTripsResponse

Champ	Type	Requis	Description
items	array	oui	
total	integer	oui	
nextCursor	string null	non	C-PR7a — id du dernier élément ; absent = fin

AllowedActions — array

CarTripFlexibility — enum : DIRECT, DETOUR_BY_AGREEMENT

CategoryCondition

Champ	Type	Requis	Description
category	ParcelCategory	oui	
priceAmountCents	integer	oui	Prix en centimes (DEV-01 : jamais de float)

CreateTripBody

Champ	Type	Requis	Description
transportMode	TransportMode	oui	
tripType	TripType	oui	
originLabel	string	oui	
originPlaceId	string null	non	
originCity	string null	non	
originCityCode	string null	non	
originRegion	string null	non	
originRegionCode	string null	non	

Champ	Type	Requis	Description
originCountry	string null	non	
originCountryCode	string null	non	
originLat	number null	non	
originLng	number null	non	
originTimezone	string null	non	
destinationLabel	string	oui	
destinationPlaceId	string null	non	
destinationCity	string null	non	
destinationCityCode	string null	non	
destinationRegion	string null	non	
destinationRegionCode	string null	non	
destinationCountry	string null	non	
destinationCountryCode	string null	non	
destinationLat	number null	non	
destinationLng	number null	non	
destinationTimezone	string null	non	
departureDateLocal	string null	non	
departureTimeLocal	string null	non	
arrivalDateLocal	string null	non	
arrivalTimeLocal	string null	non	
departureAt	string null	non	
arrivalAt	string null	non	
returnDepartureAt	string null	non	
returnArrivalAt	string null	non	
flightType	FlightType null	non	
flightLayoverCities	array null	non	
trainTripType	TrainTripType null	non	
trainStopCities	array null	non	
carTripFlexibility	CarTripFlexibility null	non	
travelReference	string null	non	
acceptedCategories	array	oui	
categoryConditions	array null	non	
pricePerKgCents	integer null	non	D13 — carrier's single price per kg, in cents. Null = legacy PER_CATEGORY trip
checkedBag23PriceCents	integer null	non	PRC-04 — full 23kg checked bag flat rate (consumes 23kg of capacity). Null = not offered
cabinBag12PriceCents	integer null	non	PRC-04 — full 12kg cabin bag flat rate. Null = not offered
capacityKg	number null	non	CAP-01/D19 — carrier declared capacity in kg (immutable after publication). Required alongside pricePerKgCents to publish PER_KG (gate A28)
familyConditions	array null	non	D14 — per-family stance (NEW engine). Null/empty = all families accepted
pickupLocations	array null	non	
deliveryLocations	array null	non	
handDeliveryOnly	boolean	oui	
instantBooking	boolean	oui	
currencyCode	string	oui	
maxSlots	integer null	non	
notes	string null	non	
publish	boolean	oui	true = créer directement en PUBLISHED (si gates onboarding/Stripe/locations OK)

ErrorResponse — Enveloppe d'erreur standard (error-middleware, toute AppError)

Champ	Type	Requis	Description
status	string	oui	
message	string	oui	
errors	object	non	Erreurs par champ (formulaires) — exposé quand details.errors est présent
details		non	Contexte structuré 'safe' (ex: type=otp) — toujours exposé ; le reste hors prod uniquement

FamilyConditionMode — enum : ACCEPT, SURCHARGE, REFUSE — Carrier stance on a family: accept, surcharge (%), or refuse

FavoriteTripsResponse — Mes favoris — du plus récent au plus ancien, trajets passés inclus

Champ	Type	Requis	Description
trips	array	oui	
totalCount	integer	oui	

FlightType — enum : DIRECT , WITH_LAYOVER

HideTripRequest

Champ	Type	Requis	Description
reason	string	oui	

ImageKitAuthResponse — Paramètres d'authentification pour upload direct navigateur → ImageKit

Champ	Type	Requis	Description
success	boolean	oui	
token	string	oui	
expire	number	oui	Timestamp Unix d'expiration (~30 min)
signature	string	oui	
publicKey	string	non	IMAGEKIT_PUBLIC_KEY (absent si env non configuré)
urlEndpoint	string	non	IMAGEKIT_URL_ENDPOINT (absent si env non configuré)

LocationFlexibility — enum : EXACT , RADIUS , CITY_WIDE

LocationKind — enum : AIRPORT , TRAIN_STATION , CITY_AREA

ObjectId — string : Identifiant MongoDB (ObjectId sérialisé en hexadécimal)

ParcelCategory — enum : CLOTHES , SHOES , FASHION_ACCESSORIES , OTHER_ACCESSORIES , BOOKS , DOCUMENTS , SMALL_TOYS , PHONE , COMPUTER , OTHER_ELECTRONICS , CHECKED_BAG_23KG , CABIN_BAG_12KG — Enum actuel (pré-D14). La migration vers les 8 familles de risque CAT-02 fera l'objet d'une PR dédiée (mapping conservé).

ParcelFamily — enum : DOCUMENTS_PAPERS , CLOTHES_TEXTILE , FOOD_DRY_SEALED , ELECTRONICS_DEVICES , COSMETICS_CARE , PARTS_TOOLS , TOYS_CHILDCARE , MISC_ACCESSORIES — D14/CAT-02 — the 8 risk families (conformity/risk/protection, never price)

PricingParamsResponse — Server pricing parameters (D62 7A) — the wizard quotes with the single engine and these values

Champ	Type	Requis	Description
sizeCoef	object	oui	
minBillableKg	number	oui	
minTransportCents	integer	oui	
commissionPct	number	oui	
commissionFloorCents	integer	oui	
protectionExtendedPremiumCents	integer	oui	
protectionExtendedCapCents	integer	oui	
weightTolerancePct	number	oui	
referenceKg	number	oui	
version	integer	oui	

PublicCarrier

Champ	Type	Requis	Description
id	ObjectId	oui	
name	string null	non	
bio	string null	non	
isVerified	boolean	oui	
isSuperCarrier	boolean	oui	
ratingsAvg	number null	non	
ratingsCount	integer	oui	
totalTripsPublished	integer	oui	
totalParcelsCarried	integer	oui	

PublicNotFound — 404 de la route publique (court-circuite le middleware d'erreurs)

Champ	Type	Requis	Description
success	boolean	oui	
message	string	oui	

PublicPlace

Champ	Type	Requis	Description
label	string null	non	
placeId	string null	non	
city	string	oui	Non-null : requis par le gate de publication
cityCode	string null	non	
region	string null	non	
regionCode	string null	non	
country	string null	non	
countryCode	string null	non	
lat	number null	non	
lng	number null	non	
timezone	string null	non	

PublicTrip — Vue publique filtrée d'un trip PUBLISHED

Champ	Type	Requis	Description
id	ObjectId	oui	
status	string	oui	Seuls les trips PUBLISHED sont servis (404 sinon)
transportMode	TransportMode	oui	
tripType	TripType null	non	
origin	PublicPlace	oui	
destination	PublicPlace	oui	
dates	PublicTripDates	oui	
flightType	FlightType null	non	
trainTripType	TrainTripType null	non	
carTripFlexibility	CarTripFlexibility null	non	
flightLayoverCities	array	oui	
trainStopCities	array	oui	
travelReference	string null	non	
acceptedCategories	array	oui	
categoryConditions	array	oui	
pricePerKgCents	integer null	non	D13 — carrier's single price per kg, in cents. Null = legacy PER_CATEGORY trip
checkedBag23PriceCents	integer null	non	PRC-04 — full 23kg checked bag flat rate (consumes 23kg of capacity). Null = not offered
cabinBag12PriceCents	integer null	non	PRC-04 — full 12kg cabin bag flat rate. Null = not offered
capacityKg	number null	non	CAP-01/D19 — carrier declared capacity in kg (immutable after publication). Required alongside pricePerKgCents to publish PER_KG (gate A28)
familyConditions	array null	non	D14 — per-family stance (NEW engine). Null/empty = all families accepted
reservedKg	number null	non	CAP-02 — lets shippers derive remaining capacity client-side
pickupLocations	array	oui	
deliveryLocations	array	oui	
handDeliveryOnly	boolean	oui	
instantBooking	boolean	oui	
currencyCode	string	oui	
notes	string null	non	
maxSlots	integer null	oui	
bookedSlots	integer	oui	
remainingSlots	integer null	oui	max(0, maxSlots - bookedSlots) — null si capacité illimitée
minPriceCents	integer null	oui	
ticketVerified	boolean	oui	true ssi ticketVerificationStatus = VERIFIED
tripper	PublicTripper	oui	
publishedAt	string null	non	

Champ	Type	Requis	Description
isFavorite	boolean	non	D46 — true si l'utilisateur connecté a mis ce trajet en favori (isOptionallyAuthenticated)
viewsCount	integer	non	D5 / C-PR6 — vues dédoublonnées par visiteur et par jour (Redis), cette vue comprise ; absent si Redis indisponible

PublicTripDates

Champ	Type	Requis	Description
departureAt	string (date-time)	oui	Non-null : requis par le gate de publication
arrivalAt	string null	non	
returnDepartureAt	string null	non	
returnArrivalAt	string null	non	
departureDateLocal	string null	non	
arrivalDateLocal	string null	non	
departureTimeLocal	string null	non	
arrivalTimeLocal	string null	non	

PublicTripResponse

Champ	Type	Requis	Description
success	boolean	oui	
trip	PublicTrip	oui	

PublicTripper

Champ	Type	Requis	Description
id	ObjectId	oui	
publicSlug	string null	non	
firstName	string null	non	
lastInitial	string	oui	Initiale du nom, '' si absent (privacy)
avatarUrl	string null	oui	
memberSince	string (date-time)	oui	
carrier	PublicCarrier null	oui	

ReviewTicketRequest

Champ	Type	Requis	Description
decision	enum : VERIFY, REJECT	oui	
reason	TicketRejectionReason	non	Required when REJECT

SearchFacetsResponse

Champ	Type	Requis	Description
totalCount	integer	oui	Count avec le filtre mode appliqué
modeCount	object	oui	Counts par mode, calculés SANS le filtre mode courant
superTripperCount	integer	oui	
profileVerifiedCount	integer	oui	
instantBookingCount	integer	oui	
verifiedTicketCount	integer	oui	
familyCounts	object	non	D33 — par ParcelFamily : trips qui NE refusent PAS la famille (base sans filtre famille)

SearchLocale — enum : fr, en — Locale de formatage serveur des dates (défaut fr)

SearchTripsResponse

Champ	Type	Requis	Description
trips	array	oui	
nextCursor	string null	oui	id du dernier trip de la page — null si dernière page (pagination cursor-based)
totalCount	integer	oui	

SortOption — enum : earliest, lowestPrice, bestRated — Tri des résultats. lowestPrice trie sur comparablePriceCents (D33 : colis de référence 2 kg) et exclut les trips sans valeur.

TicketQueueItem

Champ	Type	Requis	Description
documentId	ObjectId	oui	
tripId	ObjectId	oui	
originCity	string	oui	
destinationCity	string	oui	
departureAt	string null	oui	
transportMode	string null	oui	
carrier	object	oui	
originalName	string null	oui	
contentType	string null	oui	
submittedAt	string (date-time)	oui	

TicketQueueResponse

Champ	Type	Requis	Description
items	array	oui	
expiredNow	integer	oui	

TicketRejectionReason — enum : ILLEGIBLE , DATES_MISMATCH , NAME_MISMATCH , SUSPICIOUS

TicketVerificationStatus — enum : NOT_SUBMITTED , PENDING , VERIFIED , REJECTED

TrainTripType — enum : DIRECT , WITH_CONNECTION

TransportMode — enum : PLANE , TRAIN , CAR

TransportModeFilter — enum : all , plane , train , car — Filtre mode de la recherche (all = tous)

Trip

Champ	Type	Requis	Description
id	ObjectId	oui	
userId	ObjectId	oui	
carrierPageId	ObjectId null	non	
status	TripStatus	oui	
currentStep	integer null	non	Étape du wizard (1-3)
transportMode	TransportMode null	non	
tripType	TripType null	non	
originLabel	string null	non	
originPlaceId	string null	non	
originCity	string null	non	
originCityCode	string null	non	
originRegion	string null	non	
originRegionCode	string null	non	
originCountry	string null	non	
originCountryCode	string null	non	
originLat	number null	non	
originLng	number null	non	
originTimezone	string null	non	
destinationLabel	string null	non	
destinationPlaceId	string null	non	
destinationCity	string null	non	
destinationCityCode	string null	non	
destinationRegion	string null	non	
destinationRegionCode	string null	non	
destinationCountry	string null	non	
destinationCountryCode	string null	non	
destinationLat	number null	non	
destinationLng	number null	non	
destinationTimezone	string null	non	
departureDateLocal	string null	non	

Champ	Type	Requis	Description
departureTimeLocal	string null	non	
arrivalDateLocal	string null	non	
arrivalTimeLocal	string null	non	
departureAt	string null	non	
arrivalAt	string null	non	
returnDepartureAt	string null	non	
returnArrivalAt	string null	non	
flightType	FlightType null	non	
flightLayoverCities	array null	non	
trainTripType	TrainTripType null	non	
trainStopCities	array null	non	
carTripFlexibility	CarTripFlexibility null	non	
travelReference	string null	non	
ticketVerificationStatus	TicketVerificationStatus null	non	
acceptedCategories	array	oui	
categoryConditions	array null	non	
pricePerKgCents	integer null	non	D13 — carrier's single price per kg, in cents. Null = legacy PER_CATEGORY trip
checkedBag23PriceCents	integer null	non	PRC-04 — full 23kg checked bag flat rate (consumes 23kg of capacity). Null = not offered
cabinBag12PriceCents	integer null	non	PRC-04 — full 12kg cabin bag flat rate. Null = not offered
capacityKg	number null	non	CAP-01/D19 — carrier declared capacity in kg (immutable after publication). Required alongside pricePerKgCents to publish PER_KG (gate A28)
familyConditions	array null	non	D14 — per-family stance (NEW engine). Null/empty = all families accepted
reservedKg	number null	non	CAP-02 — atomic server counter. remainingKg = capacityKg - reservedKg (derived, never stored)
pickupLocations	array null	non	
deliveryLocations	array null	non	
handDeliveryOnly	boolean	oui	
instantBooking	boolean	oui	D20 : sans effet sur la machine d'états v1 (badge)
currencyCode	string	oui	
maxSlots	integer null	non	null = capacité illimitée
bookedSlots	integer	oui	Slots réservés (chantier Booking)
notes	string null	non	
minPriceCents	integer null	non	min(categoryConditions.priceAmountCents) — null si aucune condition ; les trips PER_KG restent null (moteurs incomparables, exclus du tri lowestPrice — A28)
departureHourLocal	integer null	non	Heure locale de départ (0-23), pour les buckets de recherche
carrierRatingSnapshot	number null	non	Note carrier figée à la publication (tri bestRated)
documents	array null	non	
user	TripUserSummary null	non	
carrierPage	TripCarrierSummary null	non	
publishedAt	string null	non	
cancelledAt	string null	non	
archivedAt	string null	non	
isDeleted	boolean	non	
deletedAt	string null	non	
createdAt	string (date-time)	oui	
updatedAt	string (date-time)	oui	

TripAction — enum : edit, publish, unpublish, pause, resume, cancel, restore, archive, delete — Actions de la state machine trip (canPerform/getAllowedActions). Renvoyées dans allowedActions pour piloter les CTAs front.

TripCarrierSummary — Select partiel de la carrierPage (vue détail owner)

Champ	Type	Requis	Description
id	ObjectId	oui	
name	string null	non	
ratingsAvg	number null	non	
ratingsCount	integer	oui	

Champ	Type	Requis	Description
isVerified	boolean	oui	

TripDocument

Champ	Type	Requis	Description
id	ObjectId	oui	
type	TripDocumentType	oui	
status	TripDocumentStatus	oui	
url	string	oui	URL ImageKit du document
fileId	string	non	Identifiant ImageKit (pour suppression)
tripId	ObjectId	non	
uploadedByUserId	ObjectId null	non	
originalName	string null	non	
mimeType	string null	non	
sizeBytes	integer null	non	
title	string null	non	
description	string null	non	
verifiedAt	string null	non	
rejectedAt	string null	non	
rejectionReason	string null	non	
createdAt	string (date-time)	non	
updatedAt	string (date-time)	non	

TripDocumentStatus — enum : PENDING , VERIFIED , REJECTED , EXPIRED — Statut de modération du document. EXPIRED (C-PR4, D57 8A) : billet resté en attente sur un trajet déjà parti.

TripDocumentType — enum : TICKET_PROOF , ITINERARY_PROOF , VEHICLE_PROOF , IDENTITY_PROOF , OTHER — Type de justificatif. TICKET_PROOF pilote ticketVerificationStatus (NOT_SUBMITTED → PENDING à l'ajout, retour à NOT_SUBMITTED si dernier ticket supprimé).

TripFamilyCondition — NEW engine family condition (D14) — coexists with legacy CategoryCondition until cleanup

Champ	Type	Requis	Description
familyKey	ParcelFamily	oui	
mode	FamilyConditionMode	oui	
surchargePct	integer null	non	Required when mode=SURCHARGE (e.g. electronics +20%)

TripFavoriteState — État du favori après l'action (idempotent)

Champ	Type	Requis	Description
tripId	ObjectId	oui	
isFavorite	boolean	oui	

TripLocationPoint

Champ	Type	Requis	Description
kind	LocationKind	oui	
details	string null	non	
flexibility	LocationFlexibility	oui	
radiusKm	number null	non	Requis si flexibility=RADIUS

TripMutationResponse — Réponse des mutations renvoyant le trip (sans allowedActions)

Champ	Type	Requis	Description
success	boolean	oui	
message	string	oui	
trip	Trip	oui	

TripResponse

Champ	Type	Requis	Description
success	boolean	oui	
trip	TripWithActions	oui	

TripStatus — enum : DRAFT , PUBLISHED , PAUSED , COMPLETED , CANCELLED , ARCHIVED

TripType — enum : ONE_WAY , ROUND_TRIP

TripUserSummary — Select partiel du user (vue détail owner)

Champ	Type	Requis	Description
id	ObjectId	oui	
firstName	string null	non	
lastName	string null	non	
avatar	object null	non	

TripWithActions — Trip + allowedActions (getAllowedActions) — pilote les CTAs front

Champ	Type	Requis	Description
id	ObjectId	oui	
userId	ObjectId	oui	
carrierPageId	ObjectId null	non	
status	TripStatus	oui	
currentStep	integer null	non	Étape du wizard (1-3)
transportMode	TransportMode null	non	
tripType	TripType null	non	
originLabel	string null	non	
originPlaceId	string null	non	
originCity	string null	non	
originCityCode	string null	non	
originRegion	string null	non	
originRegionCode	string null	non	
originCountry	string null	non	
originCountryCode	string null	non	
originLat	number null	non	
originLng	number null	non	
originTimezone	string null	non	
destinationLabel	string null	non	
destinationPlaceId	string null	non	
destinationCity	string null	non	
destinationCityCode	string null	non	
destinationRegion	string null	non	
destinationRegionCode	string null	non	
destinationCountry	string null	non	
destinationCountryCode	string null	non	
destinationLat	number null	non	
destinationLng	number null	non	
destinationTimezone	string null	non	
departureDateLocal	string null	non	
departureTimeLocal	string null	non	
arrivalDateLocal	string null	non	
arrivalTimeLocal	string null	non	
departureAt	string null	non	
arrivalAt	string null	non	
returnDepartureAt	string null	non	
returnArrivalAt	string null	non	
flightType	FlightType null	non	
flightLayoverCities	array null	non	
trainTripType	TrainTripType null	non	
trainStopCities	array null	non	
carTripFlexibility	CarTripFlexibility null	non	
travelReference	string null	non	

Champ	Type	Requis	Description
ticketVerificationStatus	TicketVerificationStatus null	non	
acceptedCategories	array	oui	
categoryConditions	array null	non	
pricePerKgCents	integer null	non	D13 — carrier's single price per kg, in cents. Null = legacy PER_CATEGORY trip
checkedBag23PriceCents	integer null	non	PRC-04 — full 23kg checked bag flat rate (consumes 23kg of capacity). Null = not offered
cabinBag12PriceCents	integer null	non	PRC-04 — full 12kg cabin bag flat rate. Null = not offered
capacityKg	number null	non	CAP-01/D19 — carrier declared capacity in kg (immutable after publication). Required alongside pricePerKgCents to publish PER_KG (gate A28)
familyConditions	array null	non	D14 — per-family stance (NEW engine). Null/empty = all families accepted
reservedKg	number null	non	CAP-02 — atomic server counter. remainingKg = capacityKg - reservedKg (derived, never stored)
pickupLocations	array null	non	
deliveryLocations	array null	non	
handDeliveryOnly	boolean	oui	
instantBooking	boolean	oui	D20 : sans effet sur la machine d'états v1 (badge)
currencyCode	string	oui	
maxSlots	integer null	non	null = capacité illimitée
bookedSlots	integer	oui	Slots réservés (chantier Booking)
notes	string null	non	
minPriceCents	integer null	non	min(categoryConditions.priceAmountCents) — null si aucune condition ; les trips PER_KG restent null (moteurs incomparables, exclus du tri lowestPrice — A28)
departureHourLocal	integer null	non	Heure locale de départ (0-23), pour les buckets de recherche
carrierRatingSnapshot	number null	non	Note carrier figée à la publication (tri bestRated)
documents	array null	non	
user	TripUserSummary null	non	
carrierPage	TripCarrierSummary null	non	
publishedAt	string null	non	
cancelledAt	string null	non	
archivedAt	string null	non	
isDeleted	boolean	non	
deletedAt	string null	non	
createdAt	string (date-time)	oui	
updatedAt	string (date-time)	oui	
allowedActions	AllowedActions	oui	

TripsListResponse

Champ	Type	Requis	Description
success	boolean	oui	
trips	array	oui	
count	integer	oui	

UiParcelCategory — enum : clothes, shoes, fashion-accessories, other-accessories, books, documents, small-toys, phone, computer, other-electronics, checked-bag-23kg, cabin-bag-12kg — Catégorie colis en convention UI (kebab-case)

UiTransportMode — enum : plane, train, car — Mode de transport en convention UI (≠ enum Prisma)

UnauthorizedResponse — 401 du middleware isAuthenticated (token absent, invalide, expiré, ou compte introuvable) — hors error-middleware

Champ	Type	Requis	Description
message	string	oui	

UnhandledError — 500 non géré (exception hors AppError) — champ error, pas message

Champ	Type	Requis	Description
status	string	oui	
error	string	oui	

UpdateTripBody

Champ	Type	Requis	Description
transportMode	TransportMode	non	
tripType	TripType	non	
originLabel	string	non	
originPlaceId	string null	non	
originCity	string null	non	
originCityCode	string null	non	
originRegion	string null	non	
originRegionCode	string null	non	
originCountry	string null	non	
originCountryCode	string null	non	
originLat	number null	non	
originLng	number null	non	
originTimezone	string null	non	
destinationLabel	string	non	
destinationPlaceId	string null	non	
destinationCity	string null	non	
destinationCityCode	string null	non	
destinationRegion	string null	non	
destinationRegionCode	string null	non	
destinationCountry	string null	non	
destinationCountryCode	string null	non	
destinationLat	number null	non	
destinationLng	number null	non	
destinationTimezone	string null	non	
departureDateLocal	string null	non	
departureTimeLocal	string null	non	
arrivalDateLocal	string null	non	
arrivalTimeLocal	string null	non	
departureAt	string null	non	
arrivalAt	string null	non	
returnDepartureAt	string null	non	
returnArrivalAt	string null	non	
flightType	FlightType null	non	
flightLayoverCities	array null	non	
trainTripType	TrainTripType null	non	
trainStopCities	array null	non	
carTripFlexibility	CarTripFlexibility null	non	
travelReference	string null	non	
acceptedCategories	array	non	
categoryConditions	array null	non	
pricePerKgCents	integer null	non	D13 — carrier's single price per kg, in cents. Null = legacy PER_CATEGORY trip
checkedBag23PriceCents	integer null	non	PRC-04 — full 23kg checked bag flat rate (consumes 23kg of capacity). Null = not offered
cabinBag12PriceCents	integer null	non	PRC-04 — full 12kg cabin bag flat rate. Null = not offered
capacityKg	number null	non	CAP-01/D19 — carrier declared capacity in kg (immutable after publication). Required alongside pricePerKgCents to publish PER_KG (gate A28)
familyConditions	array null	non	D14 — per-family stance (NEW engine). Null/empty = all families accepted
pickupLocations	array null	non	
deliveryLocations	array null	non	
handDeliveryOnly	boolean	non	
instantBooking	boolean	non	
currencyCode	string	non	
maxSlots	integer null	non	
notes	string null	non	
publish	boolean	non	true = créer directement en PUBLISHED (si gates onboarding/Stripe/locations OK)

Champ	Type	Requis	Description
id	ObjectId	oui	
fromCity	string	oui	
fromCityCode	string	non	
fromCountry	string	non	
toCity	string	oui	
toCityCode	string	non	
toCountry	string	non	
travelDate	string	oui	Formaté serveur selon locale
departureTime	string	oui	
arrivalTime	string	oui	
nextDay	boolean	non	Arrivée le lendemain (absent si false)
durationMinutes	integer	non	
stopovers	integer	non	
stopoverCity	string	non	Présent uniquement si exactement 1 escale
minPrice	number	oui	En unités (euros), PAS en centimes — déjà divisé par 100. 0 pour un trip PER_KG (voir pricePerKg)
pricePerKg	number null	non	D13 — moteur PER_KG : prix au kilo en unités (euros). Null = trip legacy PER_CATEGORY
remainingKg	number null	non	CAP-02 — capacityKg – reservedKg, dérivé. Null si legacy
weightKg	number	non	D33 V2 — écho du poids saisi par l'Expéditeur (kg) ; absent sinon
transportForWeight	number null	non	D33 V2 — transport (net Voyageur) pour ce poids, en euros. Null = aucun moteur
totalForWeight	number null	non	D33 V2 — total Expéditeur (transport + service D16) pour ce poids, en euros
familyConditions	array	non	D14 — positions ≠ ACCEPT du Voyageur (compact). Absent/vide = tout accepté
pricesByCategory	object	oui	Clés = UiParcelCategory, valeurs en unités (euros)
currency	string	oui	Symbole, pas le code ISO
transportMode	UiTransportMode	oui	
allowedCategories	array	oui	
remainingSlots	integer	non	Absent si capacité illimitée (maxSlots null)
superTripper	boolean	oui	
profileVerified	boolean	oui	
instantBooking	boolean	oui	
verifiedTicket	boolean	oui	
rating	number	non	Absent si ratingsCount = 0 (jamais de 0,0)
reviewCount	integer	non	
travelerFirstName	string	non	
travelerLastName	string	non	Initiale uniquement (privacy)
travelerAvatarUrl	string	non	
isFavorite	boolean	non	D46 — true si l'utilisateur connecté a mis ce trajet en favori (absent/false pour un visiteur)
viewsCount	integer	non	D5 / C-PR6 — vues de la page publique, dédoublonnées par visiteur et par jour (Redis) ; absent si Redis indisponible

deal-service — port 6003

Réservations (deals), paiement, transport, litiges, versements, notation, suivi destinataire. Document vivant : `http://localhost:6003/docs` (Scalar) et `apps/deal-service/openapi.json`. Via le gateway : `http://localhost:8080/api` (préfixes ci-dessous).

deal-service › system

Health & meta

`GET /openapi.json`

This OpenAPI 3.1 document

operationId : `getOpenApiDocument` · Authentification : aucune (public)

Code	Réponse
200	objet { } — OpenAPI 3.1 document (generated from Zod)

GET /health

Health check

operationId : getHealth · Authentication : aucune (public)

Code	Réponse
200	objet { status, service } — Service is up (no DB dependency)

deal-service › deals

Deal read views, role-based (auth required)

GET /deals

Deals of one of my trips (carrier view)

operationId : listTripDeals · Authentication : cookieAuth, bearerAuth

Returns the deals attached to a trip OWNED by the caller (A12: ownership checked by direct read-only Trip lookup). Carrier view: no delivery code, no code hash, no shipper totals — earnings only.

Paramètre	Dans	Requis	Type	Description
tripId	query	oui	ObjectId	Trip whose deals are requested — must belong to the caller
status	query	non	BookingStatus	Filter by booking status (exact match)

Code	Réponse
200	TripDealsResponse — Deals of the trip (CarrierBookingView[])
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /deals

Create a deal request (PENDING) — step 2 (D37)

operationId : createBooking · Authentication : cookieAuth, bearerAuth

Re-validates everything server-side (trip bookable, quote identical — D17, payment authorized and matching), then in ONE Mongo transaction: conditional reservedKg increment (CAP-01), Booking with 5 frozen snapshots, and 2 outbox events (booking.requested, booking.payment_authorized). The carrier is notified by the relay; the 24h acceptance window starts (DEA-01).

Corps (requis) : CreateBookingRequest

Code	Réponse
201	CreateBookingResponse — Deal created (PENDING)
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ QUOTE_DIVERGENCE / CAPACITY_EXCEEDED / FAMILY_REFUSED / TRIP_NOT_BOOKABLE / OWN_TRIP / PAYMENT_NOT_AUTHORIZED / PAYMENT_MISMATCH / PAYMENT_ALREADY_USED / NEW_ACCOUNT_CAP (D71: details.cap ∈ DECLARED_VALUE / WEIGHT / SHIPMENTS_PER_MONTH, limit, value)
500	UnhandledError — Unhandled server error

POST /deals/payment-intents

Authorize the shipper's payment for a quote (step 1 of a request — D37)

operationId : createPaymentIntent · Authentication : cookieAuth, bearerAuth

The server recomputes the quote with the single pricing engine (@packages/pricing, D34) and authorizes the total with the PaymentProvider (D11, manual capture — captured at acceptance, D31). Nothing is persisted: an abandoned intent simply expires. 409 with details.code when the total the shipper saw differs (QUOTE_DIVERGENCE), the family is refused, or the trip is not bookable.

Corps (requis) : CreatePaymentIntentRequest

Code	Réponse
201	CreatePaymentIntentResponse — Authorization created (clientSecret null for FAKE)
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
404	ErrorResponse — Deal or trip not found (NotFoundError)

Code	Réponse
409	ErrorResponse — Business conflict — details.code ∈ QUOTE_DIVERGENCE / CAPACITY_EXCEEDED / FAMILY_REFUSED / TRIP_NOT_BOOKABLE / OWN_TRIP / PAYMENT_NOT_AUTHORIZED / PAYMENT_MISMATCH / PAYMENT_ALREADY_USED / NEW_ACCOUNT_CAP (D71: details.cap ∈ DECLARED_VALUE / WEIGHT / SHIPMENTS_PER_MONTH, limit, value)
500	UnhandledError — Unhandled server error

POST /deals/{id}/accept

Accept a deal request (carrier) — capture at acceptance (D39)

operationId : acceptDeal · Authentication : cookieAuth, bearerAuth

Carrier only, charter checkbox required. The D31 gate (completed profile + Stripe onboarding) is enforced HERE — no longer at trip publication. The shipper's authorization is CAPTURED (money moves now — an authorization expires in ~7 days, capturing at D-1 would break early acceptances), then in ONE Mongo transaction: conditional PENDING→ACCEPTED, acceptedAt/capturedAt, outbox booking.accepted.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : AcceptDealRequest

Code	Réponse
200	DealTransitionResponse — Deal accepted (refundAmountCents null)
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (the state machine refused: status, role or guard — details carry its reason) / CARRIER_ONBOARDING_REQUIRED (D31 gate: profile or Stripe onboarding incomplete) / PAYMENT_STATE_CONFLICT (the provider-side payment state forbids the operation)
500	UnhandledError — Unhandled server error

POST /deals/{id}/decline

Decline a deal request (carrier)

operationId : declineDeal · Authentication : cookieAuth, bearerAuth

Carrier only, optional reason among 5 (spec É2). The authorization is released (never captured), then in ONE Mongo transaction: conditional PENDING→DECLINED, reserved kg released (CAP-02), outbox booking.declined + booking.refund_issued (full amount).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (facultatif) : DeclineDealRequest

Code	Réponse
200	DealTransitionResponse — Deal declined, full amount returned to the shipper
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (the state machine refused: status, role or guard — details carry its reason) / CARRIER_ONBOARDING_REQUIRED (D31 gate: profile or Stripe onboarding incomplete) / PAYMENT_STATE_CONFLICT (the provider-side payment state forbids the operation)
500	UnhandledError — Unhandled server error

POST /deals/{id}/cancel

Cancel a deal (shipper) — ANN-01 policy

operationId : cancelDeal · Authentication : cookieAuth, bearerAuth

Shipper only. PENDING: the authorization is released in full. ACCEPTED (payment captured — D39): a real refund per ANN-01 — 100% until 48h before departure, then a 50% retention (CANCEL_LATE_RETENTION_PCT, owed to the carrier — paid out with the B4 payout infrastructure). After PICKED_UP cancellation is impossible (dispute is the only path). One Mongo transaction: conditional transition, kg released (CAP-02), outbox booking.cancelled + booking.refund_issued.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (facultatif) : CancelDealRequest

Code	Réponse
200	DealTransitionResponse — Deal cancelled, refundAmountCents per ANN-01
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (the state machine refused: status, role or guard — details carry its reason) / CARRIER_ONBOARDING_REQUIRED (D31 gate: profile or Stripe onboarding incomplete) / PAYMENT_STATE_CONFLICT (the provider-side payment state forbids the operation)
500	UnhandledError — Unhandled server error

POST /deals/{id}/pickup

Confirm the parcel pickup (carrier) — generates the delivery code

operationId : confirmPickup · Authentication : cookieAuth, bearerAuth

Carrier only. Requires ALL 5 inspection items (CNF-04) and 1 to 5 photo URLs already uploaded to ImageKit by the browser (D42 — no bytes go through this API). ACCEPTED→PICKED_UP in ONE Mongo transaction with the server-generated 6-digit delivery code stored twice (bcrypt for validation, AES-256-GCM for shipper re-display — D43), the frozen checklist and photos, outbox booking.picked_up. The code is revealed to the SHIPPER on GET /deals/:id only — never in this response, never to the carrier.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : ConfirmPickupRequest

Code	Réponse
200	DealTransitionResponse — Parcel picked up (status PICKED_UP, refundAmountCents null)
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused, or a concurrent write won) / PAYMENT_STATE_CONFLICT (refund impossible) / DELIVERY_CODE_INVALID (details.attemptsLeft) / DELIVERY_LOCKED (details.lockedUntil — 3 failures, 15 min) / DELIVERY_CODE_UNAVAILABLE (pre-B3 record without a code) / TRACKING_STEP_NOT_ALLOWED (strict sequence, duplicate or wrong status) / CODE_REGENERATION_LIMIT (5 reached, or not in transit)
500	UnhandledError — Unhandled server error

POST /deals/{id}/pickup/refuse

Refuse the parcel at pickup (carrier) — full refund, no penalty

operationId : refusePickup · Authentication : cookieAuth, bearerAuth

Carrier only, optional reason among 5 (A40). The captured payment is REFUNDED in full at the provider (money first), then ACCEPTED→CANCELLED (closedBy CARRIER, pickupRefusalReason), reserved kg released (CAP-02), outbox booking.picked_up_refused + booking.refund_issued. No reputation penalty (CNF-07).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (facultatif) : RefusePickupRequest

Code	Réponse
200	DealTransitionResponse — Deal cancelled, full amount returned to the shipper
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused, or a concurrent write won) / PAYMENT_STATE_CONFLICT (refund impossible) / DELIVERY_CODE_INVALID (details.attemptsLeft) / DELIVERY_LOCKED (details.lockedUntil — 3 failures, 15 min) / DELIVERY_CODE_UNAVAILABLE (pre-B3 record without a code) / TRACKING_STEP_NOT_ALLOWED (strict sequence, duplicate or wrong status) / CODE_REGENERATION_LIMIT (5 reached, or not in transit)
500	UnhandledError — Unhandled server error

POST /deals/{id}/events

Confirm an optional tracking milestone (carrier)

operationId : confirmTrackingStep · Authentication : cookieAuth, bearerAuth

Carrier only, while PICKED_UP. Strict sequence AT_AIRPORT → FLIGHT_DEPARTED → FLIGHT_ARRIVED, no skip, no duplicate (409 TRACKING_STEP_NOT_ALLOWED). No status transition: the milestone is pushed in ONE transaction guarded by its absence, outbox booking.tracking_event (shipper in-app only, no email). The 5-second undo is client-side (A39): call this endpoint AFTER the undo window — there is no server undo.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : ConfirmTrackingStepRequest

Code	Réponse
200	TrackingStepResponse — Milestone confirmed — full sequence returned
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused, or a concurrent write won) / PAYMENT_STATE_CONFLICT (refund impossible) / DELIVERY_CODE_INVALID (details.attemptsLeft) / DELIVERY_LOCKED (details.lockedUntil — 3 failures, 15 min) / DELIVERY_CODE_UNAVAILABLE (pre-B3 record without a code) / TRACKING_STEP_NOT_ALLOWED (strict sequence, duplicate or wrong status) / CODE_REGENERATION_LIMIT (5 reached, or not in transit)
500	UnhandledError — Unhandled server error

POST /deals/{id}/code/regenerate

Regenerate the delivery code (shipper) — max 5

operationId : regenerateDeliveryCode · Authentication : cookieAuth, bearerAuth

Shipper only, while PICKED_UP, at most MAX_CODE_REGENERATIONS (5). A new code replaces the previous one (old hash invalid immediately), delivery attempts and lock are reset, outbox booking.code_regenerated (count only — the code never travels in events or emails). The NEW code is returned here and on GET /deals/:id (shipper view). Optimistic guard on the regeneration counter (two clicks = one regeneration).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Code	Réponse
200	RegenerateCodeResponse — New 6-digit code and remaining regenerations
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused, or a concurrent write won) / PAYMENT_STATE_CONFLICT (refund impossible) / DELIVERY_CODE_INVALID (details.attemptsLeft) / DELIVERY_LOCKED (details.lockedUntil — 3 failures, 15 min) / DELIVERY_CODE_UNAVAILABLE (pre-B3 record without a code) / TRACKING_STEP_NOT_ALLOWED (strict sequence, duplicate or wrong status) / CODE_REGENERATION_LIMIT (5 reached, or not in transit)
500	UnhandledError — Unhandled server error

POST /deals/{id}/deliver

Validate the delivery code (carrier) — PICKED_UP → DELIVERED

operationId : deliverDeal · Authentication : cookieAuth, bearerAuth

Carrier only. The 6-digit code given by the recipient is compared with bcrypt server-side. Wrong code: attempts +1 (conditional write on the counter read — A38) → 409 DELIVERY_CODE_INVALID with attemptsLeft; 3rd failure → 15-minute lock AND counter reset → 409 DELIVERY_LOCKED with lockedUntil; an active lock is refused by the state machine before any comparison. Valid code: PICKED_UP→DELIVERED in ONE transaction, payoutDueAt = deliveredAt + 4 days (shipper verification window, B4), outbox booking.delivered.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : DeliverDealRequest

Code	Réponse
200	DeliverDealResponse — Delivered — verification window started
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused, or a concurrent write won) / PAYMENT_STATE_CONFLICT (refund impossible) / DELIVERY_CODE_INVALID (details.attemptsLeft) / DELIVERY_LOCKED (details.lockedUntil — 3 failures, 15 min) / DELIVERY_CODE_UNAVAILABLE (pre-B3 record without a code) / TRACKING_STEP_NOT_ALLOWED (strict sequence, duplicate or wrong status) / CODE_REGENERATION_LIMIT (5 reached, or not in transit)

Code	Réponse
500	UnhandledError — Unhandled server error

POST /deals/{id}/confirm

Confirm the delivery early (shipper) — DELIVERED → COMPLETED, payout released (INV-3: final)

operationId : confirmDeal · Authentication : cookieAuth, bearerAuth

Shipper only, while DELIVERED. ONE conditional transaction: COMPLETED, completedBy=SHIPPER, payoutStatus=PENDING, outbox booking.completed (D49). Then the carrier payout is executed INLINE (A67): PaymentProvider.transfer of pricing.transportCents (carrier net, D50) to the carrier's Connect account, idempotency key = booking id, source_transaction = capture charge (A69). Success → payoutStatus SENT + outbox booking.payout_sent; provider refusal → FAILED (retried by the payout cron every 5 min, up to 10 attempts). Irreversible: the right to dispute is gone.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Code	Réponse
200	ConfirmDealResponse — Completed — payoutStatus tells whether the transfer went through
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused: not DELIVERED, verification window over, parcel in transit for less than 48h after departure, or a concurrent write won — details carry its reason)
500	UnhandledError — Unhandled server error

POST /deals/{id}/dispute

Open a dispute (shipper) — DELIVERED (before D+4) or PICKED_UP (not delivered, 48h after departure) → DISPUTED

operationId : disputeDeal · Authentication : cookieAuth, bearerAuth

Shipper only. From DELIVERED before payoutDueAt (INV-4), or from PICKED_UP once the trip departure is 48h past (category MUST be NOT_DELIVERED — 400 otherwise). Body: category, description ≥ 50 chars, pledgeAccepted=true, up to 5 ImageKit photo URLs (D42), optional desiredOutcome. ONE transaction: DISPUTED, ticket YAM-XXXX (CSPRNG, unique — redrawn on collision), payoutStatus=FROZEN when a payout was scheduled (INV-5), Dispute document created (D51), outbox booking.disputed. Terminal in v1 — resolution belongs to the admin (chantier C).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : DisputeDealRequest

Code	Réponse
200	DisputeDealResponse — Dispute opened — payout frozen, ticket issued
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused: not DELIVERED, verification window over, parcel in transit for less than 48h after departure, or a concurrent write won — details carry its reason)
500	UnhandledError — Unhandled server error

POST /deals/{id}/tracking-link

Recipient tracking link (D69) — shipper only, one per deal, created on demand

operationId : issueTrackingLink · Authentication : cookieAuth, bearerAuth

Returns the public tracking token and path (/track/{token}) plus the recipient contact the shipper entered. 403 for the carrier, 409 TRACKING_NOT_AVAILABLE before acceptance or after a closure without delivery.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Code	Réponse
200	TrackingLinkResponse — Tracking link
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)

Code	Réponse
409	ErrorResponse — Business conflict — details.code ∈ QUOTE_DIVERGENCE / CAPACITY_EXCEEDED / FAMILY_REFUSED / TRIP_NOT_BOOKABLE / OWN_TRIP / PAYMENT_NOT_AUTHORIZED / PAYMENT_MISMATCH / PAYMENT_ALREADY_USED / NEW_ACCOUNT_CAP (D71: details.cap ∈ DECLARED_VALUE / WEIGHT / SHIPMENTS_PER_MONTH, limit, value)
500	UnhandledError — Unhandled server error

GET /track/{token}

Recipient tracking page (D69) — public, minimal content

operationId : getPublicTracking · Authentication : aucune (public)

No session. Milestones (accepted → picked up → in transit → arrived → delivered, or closed), first names, corridor and dates. Never an address, a phone number, the delivery code, photos or amounts. 404 once the recipient snapshot is redacted (D63 5A), the link revoked or the deal deleted.

Paramètre	Dans	Requis	Type	Description
token	path	oui	string	

Code	Réponse
200	PublicTrackingResponse — Tracking view
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

GET /deals/{id}/rating

Rating context (B5, D53) — who I rate, my rating, whether the other rated, reveal

operationId : getDealRatingContext · Authentication : cookieAuth, bearerAuth

Either party. Double-blind: counterpartRating is null until both rated or the 14-day window elapsed. canRate is the state machine verdict (COMPLETED, window open, not rated yet by this role).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Code	Réponse
200	RatingContextResponse — Rating context
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /deals/{id}/rating

Rate the other party (B5, D53) — once per role, COMPLETED, within 14 days

operationId : submitDealRating · Authentication : cookieAuth, bearerAuth

ONE transaction: Review (revealedAt null) + booking mark (optimistic lock on updatedAt). If the counterpart had already rated, both reviews are revealed now and outbox booking.rating_revealed (BOTH_RATED) is written; otherwise the cron reveals at the end of the window (WINDOW_ELAPSED). Only revealed reviews feed the public reputation (D29^①). Criteria outside the rated role are dropped.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : SubmitRatingRequest

Code	Réponse
201	SubmitRatingResponse — Rating recorded
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused: not DELIVERED, verification window over, parcel in transit for less than 48h after departure, or a concurrent write won — details carry its reason)
500	UnhandledError — Unhandled server error

POST /deals/{id}/dispute/statement

Carrier's statement on an open dispute (C-PR2, D55) — once, ≥ 50 chars, ≤ 5 photos
operationId : submitCarrierDisputeStatement · Authentication : cookieAuth, bearerAuth

Carrier only, status DISPUTED, dispute OPEN. Writes the statement + outbox booking.dispute_carrier_responded in one transaction. The admin may decide as soon as the statement is in, or 72h after the dispute was filed.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : CarrierDisputeStatementRequest

Code	Réponse
201	CarrierDisputeStatementResponse — Statement recorded
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused: not DELIVERED, verification window over, parcel in transit for less than 48h after departure, or a concurrent write won — details carry its reason)
500	UnhandledError — Unhandled server error

GET /deals/{id}

One deal, role-based view
operationId : getDeal · Authentication : cookieAuth, bearerAuth

The DTO shape depends on the authenticated caller's role in the deal: ShipperBookingView (full pricing, delivery code surface) or CarrierBookingView (earnings only — the delivery code NEVER appears in any carrier payload). allowedActions = getAllowedActions(booking, role): the frontend reflects, never decides.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Code	Réponse
200	DealResponse — The deal, shaped by viewerRole
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

deal-service · admin

Back-office (chantier C, D54) — ADMIN session with TOTP only

GET /admin/disputes

Arbitration queue (chantier C, D54) — open disputes + retentions held for mediation
operationId : adminListArbitrationQueue · Authentication : cookieAuth, bearerAuth

ADMIN session only (admin_access_token cookie carrying amr totp). One queue, two kinds: DISPUTE (DISPUTED + Dispute OPEN) and RETENTION (CANCELLED after departure without pickup, retentionDisposition HELD_FOR_MEDIATION, A81). Oldest first.

Code	Réponse
200	ArbitrationQueueResponse — Queue
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
500	UnhandledError — Unhandled server error

GET /admin/disputes/{id}

Mediation file (chantier C, D54) — both parties' evidence, money state, never the delivery code
operationId : adminGetDisputeFile · Authentication : cookieAuth, bearerAuth

ADMIN session only. Declaration photos, pickup checklist + photos, tracking events, delivery photos, dispute file (description, photos, desired outcome), money snapshot and payout/refund state. Reading a file is journaled (AdminAction DISPUTE_VIEWED). 404 when the deal is not awaiting arbitration.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Code	Réponse
200	AdminDisputeFile — File
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /admin/disputes/{id}/resolve

Decide a dispute (C-PR2, D55) — REJECTED / PARTIAL_REFUND / FULL_REFUND, reason ≥ 50 chars, irreversible

operationId : adminResolveDispute · Authentication : cookieAuth, bearerAuth

ADMIN session only. Refund FIRST (provider), then ONE transaction: DISPUTED → COMPLETED (rejected/partial) or → CANCELLED (full), dispute resolution, internal 'disputes lost' counter, AdminAction DISPUTE_RESOLVED, outbox booking.dispute_resolved. Then the carrier transfer through the payout executor (D49, retried by the cron on failure). 409 while the carrier still has time to answer.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : AdminResolveDisputeRequest

Code	Réponse
200	AdminResolutionResponse — Decided
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused: not DELIVERED, verification window over, parcel in transit for less than 48h after departure, or a concurrent write won — details carry its reason)
500	UnhandledError — Unhandled server error

POST /admin/disputes/{id}/retention

Arbitrate a held retention (C-PR2, D55 3A) — COMPENSATE_CARRIER (pro-rata A79) or RESTITUTE_SHIPPER

operationId : adminResolveRetention · Authentication : cookieAuth, bearerAuth

ADMIN session only. CANCELLED + retentionDisposition HELD_FOR_MEDIATION. Amounts are server-computed. Journalled (RETENTION_ARBITRATED).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : AdminResolveRetentionRequest

Code	Réponse
200	AdminResolutionResponse — Arbitrated
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ TRANSITION_NOT_ALLOWED (state machine refused: not DELIVERED, verification window over, parcel in transit for less than 48h after departure, or a concurrent write won — details carry its reason)
500	UnhandledError — Unhandled server error

GET /admin/finances/queue

Exception queues (D58 2A) — failed payouts, reversed transfers, retentions held

operationId : adminListFinanceQueue · Authentication : adminCookieAuth

Permission finances.read. FAILED = payoutStatus FAILED on COMPLETED / CANCELLED deals (reason kind, attempts, next retry — A111); REVERSED = transfer.reversed not yet resolved by an admin; HELD = retention HELD_FOR_MEDIATION. Amounts come from the booking, never recomputed.

Paramètre	Dans	Requis	Type	Description
kind	query	non	FinanceQueueKind	FAILED (défaut) · REVERSED · HELD

Code	Réponse
200	FinanceQueueResponse — Queue
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
500	UnhandledError — Unhandled server error

GET /admin/deals/{id}/money

Money file of any deal (D58 4A) — snapshot, payment, payout, retention, timeline, journal

operationId : adminGetDealMoneyFile · Authentication : adminCookieAuth

Permission finances.read. Provider identifiers in clear (finance needs them). Reading is journaled (AdminAction DEAL_MONEY_VIEWED). allowedActions reflects state AND conflict of interest.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Code	Réponse
200	AdminDealMoneyFile — File
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /admin/deals/{id}/money/reconcile

Reconcile with the payment provider (D58 4A, A112) — read-only, journaled

operationId : adminReconcileDeal · Authentication : adminCookieAuth

Permission finances.read. Calls PaymentProvider.inspect (intent, refunds, transfer) and lists divergences with the database (REFUND_NOT_RECORDED = a refund left the provider without a database write, D39). Never modifies the deal. Journal DEAL_RECONCILED.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Code	Réponse
200	PaymentReconciliation — Reconciliation
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /admin/deals/{id}/payout/retry

Retry a payout now (D58 3A) — same executor, idempotent

operationId : adminRetryPayout · Authentication : adminCookieAuth

Permission payouts.retry. Deal COMPLETED or CANCELLED with payoutStatus FAILED or PENDING; 403 when the admin is a party. Runs the unique payout executor (A65) without waiting for the retry schedule. Journal PAYOUT_RETRIED.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Code	Réponse
200	RetryPayoutResponse — Outcome
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /admin/deals/{id}/payout/reversal

Resolve a reversed transfer (D58 3A) — RESENT or WRITTEN_OFF, reason ≥ 20

operationId : adminResolvePayoutReversal · Authentication : adminCookieAuth

Permission payouts.resolve. Only on payoutStatus REVERSED not yet resolved (optimistic guard). RESENT: PENDING + a NEW idempotency key, then the executor sends a new transfer; WRITTEN_OFF: closed, nothing is sent. Resolution + journal PAYOUT_REVERSAL_RESOLVED in one transaction.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : ResolveReversalRequest

Code	Réponse
200	ResolveReversalResponse — Outcome
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

GET /admin/finances/report

Monthly finance report per currency (D58 5A) — computed from bookings, no ledger

operationId : adminGetFinanceReport · Authentication : adminCookieAuth

Permission finances.read. months = 1..24 (default 12, UTC months). Each money fact counts in ITS month: capture → capturedAt, refund → refundedAt, payout → payoutSentAt, revenue (commission + premium of COMPLETED deals) → completedAt, retention → closedAt. snapshot = today's liabilities (pending / frozen payouts, open reversals, retentions held, proposed refunds). Provider fees are not stored: reconcile with the Stripe export.

Paramètre	Dans	Requis	Type	Description
months	query	non	integer	

Code	Réponse
200	FinanceReport — Report
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
500	UnhandledError — Unhandled server error

GET /admin/finances/export

CSV export per deal (D58 5A) — journaled, FINANCE only

operationId : adminExportFinanceCsv · Authentication : adminCookieAuth

Permission finances.export. from / to ISO date-times, at most 366 days. One line per deal with a money fact in the period: frozen amounts, refund (+ refundId), payout (+ transferId), retention, dates, dispute ticket, paymentIntentId, chargeld. Formula-looking cells are neutralised. Journal FINANCE_EXPORTED (period, row count).

Paramètre	Dans	Requis	Type	Description
from	query	oui	string	
to	query	oui	string	

Code	Réponse
200	string — CSV (UTF-8 with BOM), Content-Disposition attachment, X-Row-Count
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
500	UnhandledError — Unhandled server error

POST /admin/deals/{id}/refund/propose

Propose a commercial refund (D58 3A-c) — FINANCE / SUPPORT

operationId : adminProposeManualRefund · Authentication : adminCookieAuth

Permission refunds.manual.propose. Closed deal (COMPLETED / CANCELLED) with a captured payment; amount ≤ total paid – already refunded; reason ≥ 50. No money moves. Journal REFUND_MANUAL_PROPOSED. 403 when the admin is a party.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : ManualRefundRequest

Code	Réponse
200	objet { ok, proposedAt } — Proposed
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /admin/deals/{id}/refund

Apply a commercial refund (D58 3A-c) — SUPER_ADMIN only, money first

operationId : adminApplyManualRefund · Authentication : adminCookieAuth

Permission refunds.manual.apply (SUPER_ADMIN). Same bounds as the proposal. D39: provider.refund FIRST, then ONE conditional transaction (optimistic lock on the refunded total — two admins never refund twice): refundAmountCents cumulated, refundId, manual refund fields, proposal cleared, outbox booking.refund_issued (actor ADMIN → the shipper receives the standard refund email), journal REFUND_MANUAL_APPLIED. The carrier's payout is untouched: Yamba bears the gesture. A refund issued without a database write shows up in the reconciliation.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Corps (requis) : ManualRefundRequest

Code	Réponse
200	ManualRefundResponse — Refunded
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
409	ErrorResponse — Business conflict — details.code ∈ QUOTE_DIVERGENCE / CAPACITY_EXCEEDED / FAMILY_REFUSED / TRIP_NOT_BOOKABLE / OWN_TRIP / PAYMENT_NOT_AUTHORIZED / PAYMENT_MISMATCH / PAYMENT_ALREADY_USED / NEW_ACCOUNT_CAP (D71: details.cap ∈ DECLARED_VALUE / WEIGHT / SHIPMENTS_PER_MONTH, limit, value)
500	UnhandledError — Unhandled server error

GET /admin/alerts

Threshold alerts (D59 3A / 4A) — computed on read, no state

operationId : adminListOpsAlerts · Authentication : adminCookieAuth

Permission kpi.read. Rules with versioned thresholds: failed payouts > 48h, decidable disputes undecided > 72h, retentions held > 7d, open reversals > 48h, parked outbox events, relay lag > 15 min, failed emails (24h), no trip published for 7d, acceptance rate < 30% over 7d (≥ 5 requests). The hourly cron emails support once per rule and per day (Redis dedup); this endpoint always recomputes.

Code	Réponse
200	OpsAlertsResponse — Alerts
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
500	UnhandledError — Unhandled server error

GET /admin/deals/{id}/history

Everything that happened to this deal (D59 5A) — outbox, admin journal, notifications, emails

operationId : adminGetDealHistory · Authentication : adminCookieAuth

Permission deals.history.read (MEDIATOR, SUPPORT, FINANCE). Read-only merge of the outbox (with relay state: published / pending / parked), the admin journal, in-app notifications and email deliveries, sorted by time. Outbox payloads are reduced to a whitelist — never a delivery code, a photo or an address. Reading is journaled (DEAL_HISTORY_VIEWED).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Deal (booking) identifier

Code	Réponse
200	DealHistoryResponse — History

Code	Réponse
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
403	ErrorResponse — Authenticated caller is not a party to this deal, or does not own the trip (ForbiddenError)
404	ErrorResponse — Deal or trip not found (NotFoundError)
500	UnhandledError — Unhandled server error

deal-service › me

Authenticated user's own resources

GET /me/wallet

Finances — carrier payouts and shipper payments, totals computed server-side (A83)

operationId : getMyWallet · Authentication : cookieAuth, bearerAuth

Both roles of the caller in one payload. Carrier: UPCOMING (delivered, verification running) · PENDING · BLOCKED (Stripe account not ready) · FROZEN (dispute) · SENT · HELD (late cancellation after departure). Shipper: AUTHORIZED · HELD · RELEASED · RELEASED_NO_CHARGE · REFUNDED · PARTIALLY_REFUNDED. Amounts are integer cents; the frontend never recomputes.

Code	Réponse
200	WalletResponse — Wallet for both roles
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
500	UnhandledError — Unhandled server error

GET /me/deals

My received deals (carrier view, all my trips)

operationId : listMyDeals · Authentication : cookieAuth, bearerAuth

All deals where the caller is the carrier, across all trips, newest first. CarrierBookingView (earnings only, no delivery code). One read for the trips list, the dashboard inbox and the sidebar badge.

Paramètre	Dans	Requis	Type	Description
status	query	non	BookingStatus	Filter by booking status (exact match)

Code	Réponse
200	MyDealsResponse — My received deals (CarrierBookingView[])
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
500	UnhandledError — Unhandled server error

GET /me/bookings

My shipments (shipper view)

operationId : listMyBookings · Authentication : cookieAuth, bearerAuth

All deals where the caller is the shipper, newest first ('Mes envois').

Paramètre	Dans	Requis	Type	Description
status	query	non	BookingStatus	Filter by booking status (exact match)

Code	Réponse
200	MyBookingsResponse — My shipments (ShipperBookingView[])
400	ErrorResponse — Malformed request: invalid ObjectId or unknown status value (ValidationError)
401	UnauthorizedResponse — Token missing, invalid, expired, or account not found (isAuthenticated middleware)
500	UnhandledError — Unhandled server error

deal-service › schémas utilisés (114)

AcceptDealRequest

Champ	Type	Requis	Description
charterAccepted	boolean	oui	Carrier charter (verification, forbidden items, punctuality) — must be true (RGP-03)

AdminDealMoneyFile

Champ	Type	Requis	Description
id	ObjectId	oui	

Champ	Type	Requis	Description
status	string	oui	
disputeTicket	string null	oui	
corridor	object	oui	
shipper	object	oui	
carrier	object	oui	
pricing	object	oui	
payment	object	oui	
payout	object	oui	
retention	object null	oui	
dates	object	oui	
timeline	array	oui	
adminActions	array	oui	
manualRefund	object	oui	
allowedActions	object	oui	

AdminDisputeFile — Full mediation file for an admin — never the delivery code (D43)

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
kind	ArbitrationKind	oui	
status	string	oui	
timeline	object	oui	
corridor	object	oui	
parcel	object	oui	
recipient	object	oui	
money	object	oui	
shipper	object	oui	
carrier	object	oui	
pickup	object null	oui	
trackingEvents	array	oui	
deliveryPhotoUrls	array	oui	
dispute	object null	oui	
retentionDecision	object null	oui	
canDecide	boolean	oui	A decision is possible now (responded, or 72h elapsed, or RETENTION) and none was taken yet
decidableAt	string null	oui	
proposedAmounts	object	oui	Server-computed amounts shown in the admin recap (D55 2A/3A)

AdminResolutionResponse

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
kind	ArbitrationKind	oui	
finalStatus	enum : COMPLETED, CANCELLED	oui	
outcome	string	oui	
refundCents	integer	oui	
carrierPayoutCents	integer	oui	
payoutStatus	string null	oui	SENT / FAILED (retried by the cron) / null when nothing is owed to the carrier
resolvedAt	string (date-time)	oui	

AdminResolveDisputeRequest — Irreversible. Refund first, then the carrier transfer through the payout executor (D49).

Champ	Type	Requis	Description
outcome	DisputeResolutionOutcome	oui	
refundCents	integer	non	PARTIAL_REFUND only: $1 \leq \text{amount} \leq \text{totalShipperCents} - 1$
reason	string	oui	

AdminResolveRetentionRequest — Amounts are computed server-side (pro-rata A79 or the retained amount)

Champ	Type	Requis	Description
outcome	RetentionArbitrationOutcome	oui	
reason	string	oui	

ArbitrationKind — enum : DISPUTE , RETENTION

ArbitrationQueueItem

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
kind	ArbitrationKind	oui	
ticketNumber	string null	oui	
category	DisputeCategory null	oui	
openedAt	string (date-time)	oui	
originCity	string	oui	
destinationCity	string	oui	
amountCents	integer	oui	Total paid by the shipper (DISPUTE) or retained amount (RETENTION)
currencyCode	string	oui	
shipperFirstName	string	oui	
carrierFirstName	string	oui	
carrierResponded	boolean	oui	DISPUTE: the carrier gave their side
decidableAt	string (date-time)	oui	When a decision becomes possible: now if responded / RETENTION, else disputedAt + 72h

ArbitrationQueueResponse

Champ	Type	Requis	Description
items	array	oui	
counts	object	oui	

BookingActor — enum : SHIPPER , CARRIER , SYSTEM , ADMIN

BookingCounterpart — Minimal profile of the other party (explicit join — Booking has no Prisma relations)

Champ	Type	Requis	Description
id	ObjectId	oui	
firstName	string null	non	
lastInitial	string	oui	Last-name initial, '' if absent (privacy)
avatarUrl	string null	oui	
publicSlug	string null	oui	Public profile slug (/u/[slug]) — null for accounts without one (A45)

BookingParcelSnapshot

Champ	Type	Requis	Description
category	ParcelCategory	oui	
categoryFamily	string null	non	D14 risk-family mapping, frozen at creation
description	string	oui	
declaredValueCents	integer	oui	
photoUrls	array	oui	Shipper declaration photos (R2, timestamped)

BookingPickupInfo

Champ	Type	Requis	Description
confirmedAt	string (date-time)	oui	
photoUrls	array	oui	Carrier pickup photos (ImageKit URLs — D42, 1..5, server-timestamped)
notes	string null	non	
checklist	array	oui	The 5 inspection items ticked at pickup (CNF-04 attestation, frozen — B3). Empty on pre-B3 records.

BookingPlaceInput — Chosen among the trip's pickup/delivery points

Champ	Type	Requis	Description
kind	LocationKind	oui	
details	string null	non	

BookingPlaceSnapshot — Meeting point chosen at booking (frozen)

Champ	Type	Requis	Description
kind	LocationKind	oui	
details	string null	non	

BookingRatingState

Champ	Type	Requis	Description
windowEndsAt	string null	oui	completedAt + 14 days
ratedByMe	boolean	oui	
counterpartHasRated	boolean	oui	
revealedAt	string null	oui	
canRate	boolean	oui	COMPLETED, window open, not rated by me — the state machine verdict (canRate)

BookingRecipientSnapshot — Recipient contact — visible to both roles (the carrier needs it to deliver)

Champ	Type	Requis	Description
firstName	string	oui	
lastName	string	oui	
phoneE164	string	oui	
email	string null	oui	Optional at request time

BookingStatus — enum : PENDING , ACCEPTED , PICKED_UP , DELIVERED , COMPLETED , DECLINED , EXPIRED , CANCELLED , DISPUTED

BookingTrackingEvent

Champ	Type	Requis	Description
step	TrackingStep	oui	
confirmedAt	string (date-time)	oui	

BookingTransitionAction — enum : accept , decline , expire , cancel , pickup , refusePickup , deliver , confirmEarly , autoComplete , resolveDisputeKeep , resolveDisputeRefund , dispute — Actions of the booking state machine (canPerform/getAllowedActions). Returned in allowedActions to drive frontend CTAs — the frontend reflects, never decides.

BookingTripSnapshot — Trip facts frozen at booking creation — the deal stays readable even if the trip changes

Champ	Type	Requis	Description
originCity	string	oui	
originCountryCode	string null	non	
originTimezone	string null	non	
destinationCity	string	oui	
destinationCountryCode	string null	non	
destinationTimezone	string null	non	
departureAt	string (date-time)	oui	UTC — local rendering uses the two IANA timezones (D24)
transportMode	string null	non	

BookingViewerRole — enum : SHIPPER , CARRIER — Role of the authenticated viewer for a deal (drives the DTO shape)

CancelDealRequest

Champ	Type	Requis	Description
reason	string null	non	

CancellationPreview — Server-computed ANN-01 preview shown before the shipper confirms a cancellation. Informative snapshot at read time — the refund is recomputed at the actual cancel.

Champ	Type	Requis	Description
refundCents	integer	oui	Amount returned to the shipper if they cancel NOW (full total while PENDING; ANN-01 scale once ACCEPTED)
retentionCents	integer	oui	totalShipperCents - refundCents (0 while full refund applies)
retentionPct	number	oui	Retention percentage applied after the full-refund deadline (server parameter §13)
fullRefundUntil	string (date-time)	oui	departureAt - 48h — cancelling before this instant refunds 100% (ANN-01/D39)
currencyCode	string	oui	

CarrierBookingView — Deal as seen by the carrier — no delivery code, no code hash, no regeneration counter, no shipper total

Champ	Type	Requis	Description
id	ObjectId	oui	

Champ	Type	Requis	Description
tripId	ObjectId	oui	
shipperId	ObjectId	oui	
status	BookingStatus	oui	
trip	BookingTripSnapshot	oui	
pricing	CarrierPricing	oui	
parcel	BookingParcelSnapshot	oui	
recipient	BookingRecipientSnapshot	oui	
pickupPlace	BookingPlaceSnapshot null	non	
deliveryPlace	BookingPlaceSnapshot null	non	
shipper	BookingCounterpart	oui	
requestedAt	string (date-time)	oui	
expiresAt	string (date-time)	oui	requestedAt + 24h acceptance deadline
acceptedAt	string null	non	
pickedUpAt	string null	non	
deliveredAt	string null	non	
payoutDueAt	string null	non	deliveredAt + D+4 (verification window end)
completedAt	string null	non	
closedAt	string null	non	Set on DECLINED / EXPIRED / CANCELLED
closedBy	BookingActor null	non	
declineReason	string null	non	
pickupRefusalReason	string null	non	Set when the carrier refused the parcel at pickup (PickupRefusalReason — B3/A40)
disputeTicket	string null	non	
disputedAt	string null	non	
payoutStatus	PayoutStatus null	non	Carrier payout state (B4/A68) — both roles read it; the transfer id is served to nobody
payoutSentAt	string null	non	
deliveryPhotoUrls	array	oui	Optional handover photos taken by the carrier at delivery (B4-PR3/A76) — served to both parties
createdAt	string (date-time)	oui	
updatedAt	string (date-time)	oui	
deliveryAttemptsLeft	integer	oui	MAX_DELIVERY_ATTEMPTS (3) minus attempts used — the code itself is NEVER exposed here
deliveryLockedUntil	string null	non	Anti brute-force lock (15 min after 3 failed attempts, server-side)
pickup	BookingPickupInfo null	non	
trackingEvents	array	oui	
disputeCategory	DisputeCategory null	non	Why the shipper disputed (status DISPUTED) — the carrier sees the category, never the file (A68)
dispute	CarrierDisputeView null	non	C-PR2 (D55): the carrier's view of the dispute — statement state, deadline, decision. Served while DISPUTED and after the decision.
retentionDecision	RetentionDecisionView null	non	C-PR2 (D55 3A): how a held retention was arbitrated
payoutAmountCents	integer null	non	Amount paid out to the carrier: the net at COMPLETED, the ANN-01 compensation at late CANCELLED (D50/A82)
retentionDisposition	string null	non	Late cancellation only: the retention went to the carrier (compensation), back to the shipper (mediation, C-PR2) or is held for mediation (cancelled after departure, A81)
rating	BookingRatingState null	oui	null unless COMPLETED (B5)
payoutBlocker	string null	oui	Coarse cause when payoutStatus = FAILED (A75): ACCOUNT_NOT_READY → finish the Stripe onboarding (CTA) · RETRYING → provider-side error, retried automatically, nothing to do. Never the raw provider message. null otherwise.
allowedActions	array	oui	getAllowedActions(booking, 'CARRIER') — drives the frontend CTAs

CarrierDisputeStatementRequest — The carrier's side of the story — once, while the dispute is open (D55 1A)

Champ	Type	Requis	Description
statement	string	oui	
photoUrls	array	oui	

CarrierDisputeStatementResponse

Champ	Type	Requis	Description
bookingId	ObjectId	oui	

Champ	Type	Requis	Description
ticketNumber	string	oui	
respondedAt	string (date-time)	oui	

CarrierDisputeView — What the carrier sees of a dispute: ticket, category, its own statement state, the decision — never the shipper's file (A68)

Champ	Type	Requis	Description
ticketNumber	string	oui	
category	DisputeCategory	oui	
disputedAt	string (date-time)	oui	
canRespond	boolean	oui	DISPUTED, no statement yet — the front reflects, never decides
responseDeadlineAt	string (date-time)	oui	disputedAt + 72h: after that the admin may decide without the carrier's statement
respondedAt	string null	oui	
resolution	DisputeResolutionView null	oui	

CarrierPricing — Earnings-only pricing view — commission and shipper total are never exposed to the carrier

Champ	Type	Requis	Description
pricingModel	PricingModel	oui	
weightKg	number	oui	
categoryPriceCents	integer null	non	
pricePerKgCents	integer null	non	
sizeClass	string null	non	
transportCents	integer	oui	Carrier net earnings for this deal (COM-03)
currencyCode	string	oui	

CarrierWallet

Champ	Type	Requis	Description
upcomingCents	integer	oui	Σ UPCOMING
pendingCents	integer	oui	Σ PENDING + BLOCKED + FROZEN
blockedCents	integer	oui	Σ BLOCKED — drives the 'finish your Stripe account' banner
sentCents	integer	oui	Σ SENT, all time
sentThisMonthCents	integer	oui	Σ SENT with payoutSentAt in the current calendar month (UTC)
currencyCode	string	oui	
items	array	oui	Most recent first

ConfirmDealResponse — Early confirmation done — payout released (INV-3: final)

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
status	string	oui	
completedAt	string (date-time)	oui	
payoutStatus	PayoutStatus	oui	SENT if the transfer succeeded inline, FAILED if the provider refused (retried by the cron — A67)
payoutAmountCents	integer	oui	= pricing.transportCents (carrier net, D50)
currencyCode	string	oui	

ConfirmPickupRequest

Champ	Type	Requis	Description
checklist	array	oui	Must contain ALL 5 items — the server refuses a partial inspection (CNF-04)
photoUrls	array	oui	1 to 5 photo URLs already uploaded to ImageKit by the browser (D42)
notes	string null	non	

ConfirmTrackingStepRequest

Champ	Type	Requis	Description
step	TrackingStep	oui	

CreateBookingRequest

Champ	Type	Requis	Description
product	ParcelProduct	oui	

Champ	Type	Requis	Description
family	ParcelFamily	oui	
sizeClass	SizeClass null	non	Required for PARCEL
weightKg	number null	non	Declared weight (kg) — required for PARCEL; ignored for bags
protection	ProtectionTier	oui	
tripId	ObjectId	oui	
paymentIntentId	string	oui	
expectedTotalCents	integer	oui	
description	string	oui	Min 5 — same floor as the wizard (spec: recommended, low friction)
declaredValueCents	integer	oui	
photoUrls	array	oui	
recipient	object	oui	
pickupPlace	BookingPlacelInput null	non	
deliveryPlace	BookingPlacelInput null	non	
charterAccepted	boolean	oui	Shipper charter (CNF-02) — must be true
termsAccepted	boolean	oui	

CreateBookingResponse

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
status	BookingStatus	oui	
expiresAt	string (date-time)	oui	24h acceptance deadline (DEA-01)
totalShipperCents	integer	oui	
currencyCode	string	oui	

CreatePaymentIntentRequest

Champ	Type	Requis	Description
product	ParcelProduct	oui	
family	ParcelFamily	oui	
sizeClass	SizeClass null	non	Required for PARCEL
weightKg	number null	non	Declared weight (kg) — required for PARCEL; ignored for bags
protection	ProtectionTier	oui	
tripId	ObjectId	oui	
expectedTotalCents	integer	oui	Total the shipper saw (D17 check)

CreatePaymentIntentResponse

Champ	Type	Requis	Description
provider	PaymentProviderName	oui	
paymentIntentId	string	oui	
clientSecret	string null	oui	null for the FAKE provider
amountCents	integer	oui	
currencyCode	string	oui	
quote	ShipperPricing	oui	Server-side quote — what will be frozen (D17/D34)

DealHistoryEvent

Champ	Type	Requis	Description
at	string (date-time)	oui	
source	DealHistorySource	oui	
type	string	oui	eventType outbox, action admin, type de notification ou template d'email
actor	string null	oui	SHIPPER / CARRIER / SYSTEM / ADMIN, ou le nom court de l'admin
recipient	string null	oui	Notification / email : SHIPPER ou CARRIER
summary	object	oui	Sous-ensemble WHITELISTÉ du payload — jamais un code, un secret ou une photo
relay	object null	oui	Outbox seulement
status	string null	oui	Email : PENDING / SENT / FAILED · notification : lue ou non

DealHistoryResponse — Tout ce qui est arrivé à ce deal (D59 5A), lecture seule, consultation journalisée

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
events	array	oui	
counts	object	oui	
generatedAt	string (date-time)	oui	

DealHistorySource — enum : OUTBOX , ADMIN , NOTIFICATION , EMAIL

DealResponse

Champ	Type	Requis	Description
success	boolean	oui	
viewerRole	BookingViewerRole	oui	
deal	ShipperBookingView CarrierBookingView	oui	ShipperBookingView when viewerRole=SHIPPER, CarrierBookingView when viewerRole=CARRIER

DealTransitionResponse

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
status	BookingStatus	oui	
refundAmountCents	integer null	oui	Amount returned to the shipper (cents). Full total on PENDING closures (authorization released), ANN-01 amount on post-acceptance cancellation, null when nothing is returned (accept).
currencyCode	string	oui	

DeclineDealRequest

Champ	Type	Requis	Description
reason	DeclineReason null	non	Optional — one of 5 (spec É2)

DeclineReason — enum : CATEGORY_NOT_CARRIED , TOO_HEAVY , PLACES_INCOMPATIBLE , TIMING , OTHER — The 5 optional decline reasons offered to the carrier (spec É2)

DeliverDealRequest

Champ	Type	Requis	Description
code	DeliveryCode	oui	Code given by the recipient — compared with bcrypt server-side
photoUrls	array	oui	OPTIONAL handover photos (ImageKit, direct signed upload — D42/A76): the carrier's insurance on the parcel state

DeliverDealResponse

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
status	BookingStatus	oui	DELIVERED
deliveredAt	string (date-time)	oui	
payoutDueAt	string (date-time)	oui	deliveredAt + 4 days — end of the shipper verification window (B4 payout)

DeliveryCode — string : 6 decimal digits (100000-999999)

DisputeCategory — enum : NOT_DELIVERED , CONTENT_MISSING , DAMAGED , SIGNIFICANT_DELAY , RECIPIENT_ISSUE , OTHER — Dispute category (spec §3.6 — one of six)

DisputeDealRequest — Open a dispute (shipper, DELIVERED, before payoutDueAt)

Champ	Type	Requis	Description
category	DisputeCategory	oui	
description	string	oui	
pledgeAccepted	boolean	oui	Honour pledge — must be true (anti-abuse, spec §3.6)
photoUrls	array	oui	ImageKit URLs (direct signed upload — D42), optional but recommended
desiredOutcome	DisputeDesiredOutcome	non	

DisputeDealResponse — Dispute opened — payout frozen, ticket issued (INV-4/INV-5)

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
status	string	oui	
ticketNumber	string	oui	
disputedAt	string (date-time)	oui	

DisputeDesiredOutcome — enum : FULL_REFUND , PARTIAL_REFUND , CONTACT_CARRIER , YAMBA_DECIDES — What the shipper asks for (optional, informs mediation)

DisputeResolutionOutcome — enum : REJECTED , PARTIAL_REFUND , FULL_REFUND — REJECTED: carrier paid in full · PARTIAL_REFUND: free amount · FULL_REFUND: shipper refunded everything (D54 3A)

DisputeResolutionView

Champ	Type	Requis	Description
outcome	DisputeResolutionOutcome	oui	
refundCents	integer	oui	Refunded to the shipper
carrierPayoutCents	integer	oui	Paid out to the carrier (net – refund, floor 0)
reason	string	oui	Admin's written reason — read by BOTH parties
resolvedAt	string (date-time)	oui	

ErrorResponse — Enveloppe d'erreur standard (error-middleware, toute AppError)

Champ	Type	Requis	Description
status	string	oui	
message	string	oui	
errors	object	non	Erreurs par champ (formulaires) — exposé quand details.errors est présent
details		non	Contexte structuré 'safe' (ex: type=otp) — toujours exposé ; le reste hors prod uniquement

FinanceQueueItem

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
kind	FinanceQueueKind	oui	
status	string	oui	
corridor	object	oui	
shipper	object	oui	
carrier	object	oui	
amountCents	integer	oui	Montant concerné : versement (FAILED / REVERSED) ou retenue (HELD)
currencyCode	string	oui	
payoutStatus	string null	oui	
payoutAttempts	integer	oui	
payoutFailureKind	PayoutFailureKind null	oui	
payoutFailureDetail	string null	oui	Message fournisseur brut — admin seulement, jamais servi au Voyageur (A75)
lastAttemptAt	string null	oui	
nextRetryAt	string null	oui	
disputeTicket	string null	oui	
since	string (date-time)	oui	Depuis quand l'exception existe (fin du deal ou dernière écriture)

FinanceQueueKind — enum : FAILED , REVERSED , HELD , PROPOSED_REFUNDS — FAILED = versements en échec · REVERSED = transferts renversés non clos · HELD = retenues conservées à arbitrer · PROPOSED_REFUNDS = remboursements manuels proposés, à décider (C-PR5b)

FinanceQueueResponse

Champ	Type	Requis	Description
kind	FinanceQueueKind	oui	
items	array	oui	
generatedAt	string (date-time)	oui	

FinanceReport — Aucun grand livre (D58 1A) : agrégats des champs posés par les transitions. Les frais Stripe ne sont pas en base — rapprocher avec l'export Stripe.

Champ	Type	Requis	Description
from	string (date-time)	oui	
to	string (date-time)	oui	
generatedAt	string (date-time)	oui	
months	array	oui	
snapshot	array	oui	

FinanceReportMonth

Champ	Type	Requis	Description
month	string	oui	YYYY-MM (UTC)
currencyCode	string	oui	
capturedCents	integer	oui	Encaissé : total payé des deals capturés ce mois
capturedCount	integer	oui	
refundedCents	integer	oui	Remboursé aux Expéditeurs ce mois (toutes causes, manuel compris)
refundCount	integer	oui	
paidOutCents	integer	oui	Versé aux Voyageurs ce mois (versements SENT, renversés inclus car partis)
payoutCount	integer	oui	
revenueCents	integer	oui	Revenu reconnu : commission + prime des deals terminés ce mois (COMPLETED)
completedCount	integer	oui	
retentionCents	integer	oui	Retenues nées ce mois (annulations tardives), quel que soit leur sort
cancelledCount	integer	oui	

FinanceSnapshot

Champ	Type	Requis	Description
currencyCode	string	oui	
pendingPayoutCents	integer	oui	Dû aux Voyageurs : PENDING + FAILED (passif)
frozenPayoutCents	integer	oui	Gelé par un litige (FROZEN)
reversedOpenCents	integer	oui	Transferts renversés non clos (argent revenu, à décider)
heldRetentionCents	integer	oui	Retenues conservées à arbitrer (passif, pas un revenu)
proposedRefundCents	integer	oui	Remboursements manuels proposés, non appliqués

LocationKind — enum : AIRPORT , TRAIN_STATION , CITY_AREA

ManualRefundRequest — Montant en centimes ≤ total payé – déjà remboursé ; motif ≥ 50 caractères (geste commercial, jamais une décision de litige)

Champ	Type	Requis	Description
amountCents	integer	oui	
reason	string	oui	

ManualRefundResponse

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
refundedCents	integer	oui	
totalRefundedCents	integer	oui	
refundId	string null	oui	
currencyCode	string	oui	

MoneyTimelineEvent

Champ	Type	Requis	Description
at	string (date-time)	oui	
kind	MoneyTimelineKind	oui	
amountCents	integer null	oui	
detail	string null	oui	

MoneyTimelineKind — enum : AUTHORIZED , CAPTURED , REFUNDED , DISPUTED , COMPLETED , CANCELLED , PAYOUT_SENT , PAYOUT_FAILED , PAYOUT_REVERSED , REVERSAL_RESOLVED , RETENTION , RETENTION_DECIDED

MyBookingsResponse

Champ	Type	Requis	Description
success	boolean	oui	
bookings	array	oui	
count	integer	oui	

MyDealsResponse

Champ	Type	Requis	Description
success	boolean	oui	
deals	array	oui	

Champ	Type	Requis	Description
count	integer	oui	

MyRating

Champ	Type	Requis	Description
rating	integer	oui	
criteria	object null	oui	
comment	string null	oui	
submittedAt	string (date-time)	oui	

ObjectId — string : Identifiant MongoDB (ObjectId sérialisé en hexadécimal)

OpsAlert

Champ	Type	Requis	Description
rule	OpsAlertRule	oui	
severity	enum : warning, critical	oui	
title	string	oui	
detail	string	oui	
count	integer null	oui	Éléments concernés (null pour une alerte de liquidité)
href	string	oui	Chemin admin où agir

OpsAlertRule — enum : PAYOUT_FAILED_48H, DISPUTE_UNDECIDED_72H, RETENTION_HELD_7D, REVERSAL_OPEN_48H, OUTBOX_PARKED, OUTBOX_LAGGING_15MIN, EMAILS_FAILED_24H, NO_TRIP_PUBLISHED_7D, ACCEPTANCE_RATE_LOW_7D

OpsAlertsResponse — Sans état (D59 4A) : recalculées à chaque lecture ; le cron horaire n'envoie un email qu'à la première apparition du jour

Champ	Type	Requis	Description
alerts	array	oui	
evaluatedAt	string (date-time)	oui	
thresholds	object	oui	

ParcelCategory — enum : CLOTHES, SHOES, FASHION_ACCESSORIES, OTHER_ACCESSORIES, BOOKS, DOCUMENTS, SMALL_TOYS, PHONE, COMPUTER, OTHER_ELECTRONICS, CHECKED_BAG_23KG, CABIN_BAG_12KG — Enum actuel (pré-D14). La migration vers les 8 familles de risque CAT-02 fera l'objet d'une PR dédiée (mapping conservé).

ParcelFamily — enum : DOCUMENTS_PAPERS, CLOTHES_TEXTILE, FOOD_DRY_SEALED, ELECTRONICS_DEVICES, COSMETICS_CARE, PARTS_TOOLS, TOYS_CHILDCARE, MISC_ACCESSORIES — D14/CAT-02 — the 8 risk families (conformity/risk/protection, never price)

ParcelProduct — enum : PARCEL, CHECKED_BAG_23KG, CABIN_BAG_12KG — PRC-04 — parcel priced per kg, or a whole bag at a flat rate

PaymentProviderName — enum : STRIPE, FAKE — FAKE only outside production (D11/D30)

PaymentReconciliation — Lecture seule chez le fournisseur ; la base n'est jamais modifiée par un rapprochement (D58 4A)

Champ	Type	Requis	Description
provider	string	oui	
checkedAt	string (date-time)	oui	
live	object null	oui	
divergences	array	oui	

PayoutFailureKind — enum : ACCOUNT_NOT_READY, PROVIDER_ERROR, REVERSED

PayoutStatus — enum : PENDING, SENT, FAILED, FROZEN, REVERSED — Carrier payout state (B4/D49). PENDING = deal COMPLETED, transfer not yet executed · SENT = transfer executed (payoutSentAt) · FAILED = transfer refused (retried by the payout cron) · FROZEN = dispute open (INV-5) · REVERSED = Stripe reversed the transfer (never re-sent automatically — admin, A87). null before COMPLETED / DISPUTED.

PickupChecklistItem — enum : CONTENT_MATCHES, WEIGHT_OK, NO_FORBIDDEN, PACKAGING_OK, ITEMS_IDENTIFIED — One of the 5 mandatory inspection items (spec É4a, CNF-04)

PickupRefusalReason — enum : CONTENT_MISMATCH, SUSPICIOUS_CONTENT, OVERWEIGHT, BAD_PACKAGING, OTHER — The 5 optional refusal reasons offered to the carrier at pickup (spec É4a)

PricingModel — enum : PER_CATEGORY, PER_KG — Pricing engine at booking creation time. PER_CATEGORY = current flat price per category; PER_KG = D13 target (price/kg x S/M/L size class). Snapshots are never migrated.

ProtectionTier — enum : BASIC, EXTENDED_500 — GAR-02 — Yamba Guarantee tier (D22)

PublicTrackingResponse — D69 — recipient tracking page: minimal content, no address, no phone, no delivery code

Champ	Type	Requis	Description
milestone	TrackingMilestone	oui	
steps	array	oui	
recipientFirstName	string	oui	
shipperFirstName	string	oui	
carrier	object	oui	
corridor	object	oui	
departureAt	string null	oui	
arrivalAt	string null	oui	

RatingContextResponse — Everything the rating screen needs — the frontend reflects, never decides

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
viewerRole	BookingViewerRole	oui	
ratedRole	BookingViewerRole	oui	The role being rated (opposite of viewerRole) — drives the criteria set
person	object	oui	
corridor	object	oui	
completedAt	string null	oui	
windowEndsAt	string null	oui	completedAt + 14 days: rating closes and reviews are revealed
canRate	boolean	oui	Server verdict (state machine): COMPLETED, within the window, not rated yet by this role
cannotRateReason	string null	oui	
myRating	MyRating null	oui	
counterpartHasRated	boolean	oui	
revealedAt	string null	oui	Double-blind lifted: both rated, or window elapsed
counterpartRating	MyRating null	oui	

RatingCriterion — enum : PUNCTUALITY, COMMUNICATION, PARCEL_CARE, DECLARATION_CLARITY, RESPONSIVENESS — Optional thumbs criteria — carrier: punctuality, communication, parcel care · shipper: declaration clarity, responsiveness, punctuality

RatingVote — enum : UP, DOWN

ReconciliationDivergenceCode — enum : CAPTURE_NOT_RECORDED, CAPTURE_RECORDED_NOT_LIVE, REFUND_NOT_RECORDED, REFUND_RECORDED_NOT_LIVE, TRANSFER_MISSING, TRANSFER_AMOUNT_MISMATCH, TRANSFER_REVERSED_NOT_MARKED, TRANSFER_MARKED_REVERSED_BUT_LIVE_OK, INTENT_NOT_FOUND

RefusePickupRequest

Champ	Type	Requis	Description
reason	PickupRefusalReason null	non	Optional — one of 5 (spec É4a)

RegenerateCodeResponse

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
deliveryCode	DeliveryCode	oui	The NEW code — the only write-path surface where it appears (shipper only). The previous code is invalid.
codeRegenerationsLeft	integer	oui	

ResolveReversalRequest

Champ	Type	Requis	Description
outcome	enum : RESENT, WRITTEN_OFF	oui	RESENT = nouveau transfert par l'exécuteur unique · WRITTEN_OFF = manque à gagner assumé, rien n'est renvoyé
reason	string	oui	

ResolveReversalResponse

Champ	Type	Requis	Description
outcome	enum : RESENT, WRITTEN_OFF	oui	
payoutStatus	string null	oui	
reason	string null	oui	

RetentionArbitrationOutcome — enum : COMPENSATE_CARRIER, RESTITUTE_SHIPPER — Late cancellation after departure (A81): retention goes to the carrier (pro-rata A79) or back to the shipper

RetentionDecisionView

Champ	Type	Requis	Description
outcome	RetentionArbitrationOutcome	oui	
reason	string	oui	
decidedAt	string (date-time)	oui	

RetryPayoutResponse

Champ	Type	Requis	Description
payoutStatus	enum : SENT, FAILED	oui	
reason	string null	oui	
transferId	string null	oui	

ShipperBookingView — Deal as seen by the shipper ('Mes envois')

Champ	Type	Requis	Description
id	ObjectId	oui	
tripId	ObjectId	oui	
carrierId	ObjectId	oui	
status	BookingStatus	oui	
trip	BookingTripSnapshot	oui	
pricing	ShipperPricing	oui	
parcel	BookingParcelSnapshot	oui	
recipient	BookingRecipientSnapshot	oui	
pickupPlace	BookingPlaceSnapshot null	non	
deliveryPlace	BookingPlaceSnapshot null	non	
carrier	BookingCounterpart	oui	
requestedAt	string (date-time)	oui	
expiresAt	string (date-time)	oui	requestedAt + 24h acceptance deadline
acceptedAt	string null	non	
pickedUpAt	string null	non	
deliveredAt	string null	non	
payoutDueAt	string null	non	deliveredAt + D+4 (verification window end)
completedAt	string null	non	
closedAt	string null	non	Set on DECLINED / EXPIRED / CANCELLED
closedBy	BookingActor null	non	
declineReason	string null	non	
pickupRefusalReason	string null	non	Set when the carrier refused the parcel at pickup (PickupRefusalReason — B3/A40)
disputeTicket	string null	non	
disputedAt	string null	non	
payoutStatus	PayoutStatus null	non	Carrier payout state (B4/A68) — both roles read it; the transfer id is served to nobody
payoutSentAt	string null	non	
deliveryPhotoUrls	array	oui	Optional handover photos taken by the carrier at delivery (B4-PR3/A76) — served to both parties
createdAt	string (date-time)	oui	
updatedAt	string (date-time)	oui	
deliveryCode	string null	oui	6-digit delivery code, revealed to the shipper ONLY while the parcel is in transit (PICKED_UP) and only on GET /deals/:id (never in lists — D43, AES-256-GCM at rest). null otherwise. Never present in any carrier payload.
codeRegenerationsLeft	integer	oui	MAX_CODE_REGENERATIONS (5) minus regenerations used — server is the only judge
pickup	BookingPickupInfo null	non	
trackingEvents	array	oui	
allowedActions	array	oui	getAllowedActions(booking, 'SHIPPER') — drives the frontend CTAs
cancellationPreview	CancellationPreview null	oui	Non-null exactly when 'cancel' is in allowedActions (PENDING or ACCEPTED)
capturedAt	string null	non	When the shipper's card was actually charged (at acceptance, D39)
refundedAt	string null	non	
refundAmountCents	integer null	non	Amount returned (ANN-01 scale, plus a mediation refund or restitution); null when nothing was refunded
retentionDecision	RetentionDecisionView null	non	C-PR2 (D55 3A): how a held retention was arbitrated
retentionCents	integer null	non	Late cancellation: amount retained (ANN-01), null otherwise

Champ	Type	Requis	Description
dispute	ShipperDisputeView null	non	The dispute file — shipper only (A68); served while DISPUTED and after the decision (resolution), absent otherwise
rating	BookingRatingState null	oui	null unless COMPLETED (B5)
disputeOpensAt	string null	oui	PICKED_UP only: when the 'not delivered' dispute becomes possible (trip departure + 48h — B4/D51). Served, never computed by the front (A72). null otherwise.

ShipperDisputeView — The dispute file as filed by the shipper (never served to the carrier)

Champ	Type	Requis	Description
ticketNumber	string	oui	
category	DisputeCategory	oui	
description	string	oui	
desiredOutcome	DisputeDesiredOutcome null	oui	
photoUrls	array	oui	
createdAt	string (date-time)	oui	
carrierRespondedAt	string null	oui	The carrier gave their side (content never served to the shipper — D55 5A)
resolution	DisputeResolutionView null	oui	

ShipperPricing — Full pricing snapshot — shipper view only (all amounts in integer cents, A2)

Champ	Type	Requis	Description
pricingModel	PricingModel	oui	
weightKg	number	oui	
categoryPriceCents	integer null	non	PER_CATEGORY engine
pricePerKgCents	integer null	non	PER_KG engine (D13)
sizeClass	string null	non	
transportCents	integer	oui	Carrier net (COM-03)
commissionPct	number	oui	Frozen at creation (COM-04)
commissionCents	integer	oui	Floor already applied (D16)
protectionProvider	string null	non	
protectionTier	string null	non	
premiumCents	integer	oui	Protection premium, separate flow (D22)
totalShipperCents	integer	oui	Total charged to the shipper
currencyCode	string	oui	
product	string null	non	PARCEL / CHECKED_BAG_23KG / CABIN_BAG_12KG
billableWeightKg	number null	non	max(weight, 0.5) — D32
sizeCoef	number null	non	
familySurchargePct	number null	non	
rawTransportCents	integer null	non	Before the 8 € floor
minimumApplied	boolean null	non	
serviceCents	integer null	non	commission + premium (COM-03)

ShipperWallet

Champ	Type	Requis	Description
heldCents	integer	oui	Σ HELD — captured, not yet settled
spentCents	integer	oui	Σ RELEASED totals + retentions of PARTIALLY_REFUNDED
refundedCents	integer	oui	Σ refunds actually returned (REFUNDED + PARTIALLY_REFUNDED)
currencyCode	string	oui	
items	array	oui	Most recent first

SizeClass — enum : S, M, L — PRC-03 — visual size class (coef 1 / 1.1 / 1.25)

SubmitRatingRequest

Champ	Type	Requis	Description
rating	integer	oui	1 (disappointing) ... 5 (excellent) — the only required field
criteria	object	non	Optional thumbs, only the criteria of the rated role are kept
comment	string	non	Public, attributed, immutable — 280 chars max

SubmitRatingResponse

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
submittedAt	string (date-time)	oui	
revealed	boolean	oui	true when the counterpart had already rated → both reviews revealed now
revealedAt	string null	oui	

TrackingLinkResponse

Champ	Type	Requis	Description
token	string	oui	
path	string	oui	
recipientFirstName	string	oui	
recipientPhoneE164	string null	oui	

TrackingMilestone — enum : ACCEPTED , PICKED_UP , IN_TRANSIT , ARRIVED , DELIVERED , CLOSED

TrackingStep — enum : AT_AIRPORT , FLIGHT_DEPARTED , FLIGHT_ARRIVED — Optional tracking milestones inside PICKED_UP, strictly sequential

TrackingStepResponse

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
step	TrackingStep	oui	
confirmedAt	string (date-time)	oui	
trackingEvents	array	oui	Full sequence after this confirmation

TripDealsResponse

Champ	Type	Requis	Description
success	boolean	oui	
deals	array	oui	
count	integer	oui	

UnauthorizedResponse — 401 du middleware isAuthenticated (token absent, invalide, expiré, ou compte introuvable) — hors error-middleware

Champ	Type	Requis	Description
message	string	oui	

UnhandledError — 500 non géré (exception hors AppError) — champ error , pas message

Champ	Type	Requis	Description
status	string	oui	
error	string	oui	

WalletPaymentItem

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
tripId	ObjectId	oui	
bookingStatus	BookingStatus	oui	
corridor	object	oui	
counterpartFirstName	string null	oui	Carrier first name (null if account deleted)
state	WalletPaymentState	oui	
amountCents	integer	oui	Total charged (or authorized) to the shipper
refundAmountCents	integer null	oui	REFUNDED / PARTIALLY_REFUNDED: amount returned
retentionCents	integer null	oui	PARTIALLY_REFUNDED: amount kept (ANN-01)
currencyCode	string	oui	
date	string null	oui	HELD (delivered): payoutDueAt · RELEASED: completedAt · REFUNDED: refundedAt · else: requestedAt

WalletPaymentState — enum : AUTHORIZED , HELD , RELEASED , RELEASED_NO_CHARGE , REFUNDED , PARTIALLY_REFUNDED — AUTHORIZED = hold placed, nothing debited (PENDING request) · HELD = captured, kept by Yamba until completion (ACCEPTED/PICKED_UP/DELIVERED/DISPUTED) · RELEASED = completed, carrier paid · RELEASED_NO_CHARGE = declined / expired / cancelled before capture, the hold simply vanished · REFUNDED = captured then refunded in full · PARTIALLY_REFUNDED = late cancellation, ANN-01 retention kept

WalletPayoutItem

Champ	Type	Requis	Description
bookingId	ObjectId	oui	
tripId	ObjectId	oui	
bookingStatus	BookingStatus	oui	
corridor	object	oui	
counterpartFirstName	string null	oui	Shipper first name (null if account deleted)
kind	enum : DELIVERY, LATE_CANCELLATION	oui	Net for a delivery, or ANN-01 compensation
state	WalletPayoutState	oui	
amountCents	integer null	oui	null for HELD (nothing decided yet)
currencyCode	string	oui	
date	string null	oui	UPCOMING: payoutDueAt · SENT: payoutSentAt · else: last update

WalletPayoutState — enum : UPCOMING , PENDING , BLOCKED , FROZEN , SENT , HELD , REVERSED — UPCOMING = delivered, shipper verification running (payoutDueAt) · PENDING = being sent / retried · BLOCKED = carrier Stripe account not ready (CTA onboarding) · FROZEN = dispute open · SENT = transfer executed (payoutSentAt) · HELD = late cancellation after departure, retention held for mediation (no amount) · REVERSED = Stripe reversed the transfer, under review (A87)

WalletResponse — Finances page — both roles, totals computed server-side (A83)

Champ	Type	Requis	Description
success	boolean	oui	
carrier	CarrierWallet	oui	
shipper	ShipperWallet	oui	
generatedAt	string (date-time)	oui	

message-service — port 6005

Messagerie, rendez-vous, numéro, signalement de message. Document vivant : <http://localhost:6005/docs> (Scalar) et [apps/message-service/openapi.json](http://localhost:8080/api). Via le gateway : <http://localhost:8080/api> (préfixes ci-dessous).

message-service › messages

Conversation, meetings and phone reveal (auth required, parties only)

GET /messages/conversations

My conversations, most active first

operationId : listConversations · Authentication : cookieAuth, bearerAuth

Unread count per conversation, next meeting (accepted one first, otherwise the latest proposal) and write access.

Code	Réponse
200	ConversationListResponse — Conversations
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
500	UnhandledError — Unhandled server error

GET /messages/conversations/by-deal/{bookingId}

The thread of a deal, created on first access (D61 2A)

operationId : getConversationByDeal · Authentication : cookieAuth, bearerAuth

The conversation exists from ACCEPTED onwards. Before that: 403 — the request already carries a message.

Paramètre	Dans	Requis	Type	Description
bookingId	path	oui	ObjectId	

Code	Réponse
200	ConversationThreadResponse — Thread
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Authenticated but not a party to this deal, or the deal has no conversation yet (ForbiddenError)
404	ErrorResponse — Conversation, deal or meeting not found (NotFoundError)
500	UnhandledError — Unhandled server error

GET /messages/conversations/{id}

The thread, oldest to newest, paginated backwards

operationId : getConversationThread · Authentication : cookieAuth, bearerAuth

cursor loads OLDER messages. Includes meetings and the phone state (revealed, opensAt).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Conversation id
cursor	query	non	ObjectId	Oldest message already loaded

Code	Réponse
200	ConversationThreadResponse — Thread
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Authenticated but not a party to this deal, or the deal has no conversation yet (ForbiddenError)
404	ErrorResponse — Conversation, deal or meeting not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /messages/conversations/{id}/messages

Post a message (D61 3A / 4A / 5A)

operationId : postMessage · Authentication : cookieAuth, bearerAuth

Text up to 2000 chars plus up to 5 photos. REFUSED (400 DELIVERY_CODE_IN_MESSAGE) when a six-digit group matches the deal's delivery code hash (D43). Contact details (email, phone) are detected and flagged on the message, never blocked. Read-only while a dispute is open and 14 days after the deal ends. One transaction: message + conversation timestamp + outbox event (D2).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Conversation id

Corps (requis) : PostMessageRequest

Code	Réponse
201	Message — Posted
400	ErrorResponse — Invalid request: empty message, bad slot, delivery code in the body (ValidationError)
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Authenticated but not a party to this deal, or the deal has no conversation yet (ForbiddenError)
404	ErrorResponse — Conversation, deal or meeting not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /messages/conversations/{id}/read

Mark the thread as read

operationId : markConversationRead · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Conversation id

Code	Réponse
200	objet { readAt } — Read
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Authenticated but not a party to this deal, or the deal has no conversation yet (ForbiddenError)
404	ErrorResponse — Conversation, deal or meeting not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /messages/conversations/{id}/meetups

Propose a meeting (D61 1A) — the meeting is an object, not a conversation

operationId : proposeMeetup · Authentication : cookieAuth, bearerAuth

Place and slot: at least 30 minutes ahead, at most 90 days, window up to 12 hours. A new proposal of the same kind replaces the open one (counter-proposal).

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Conversation id

Corps (requis) : ProposeMeetupRequest

Code	Réponse
201	Meetup — Proposed
400	ErrorResponse — Invalid request: empty message, bad slot, delivery code in the body (ValidationError)
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Authenticated but not a party to this deal, or the deal has no conversation yet (ForbiddenError)
404	ErrorResponse — Conversation, deal or meeting not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /messages/conversations/{id}/meetups/{meetupId}/accept

Accept a meeting — the other party only

operationId : acceptMeetup · Authentication : cookieAuth, bearerAuth

Optimistic guard on PROPOSED: a concurrent change answers 400 rather than overwriting.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Conversation id
meetupId	path	oui	ObjectId	

Code	Réponse
200	Meetup — Accepted
400	ErrorResponse — Invalid request: empty message, bad slot, delivery code in the body (ValidationError)
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Authenticated but not a party to this deal, or the deal has no conversation yet (ForbiddenError)
404	ErrorResponse — Conversation, deal or meeting not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /messages/conversations/{id}/phone

Reveal the counterpart's phone number (D61 4A)

operationId : revealCounterpartPhone · Authentication : cookieAuth, bearerAuth

Opens at most 2 hours before the accepted PICKUP meeting, otherwise before the trip departure. Recorded once per reader (PhoneReveal) and written in the thread as a system message. Rather than letting both parties trade numbers in the thread, the platform opens it late and keeps the trace.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Conversation id

Code	Réponse
200	RevealPhoneResponse — Revealed
400	ErrorResponse — Invalid request: empty message, bad slot, delivery code in the body (ValidationError)
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Authenticated but not a party to this deal, or the deal has no conversation yet (ForbiddenError)
404	ErrorResponse — Conversation, deal or meeting not found (NotFoundError)
500	UnhandledError — Unhandled server error

POST /messages/conversations/{id}/messages/{messageId}/report

Report a message from the counterpart (F-PR3, D61 7A)

operationId : reportMessage · Authentication : cookieAuth, bearerAuth

Only a TEXT message written by the OTHER party can be reported, once per reporter. A moderation record, not a thread transition: no outbox event; support sees it in its queue and on the admin home. 409 when already reported by this user.

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Conversation id
messageId	path	oui	ObjectId	

Corps (requis) : ReportMessageRequest

Code	Réponse
201	ReportMessageResponse — Report recorded
400	ErrorResponse — Invalid request: empty message, bad slot, delivery code in the body (ValidationError)
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Authenticated but not a party to this deal, or the deal has no conversation yet (ForbiddenError)

Code	Réponse
404	ErrorResponse — Conversation, deal or meeting not found (NotFoundError)
409	ErrorResponse — Already reported by this user (ConflictError)
500	UnhandledError — Unhandled server error

GET /messages/quick-replies

Quick replies in the reader's language (D61 2A)

operationId : listQuickReplies · Authentication : cookieAuth, bearerAuth

Same keys in every locale; the client sends the text as an ordinary message.

Code	Réponse
200	QuickRepliesResponse — Quick replies
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
500	UnhandledError — Unhandled server error

message-service › admin

ADMIN session only (F-PR3, D61 7A): read a thread from a file (journaled), review reported messages

GET /admin/conversations/by-deal/{bookingId}

Read a deal's whole conversation (conversations.read) — journaled CONVERSATION_VIEWED

operationId : adminGetConversationByDeal · Authentication : cookieAuth, bearerAuth

Both parties' names, every message with its reports, meetings and phone-reveal traces (who saw the number, when — never the number). 404 when the deal has no conversation.

Paramètre	Dans	Requis	Type	Description
bookingId	path	oui	ObjectId	

Code	Réponse
200	AdminConversationResponse — Conversation
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Admin profile without this permission
404	ErrorResponse — Conversation, deal or meeting not found (NotFoundError)
500	UnhandledError — Unhandled server error

GET /admin/conversations/reports

Reported messages queue (reports.review)

operationId : adminListMessageReports · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
status	query	non	MessageReportStatus	OPEN by default

Code	Réponse
200	AdminMessageReportsResponse — Reports
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Admin profile without this permission
500	UnhandledError — Unhandled server error

PATCH /admin/conversations/reports/{id}

Review a reported message (reports.review) — decision + journal in ONE transaction

operationId : adminReviewMessageReport · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Report id

Corps (requis) : ReviewMessageReportRequest

Code	Réponse
200	objet { id, status } — Reviewed
400	ErrorResponse — Invalid request: empty message, bad slot, delivery code in the body (ValidationError)
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)

Code	Réponse
403	ErrorResponse — Admin profile without this permission
404	ErrorResponse — Conversation, deal or meeting not found (NotFoundError)
409	ErrorResponse — Already reviewed (ConflictError)
500	UnhandledError — Unhandled server error

message-service › schémas utilisés (28)

AdminConversationResponse

Champ	Type	Requis	Description
conversationId	ObjectId	oui	
bookingId	ObjectId	oui	
bookingStatus	string	oui	
corridor	object	oui	
shipper	object	oui	
carrier	object	oui	
messages	array	oui	Du plus ancien au plus récent, sans pagination : un dossier se lit en entier
meetups	array	oui	
phoneReveals	array	oui	Qui a vu le numéro de qui, quand — jamais le numéro
lastMessageAt	string null	oui	

AdminMessage

Champ	Type	Requis	Description
id	ObjectId	oui	
kind	MessageKind	oui	
authorRole	MessageAuthorRole	oui	
authorId	ObjectId null	oui	
body	string	oui	Texte de l'auteur, ou libellé de repli d'un message système
photoUrls	array	oui	
systemKey	string null	oui	Clé i18n d'un message SYSTEM / MEETUP — le client affiche sa propre copie
systemData	object null	oui	
flaggedContact	boolean	oui	D61 5A — coordonnées détectées : averti, jamais bloqué
createdAt	string (date-time)	oui	
reports	array	oui	

AdminMessageReportItem

Champ	Type	Requis	Description
id	ObjectId	oui	
status	MessageReportStatus	oui	
reason	MessageReportReason	oui	
details	string null	oui	
createdAt	string (date-time)	oui	
reporter	object	oui	
author	object	oui	
message	object	oui	
conversationId	ObjectId	oui	
bookingId	ObjectId	oui	
corridor	object	oui	

AdminMessageReportsResponse

Champ	Type	Requis	Description
items	array	oui	
total	integer	oui	

ConversationAccess

Champ	Type	Requis	Description
canRead	boolean	oui	
canWrite	boolean	oui	
reason	string null	oui	Pourquoi l'écriture est fermée (clé stable, traduite par le client)
writeClosesAt	string null	oui	

ConversationListResponse

Champ	Type	Requis	Description
items	array	oui	
totalUnread	integer	oui	

ConversationSummary

Champ	Type	Requis	Description
id	ObjectId	oui	
bookingId	ObjectId	oui	
role	enum : SHIPPER, CARRIER	oui	Le rôle de CELUI qui lit
counterpart	object	oui	
corridor	object	oui	
bookingStatus	string	oui	
lastMessage	object null	oui	
unreadCount	integer	oui	
nextMeetup	Meetup null	oui	
access	ConversationAccess	oui	

ConversationThreadResponse

Champ	Type	Requis	Description
conversation	ConversationSummary	oui	
messages	array	oui	Du plus ancien au plus récent
meetups	array	oui	
nextCursor	ObjectId null	oui	Messages plus ANCIENS à charger
phone	object	oui	D61 4A — révélé au plus tôt 2 h avant le rendez-vous

ErrorResponse — Enveloppe d'erreur standard (error-middleware, toute AppError)

Champ	Type	Requis	Description
status	string	oui	
message	string	oui	
errors	object	non	Erreurs par champ (formulaires) — exposé quand details.errors est présent
details		non	Contexte structuré 'safe' (ex: type=otp) — toujours exposé ; le reste hors prod uniquement

Meetup

Champ	Type	Requis	Description
id	ObjectId	oui	
kind	MeetupKind	oui	
status	MeetupStatus	oui	
proposedByRole	enum : SHIPPER, CARRIER	oui	
placeLabel	string	oui	
placeDetails	string null	oui	
startAt	string (date-time)	oui	
endAt	string (date-time)	oui	
acceptedAt	string null	oui	
cancelledAt	string null	oui	
createdAt	string (date-time)	oui	

MeetupKind — enum : PICKUP, DELIVERY

MeetupStatus — enum : PROPOSED, ACCEPTED, CANCELLED

Message

Champ	Type	Requis	Description
id	ObjectId	oui	
kind	MessageKind	oui	
authorRole	MessageAuthorRole	oui	
authorId	ObjectId null	oui	
body	string	oui	Texte de l'auteur, ou libellé de repli d'un message système
photoUrls	array	oui	
systemKey	string null	oui	Clé i18n d'un message SYSTEM / MEETUP — le client affiche sa propre copie
systemData	object null	oui	
flaggedContact	boolean	oui	D61 5A — coordonnées détectées : averti, jamais bloqué
createdAt	string (date-time)	oui	

MessageAuthorRole — enum : SHIPPER , CARRIER , SYSTEM

MessageKind — enum : TEXT , SYSTEM , MEETUP

MessageReportReason — enum : OFF_PLATFORM , SCAM , HARASSMENT , OTHER

MessageReportStatus — enum : OPEN , REVIEWED , DISMISSED

ObjectId — string : Identifiant MongoDB (ObjectId sérialisé en hexadécimal)

PostMessageRequest

Champ	Type	Requis	Description
body	string	oui	
photoUrls	array	non	

ProposeMeetupRequest — Proposition de rendez-vous ; une contre-proposition remplace la précédente

Champ	Type	Requis	Description
kind	MeetupKind	oui	
placeLabel	string	oui	
placeDetails	string	non	
startAt	string (date-time)	oui	
endAt	string (date-time)	oui	

QuickRepliesResponse

Champ	Type	Requis	Description
items	array	oui	

QuickReply — D61 2A — réponses rapides traduites, servies dans la langue du lecteur

Champ	Type	Requis	Description
key	string	oui	
kind	MeetupKind null	oui	
text	string	oui	

ReportMessageRequest

Champ	Type	Requis	Description
reason	MessageReportReason	oui	
details	string	non	

ReportMessageResponse

Champ	Type	Requis	Description
reportId	ObjectId	oui	
createdAt	string (date-time)	oui	

RevealPhoneResponse

Champ	Type	Requis	Description
phoneE164	string null	oui	
firstName	string	oui	
revealedAt	string (date-time)	oui	

ReviewMessageReportRequest

Champ	Type	Requis	Description
decision	enum : REVIEWED, DISMISSED	oui	
note	string	non	

UnauthorizedResponse — 401 du middleware isAuthenticated (token absent, invalide, expiré, ou compte introuvable) — hors error-middleware

Champ	Type	Requis	Description
message	string	oui	

UnhandledError — 500 non géré (exception hors AppError) — champ error, pas message

Champ	Type	Requis	Description
status	string	oui	
error	string	oui	

notification-service — port 6004

Notifications in-app, webhooks email. Document vivant : http://localhost:6004/docs (Scalar) et apps/notification-service/openapi.json. Via le gateway : http://localhost:8080/api (préfixes ci-dessous).

notification-service › —

GET /me/notifications

My notifications (latest first) with unread count

operationId : listMyNotifications · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	MyNotificationsResponse — Latest 50 notifications + unreadCount
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
500	UnhandledError — Unhandled server error

PATCH /me/notifications/read-all

Mark ALL my notifications as read (idempotent — A91)

operationId : markAllNotificationsRead · Authentication : cookieAuth, bearerAuth

Code	Réponse
200	MarkAllNotificationsReadResponse — How many were unread
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
500	UnhandledError — Unhandled server error

PATCH /me/notifications/{id}/read

Mark one notification as read (idempotent)

operationId : markNotificationRead · Authentication : cookieAuth, bearerAuth

Paramètre	Dans	Requis	Type	Description
id	path	oui	ObjectId	Notification id

Code	Réponse
200	MarkNotificationReadResponse — The updated notification
400	ErrorResponse — Malformed notification id (ValidationError)
401	UnauthorizedResponse — Missing or invalid token (isAuthenticated middleware)
403	ErrorResponse — Authenticated but not the recipient (ForbiddenError — A21)
404	ErrorResponse — Notification not found (NotFoundError)
500	UnhandledError — Unhandled server error

notification-service › schémas utilisés (10)

BookingEventType — enum : booking.requested, booking.payment_authorized, booking.accepted, booking.declined, booking.expired, booking.cancelled, booking.refund_issued, booking.picked_up, booking.pickup_refused, booking.tracking_event, booking.code_regenerated, booking.delivered, booking.completed, booking.payout_sent, booking.disputed, booking.verification_reminder, booking.rating_reminder, booking.rating_revealed, booking.dispute_carrier_responded, booking.dispute_resolved — All booking domain event keys (outbox → Kafka topics)

ErrorResponse — Enveloppe d'erreur standard (error-middleware, toute AppError)

Champ	Type	Requis	Description
status	string	oui	
message	string	oui	
errors	object	non	Erreurs par champ (formulaires) — exposé quand details.errors est présent
details		non	Contexte structuré 'safe' (ex: type=otp) — toujours exposé ; le reste hors prod uniquement

MarkAllNotificationsReadResponse — PATCH /me/notifications/read-all — how many were unread

Champ	Type	Requis	Description
updatedCount	integer	oui	

MarkNotificationReadResponse — PATCH /me/notifications/{id}/read — the updated notification

Champ	Type	Requis	Description
notification	NotificationView	oui	

MyNotificationsResponse — GET /me/notifications — latest first, unread count included

Champ	Type	Requis	Description
notifications	array	oui	
unreadCount	integer	oui	

NotificationType — union : BookingEventType | string

NotificationView — In-app notification (whitelist DTO — A13): userId/eventId never exposed

Champ	Type	Requis	Description
id	ObjectId	oui	
type	NotificationType	oui	
bookingId	ObjectId null	oui	
payload	object	oui	
counterpartFirstName	string null	oui	
readAt	string null	oui	
createdAt	string (date-time)	oui	

ObjectId — string : Identifiant MongoDB (ObjectId sérialisé en hexadécimal)

UnauthorizedResponse — 401 du middleware isAuthenticated (token absent, invalide, expiré, ou compte introuvable) — hors error-middleware

Champ	Type	Requis	Description
message	string	oui	

UnhandledError — 500 non géré (exception hors AppError) — champ error, pas message

Champ	Type	Requis	Description
status	string	oui	
error	string	oui	

8. Annexes

8.1 Schémas de sécurité (tels que déclarés dans les documents OpenAPI)

Schéma	Type	Où le jeton voyage	Émis par	Vérifié par	Portée
cookieAuth	apiKey / cookie access_token	cookie httpOnly, 15 min	/auth/login, /auth/google, /auth/refresh	isAuthenticated, isOptionallyAuthenticated	toutes les routes me
bearerAuth	http bearer (JWT)	Authorization: Bearer <access_token>	idem (le jeton n'est aujourd'hui lisible que dans le cookie)	idem	toutes les routes me
cookie refresh_token	cookie httpOnly, session ou 30 j	POST /auth/refresh, requireSudo (lecture du jti)	/auth/login ...	refreshAuthTokens + Redis (jti)	rotation, fenêtre sud

Schéma	Type	Où le jeton voyage	Émis par	Vérfié par	Portée
adminCookieAuth	apiKey / cookie admin_access_token	cookie httpOnly , 15 min ; JWT adm: true , amr: ["pwd", "totp"]	POST /auth/admin/totp/verify (ou enable)	isAdminAuthenticated puis requireAdminPermission	routes /admin/* (4 permission)
cookie admin_preauth	cookie httpOnly , 5 min	entre le mot de passe et le TOTP	POST /auth/admin/login	routes totp/*	étape 2 de la connexion
cookie admin_refresh_token	cookie httpOnly	POST /auth/admin/refresh	totp/verify	Redis admin_jti:	rotation admin
fenêtre sudo	clé Redis sudo:<userId>:<jti> , 15 min	aucune donnée client : liée à la session	POST /auth/me/sudo/verify	requireSudo	5 gestes sensibles
signature Stripe	en-tête stripe-signature sur le corps brut	—	Stripe	constructStripeWebhookEvent (deux secrets)	POST :6003/webhooks/s
signature Svix	svix-id , svix-timestamp , svix-signature	—	Resend	verifySvixSignature	POST /api/webhooks/en
jeton de suivi	segment d'URL /track/{token} (32 octets CSPRNG, base64url)	—	POST /deals/{id}/tracking-link	GET /track/{token} (404 si révoqué)	page destinataire
jeton d'invitation admin	corps de POST /auth/admin/invite/accept	—	POST /admin/admins/invite	acceptation	création d'un compte
verificationToken OTP	corps des routes register/* , password/*	—	POST /auth/register , /auth/password/verify	Redis (10 min)	inscription, réinitialisation

Toutes les routes publiques (recherche, trajet public, profil public, paramètres de prix, page destinataire, statut, maintenance, inscription, connexion) sont déclarées « Authentification : aucune » dans la référence.

8.2 Variables d'environnement côté client

Variable	Application	Rôle
NEXT_PUBLIC_API_BASE_URL	user-ui, admin-ui	Base des appels API : /api (même origine, D48, recommandé) ou http://<hôte>:8080/api . Le client axios tolère les deux formes.
API_PROXY_TARGET	user-ui, admin-ui (next.config.js)	Cible du proxy Next /api/* → gateway (http://localhost:8080). Sans elle, comportement historique (URL absolue).
NEXT_PUBLIC_SERVER_URI	user-ui	Adresse historique du serveur (héritage, à ne plus utiliser pour de nouveaux appels).
NEXT_PUBLIC_GOOGLE_MAPS_API_KEY	user-ui	Autocomplétion Google Places (adresses des trajets et alertes).
NEXT_PUBLIC_SENTRY_DSN	user-ui, admin-ui	Sentry front (instrumentation-client.ts). Inerte si absent.
NEXT_PUBLIC_POSTHOG_KEY , NEXT_PUBLIC_POSTHOG_HOST	user-ui	PostHog, chargé après acceptation du bandeau de consentement seulement.
NEXT_PUBLIC_SUPPORT_EMAIL	user-ui	Adresse affichée dans les pages d'erreur et de sanction.
NEXT_PUBLIC_USER_UI_LINK	admin-ui	Lien vers le front membre depuis le back-office.
allowedDevOrigins (next.config.js)	user-ui	Requis pour tester depuis une IP LAN avec Next 16 (sinon 403 sur /_next/*).

Côté serveur, les variables qui **changent le comportement observable par un client** : MAINTENANCE_MODE , MAINTENANCE_MESSAGE_FR/EN (503), STRIPE_WEBHOOK_SECRET , STRIPE_CONNECT_WEBHOOK_SECRET (501 sinon), RESEND_WEBHOOK_SECRET (503 sinon), SESSION_INACTIVITY_TIMEOUT_MINUTES , SESSION_STANDARD_LIFETIME_DAYS , SESSION_REMEMBER_INACTIVITY_DAYS , SESSION_ABSOLUTE_LIFETIME_DAYS (politique de session), IMAGEKIT_PUBLIC_KEY , IMAGEKIT_URL_ENDPOINT (absents → publicKey / urlEndpoint manquent dans la signature), KAFKA_BROKERS , OUTBOX_RELAY_ENABLED , NOTIFICATION_CONSUMER_ENABLED (pipeline d'événements), APP_VERSION / GIT_SHA (champ version des /health).

8.3 Glossaire

Terme	Définition
Expéditeur (shipper)	Membre qui réserve l'acheminement d'un colis. Vue ShipperBookingView .
Voyageur (carrier, « Yamber » / « Tripper » dans l'interface)	Membre qui publie un trajet et transporte ; rôle CARRIER obtenu à la fin de l'onboarding. Vue CarrierBookingView .
Destinataire (recipient)	Tiers sans compte qui reçoit le colis ; connu par un instantané dans le deal, informé par le lien de suivi.
Trajet (trip)	Annonce d'un Voyageur : corridor, dates, capacité en kg, prix au kilo, catégories, points de récupération et de remise. Statuts DRAFT , PUBLISHED , PAUSED , COMPLETED , CANCELLED , ARCHIVED .
Deal (booking)	La réservation d'un colis sur un trajet ; machine à 9 statuts ; cinq instantanés figés.
Devis (quote)	Calcul du prix par le moteur unique (@packages/pricing , D34), recalculé côté serveur et figé dans le deal (D17).
Intention de paiement	Autorisation du montant chez le fournisseur avant la demande (capture manuelle à l'acceptation, D31/D37).
Code de livraison	Six chiffres générés à la récupération, donnés de vive voix par l'Expéditeur au destinataire, saisis par le Voyageur à la remise (D43).
Fenêtre de vérification	4 jours après la remise pendant lesquels l'Expéditeur confirme ou conteste ; à l'échéance, confirmation automatique et versement.
Versement (payout)	Transfert Stripe Connect du net Voyageur (transportCents) ; statuts PENDING , SENT , FAILED , FROZEN ...
Litige (dispute)	Contestation de l'Expéditeur, ticket YAM-XXXX , versement gelé, décision par un médiateur.
Double aveugle	Notation révélée quand les deux parties ont noté, ou à 14 jours.

Terme	Définition
Conversation / Rendez-vous / Révélation du numéro	Objets du message-service (D61) : un fil par deal dès l'acceptation, un rendez-vous comme objet, un numéro ouvert au plus tôt 2 h avant.
Fenêtre sudo	15 minutes de « présence prouvée » liées à la session, exigées avant un geste sensible (D65).
Outbox / relais / topic	Événement écrit dans la transaction du changement d'état, relayé vers Kafka (booking-events , messaging-events) et consommé avec dédoublement (D2).
Instantané (snapshot)	Copie figée d'une donnée (prix, trajet, destinataire) dans le deal, jamais recalculée depuis la source.
allowedActions	Liste, calculée par le serveur, des transitions permises à l'appelant sur un trajet ou un deal ; seule source des boutons du client.
Porte 🚪	Point de décision volontairement laissé ouvert ou à durcir, tracé dans le registre.
Profils admin	SUPER_ADMIN , MEDIATOR , SUPPORT , FINANCE , OPS , PRIVACY ; cumulables (union des permissions).
Compte restreint / suspendu	Sanctions RESTRICTED (plus de création) et SUSPENDED (plus de session), appliquées par les lectures seulement.
Liste de suppression	Adresses email marquées après rebond dur ou plainte ; plus aucun envoi.

8.4 Recommandations d'expert (hors périmètre livré)

Cette section est **clairement séparée** de la description de l'existant : rien de ce qui suit n'est implémenté, et chaque point devrait, s'il est retenu, faire l'objet d'une entrée du registre des décisions avant tout code.

- Jetons dans le corps pour le mobile** — livrer la variante gravée par D36 : `POST /auth/login` et `POST /auth/refresh` acceptant `Accept: application/json` avec { `accessToken`, `refreshToken`, `expiresIn` } et un `refresh_token` accepté dans le corps, en gardant les cookies pour le web. Prérequis du jalon 4.
- Liste « safe » des details** — remplacer la liste blanche par type par une **classe d'erreur typée** (`BusinessError` avec `code` toujours exposé, `debug` jamais exposé) : la porte `SUDO_REQUIRED` / message-service en production disparaît structurellement.
- Sémantique 403/404 du trip-service** — la PR `fix/error-semantics` cataloguée ; un client générique ne peut pas distinguer aujourd'hui une saisie invalide d'un trajet introuvable.
- Versionnement d'API explicite** — un préfixe `/api/v1` au gateway (réécriture transparente vers les services) et une politique écrite : champs optionnels seulement en mineur, dépréciation annoncée par un en-tête `Deprecation / Sunset`, suppression en majeur. Coût nul aujourd'hui, très cher à introduire après un client mobile publié.
- Document OpenAPI unifié** — un `openapi.json` agrégé au gateway (fusion des cinq documents, chemins préfixés `/api`, un seul espace de noms de schémas — déjà le cas par le registre commun A22), servi sur `/api/docs`. C'est la source naturelle d'un **SDK généré** (orval / openapi-generator, D3) pour TypeScript, Kotlin et Swift, avec les types de `details.code` exposés comme unions littérales.
- ETag / If-None-Match sur les lectures chaudes** (`GET /deals/{id}`, `GET /messages/conversations/{id}`, `GET /me/notifications`) et **If-Match sur les transitions** (à partir de `updatedAt`) : le verrou optimiste existe déjà côté serveur ; l'exposer par en-tête standardise le « 409 = relire ».
- Clés d'idempotence client** (`Idempotency-Key`) sur `POST /deals`, `POST /deals/{id}/pickup`, `/deliver`, `/dispute` et `POST /messages/.../messages` : le rejeu réseau d'un mobile en zone blanche ne créerait jamais de doublon de message ni de tentative de code comptée deux fois.
- Limiteur conscient de la session** — décoder le JWT au gateway (sans le vérifier en base) pour appliquer la limite de 1 000 prévue aux membres authentifiés et une limite stricte par jeton de suivi sur `/api/track/*`.
- Webhooks sortants pour partenaires et clés d'API partenaires** (par exemple un transitaire ou une boutique qui crée des demandes pour ses clients) : un `ApiKey` haché avec portées (`deals:read`, `deals:write`), un abonnement aux événements `booking.*` filtrés et re-signés (HMAC, retry exponentiel, `event-id` pour le dédoublement) — les payloads existants sont déjà conçus pour cela (riches, sans secret).
- Temps réel** — un canal SSE ou WebSocket au gateway (`/api/stream`) alimenté par les mêmes consommateurs Kafka, pour le fil de conversation et le compteur de non-lus, à la place du polling.
- Journal des refus** — tracer côté serveur les 403 `SUDO_REQUIRED`, `ACCOUNT_RESTRICTED` et les 409 par code (compteurs Redis) pour que la page de pilotage montre où les utilisateurs butent.
- Route /docs du message-service et test de couverture des routes** (comme A145) étendu aux quatre autres services : la référence générée listerait alors aussi les exports CSV admin montés dans les routeurs (`/admin/disputes/export`, `/admin/trips/export`, `/admin/tickets/export`) qui n'apparaissent pas aujourd'hui dans les documents OpenAPI. 🚪
- Contrat de rétention exposé** — publier dans `GET /trips/pricing/params` (ou un `GET /platform/params`) les durées que les clients affichent (fenêtre d'acceptation 24 h, vérification 4 j, notation 14 j, écriture du fil 14 j, ouverture du numéro 2 h) : elles sont réglables par l'admin (D62) et ne devraient pas être codées en dur dans les fronts.

Fin du document. Les chapitres 1 à 6 et 8 sont rédigés à la main à partir du code de la branche courante ; le chapitre 7 est généré. Toute divergence constatée entre ce document et le comportement observé doit être traitée comme un défaut du document ou du code, jamais comme une interprétation.